



ΠΑΝΕΠΙΣΤΗΜΙΟ  
ΠΑΤΡΩΝ  
UNIVERSITY OF PATRAS

Πολυτεχνική Σχολή  
Τμήμα Μηχανικών Η/Υ & Πληροφορικής

Διπλωματική Εργασία

---

**Δυναμική και Ανθεκτική σε  
Σφάλματα Συνεργατική Εκτέλεση  
Μεγάλων Γλωσσικών Μοντέλων σε  
Ετερογενείς Συσκευές**

---

Γρηγόριος Δελημπαλταδάκης  
Α.Μ. 1084647

Επιβλέπων  
Παναγιώτης Χατζηδούκας  
Αναπληρωτής Καθηγητής  
Πάτρα, 2026

© Γρηγόριος Δελημπαλαδάκης, 2026

Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Η έγκριση της διπλωματικής εργασίας από το Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών & Πληροφορικής του Πανεπιστημίου Πατρών δεν υποδηλώνει απαραίτητως και αποδοχή των απόψεων του συγγραφέα εκ μέρους του Τμήματος.

*Στην οικογένεια και στους φίλους μου*

# Περίληψη

Η εκτέλεση Μεγάλων Γλωσσικών Μοντέλων (Large Language Models - LLMs) περιορίζεται συχνά από υψηλές απαιτήσεις μνήμης και υπολογιστικής ισχύος. Η παρούσα διπλωματική εργασία παρουσιάζει ένα δυναμικό, ανθεκτικό σε σφάλματα σύστημα για τη συνεργατική εκτέλεση LLMs, με αποκεντρωμένη διαχείριση, σε ένα τοπικό δίκτυο ετερογενών συσκευών. Αξιοποιώντας την τεχνική του παραλληλισμού διοχέτευσης (pipeline parallelism), το σύστημα κατανέμει τα επίπεδα μετασχηματιστών (transformer layers) και το τελικό επίπεδο εξόδου (output layer) ενός decoder-only LLM σε ένα πλήθος από κόμβους – εργάτες.

Ο συντονισμός του συστήματος πραγματοποιείται μέσω ενός Κατανεμημένου Πίνακα Κατακερματισμού (Distributed Hash Table - DHT), εξαλείφοντας την ανάγκη για έναν κεντρικό κόμβο-συντονιστή και αποτρέποντας τη δημιουργία μοναδικών σημείων αποτυχίας. Η αρχιτεκτονική του συστήματος εξασφαλίζει αυτοματοποιημένη εξισορρόπηση φορτίου, μέσω της Δυναμικής Ανακατανομής των επιπέδων του μοντέλου (Dynamic Layer Reallocation), ενώ παράλληλα ενσωματώνει μηχανισμούς Ευκαιριακής Ανάληψης (opportunistic takeover), για την βελτίωση της απόδοσης του συστήματος και αυτόματης Επισκευής της Αλυσίδας εκτέλεσης σε περίπτωση αποτυχίας, διασφαλίζοντας έτσι την ανθεκτικότητα σε σφάλματα (fault tolerance).

Για το μετριασμό της καθυστέρησης δικτύου κατά τη μετάδοση των ενδιάμεσων ενεργοποιήσεων, εφαρμόζεται 8-bit dynamic block-wise quantization. Συμπληρωματικά, το σύστημα υποστηρίζει την ταυτόχρονη παραγωγή κειμένου για πολλαπλούς χρήστες, αξιοποιώντας εναλλασόμενες Key-Value (KV) caches.

Η πειραματική αξιολόγηση καταδεικνύει ότι η Δυναμική Ανακατανομή Επιπέδων σχεδόν διπλασιάζει το throughput (11.63 TPS) σε σύγκριση με μια άπληστη κατανομή επιπέδων (6.21 TPS), ενώ η κβαντοποίηση 8-bit μειώνει το ωφέλιμο φορτίο δικτύου κατά 50% εξασφαλίζοντας υψηλότερη απόδοση σε ταχύτερες μικρότερες των 30 Mbps.

Το προκύπτον σύστημα επιτρέπει την συνεχή και αποδοτική εκτέλεση γλωσσικών μοντέλων που υπερβαίνουν τις δυνατότητες των επιμέρους συσκευών, διατηρώντας σταθερότητα παρά την τυχόν αποτυχία μεμονωμένων κόμβων. Συγκεκριμένα το σύστημα ανακάμπτει από αστοχίες κόμβων σε 15-30 δευτερόλεπτα και αποτρέπει υπολογιστικές συμφορήσεις μέσω της Ευκαιριακής Ανάληψης εντός ~3 δευτερολέπτων.

**Λέξεις-κλειδιά:** Μεγάλα Γλωσσικά Μοντέλα (LLMs), Κατανεμημένη Εκτέλεση, Αποκεντρωμένα Συστήματα, Παραλληλισμός Διοχέτευσης, Ανθεκτικότητα σε Σφάλματα, Δυναμική Εξισορρόπηση Φορτίου, Κατανεμημένος Πίνακας Κατακερματισμού (DHT), Ετερογενή Συστήματα

# Abstract

The deployment of Large Language Models (LLMs) is frequently constrained by high memory and computational requirements. This thesis presents a dynamic, fault-tolerant system that enables collaborative LLM inference with decentralized management, across a local network of heterogeneous devices. Leveraging pipeline parallelism, the system distributes the transformer layers and the final output layer of a decoder-only LLM across multiple worker nodes.

System coordination is managed by a Distributed Hash Table (DHT), eliminating the need for a centralized coordinator node and preventing single points of failure. The system's architecture ensures automatic load balancing through the Dynamic Reallocation of model layers, while simultaneously integrating mechanisms for Opportunistic Takeover, to improve the system's performance and automatic Repair of the inference chain in the event of a node failure.

To mitigate network latency during the transmission of intermediate activations, 8-bit dynamic block-wise quantization is applied. Additionally, the system supports concurrent text generation for multiple users by utilizing rotating Key-Value (KV) caches.

The experimental evaluation demonstrates that Dynamic Layer Reallocation nearly doubles the throughput (11.63 TPS) compared to a static greedy allocation (6.21 TPS), while 8-bit quantization reduces the network payload by 50%, ensuring higher performance at speeds below 30 Mbps.

The resulting system allows for the continuous and efficient execution of models that exceed the capabilities of individual devices, maintaining stability despite the potential failure of individual nodes. Specifically, the system recovers from node failures in 15 to 30 seconds and prevents computational bottlenecks through Opportunistic Takeover within approximately 3 seconds.

**Keywords:** Large Language Models (LLMs), Distributed Inference, Decentralized Systems, Pipeline Parallelism, Fault Tolerance, Dynamic Load Balancing, Distributed Hash Table (DHT), Heterogeneous Systems

# Περιεχόμενα

|                                                       |     |
|-------------------------------------------------------|-----|
| Περίληψη.....                                         | i   |
| Abstract.....                                         | ii  |
| Λίστα Εξισώσεων.....                                  | vi  |
| Λίστα Σχημάτων και Εικόνων.....                       | vii |
| Λίστα Πινάκων.....                                    | x   |
| Συντομογραφίες.....                                   | xi  |
| Απόδοση Όρων.....                                     | xii |
| 1 Εισαγωγή.....                                       | 14  |
| 1.1 Κίνητρο.....                                      | 14  |
| 1.2 Ερευνητικό Πρόβλημα και Ερευνητικά Ερωτήματα..... | 14  |
| 1.3 Στόχοι και Συνεισφορές.....                       | 16  |
| 1.4 Δομή της Διπλωματικής Εργασίας.....               | 17  |
| 2 Θεωρητικό Υπόβαθρο.....                             | 19  |
| 2.1 Large Language Models.....                        | 19  |
| 2.1.1 Transformer.....                                | 19  |
| 2.1.2 Self-Attention.....                             | 20  |
| 2.1.3 Multi – Head Attention.....                     | 22  |
| 2.1.4 Autoregressive Decoder-Only Transformers.....   | 23  |
| 2.2 Llama 3.....                                      | 24  |
| 2.2.1 Input Layer.....                                | 25  |
| 2.2.2 Transformer Layer.....                          | 25  |
| 2.2.3 Output Layer.....                               | 30  |
| 2.3 Distributed Hash Table (DHT).....                 | 31  |
| 2.4 Remote Procedure Calling (RPC).....               | 31  |
| 3 Βιβλιογραφική Ανασκόπηση.....                       | 33  |
| 3.1 Single-Node Βελτιστοποιήσεις Συμπερασμού.....     | 33  |
| 3.1.1 Αρχιτεκτονικές Βελτιστοποιήσεις.....            | 33  |
| 3.1.2 Αλγοριθμικές Βελτιστοποιήσεις.....              | 33  |
| 3.2 Τεχνικές Παραλληλοποίησης.....                    | 34  |
| 3.2.1 Tensor Parallelism (TP).....                    | 35  |
| 3.2.2 Pipeline Parallelism (PP).....                  | 36  |

|       |                                                              |    |
|-------|--------------------------------------------------------------|----|
| 3.3   | Αρχιτεκτονικές Κατανεμημένου Συμπερασμού .....               | 36 |
| 3.3.1 | Edge-Cloud Split Computing .....                             | 36 |
| 3.3.2 | Κατανεμημένος Συμπερασμός Άκρου σε Τοπικά Δίκτυα.....        | 37 |
| 3.3.3 | Ετερογενείς και Εθελοντικές GPUs .....                       | 37 |
| 3.3.4 | Θεμελιώδης Έμπνευση Αυτόνομου Κατανεμημένου Συμπερασμού..... | 39 |
| 3.4   | Συγκριτική Αποτίμηση Προσεγγίσεων και Ερευνητικό Κενό .....  | 41 |
| 4     | Αρχιτεκτονική Συστήματος .....                               | 44 |
| 4.1   | Επισκόπηση Αρχιτεκτονικής.....                               | 44 |
| 4.2   | Αρχιτεκτονική Server Node .....                              | 45 |
| 4.2.1 | Διαχείριση Κατανεμημένης Κατάστασης και Τοπολογίας .....     | 45 |
| 4.2.2 | Δυναμική Αξιολόγηση Πόρων .....                              | 46 |
| 4.2.3 | Μηχανή Συμπερασμού .....                                     | 46 |
| 4.2.4 | Ανοχή Σφαλμάτων και Ανακατανομή Επιπέδων .....               | 46 |
| 4.3   | Αρχιτεκτονική Client Node.....                               | 47 |
| 4.3.1 | Τοπική Επεξεργασία και Δρομολόγηση Συμπερασμού .....         | 47 |
| 4.3.2 | Διαχείριση Κατάστασης .....                                  | 48 |
| 4.3.3 | Εποπτεία και Επιδιόρθωση Head Server .....                   | 48 |
| 4.4   | Διαχείριση Τοπολογίας - DHT .....                            | 48 |
| 4.4.1 | Keys-Values .....                                            | 50 |
| 4.4.2 | Εισαγωγή Νέων DHT Κόμβων .....                               | 50 |
| 4.4.3 | Ανθεκτικότητα σε Σφάλματα .....                              | 51 |
| 5     | Προσαρμοσμένη Υλοποίηση και Κατανεμημένη Εκτέλεση .....      | 52 |
| 5.1   | Προσαρμοσμένη Υλοποίηση Llama3 .....                         | 52 |
| 5.1.1 | Κατάτμηση Μοντέλου και Δυναμική Φόρτωση .....                | 52 |
| 5.1.2 | Τμηματοποιημένη Εκτέλεση .....                               | 52 |
| 5.1.3 | Διαχείριση KV Cache Πολλαπλών Χρηστών .....                  | 52 |
| 5.2   | LLM Loader API.....                                          | 53 |
| 5.2.1 | Χρήση από Server Nodes .....                                 | 53 |
| 5.2.2 | Χρήση από Client Nodes.....                                  | 54 |
| 5.3   | Κατανεμημένη Εκτέλεση Μοντέλου.....                          | 54 |
| 5.4   | Βελτιστοποίηση Δικτύου (8-bit Blockwise Quantization).....   | 55 |
| 5.5   | Διεπαφή Χρήστη (Client GUI).....                             | 58 |
| 6     | Δυναμική Εξισορρόπηση και Ανθεκτικότητα σε Σφάλματα .....    | 60 |
| 6.1   | Ανάλυση Απόδοσης Κόμβων.....                                 | 60 |

|       |                                                          |    |
|-------|----------------------------------------------------------|----|
| 6.1.1 | Ρυθμός Επεξεργασίας και η μετρική TLE.....               | 60 |
| 6.1.2 | Server Node Profiling.....                               | 60 |
| 6.2   | Δυναμική Ανακατανομή Φορτίου.....                        | 61 |
| 6.2.1 | Δυναμική Ανακατανομή Επιπέδων .....                      | 61 |
| 6.2.2 | Ευκαιριακή Ανάλυση.....                                  | 62 |
| 6.3   | Μηχανισμοί Ανθεκτικότητας σε Σφάλματα.....               | 64 |
| 6.3.1 | Ανίχνευση Σφαλμάτων .....                                | 64 |
| 6.3.2 | Επιδιόρθωση Αποτυχίας Διαδόχου από Server Nodes.....     | 64 |
| 6.3.3 | Αντικατάσταση Head Server από Client Node .....          | 66 |
| 7     | Πειραματική Αξιολόγηση.....                              | 67 |
| 7.1   | Πειραματική Μεθοδολογία και Περιβάλλον Αξιολόγησης ..... | 67 |
| 7.1.1 | Υπολογιστικό Περιβάλλον .....                            | 67 |
| 7.1.2 | Μοντέλα και Πειραματικές Διαμορφώσεις.....               | 68 |
| 7.1.3 | Δικτυακές Συνθήκες .....                                 | 68 |
| 7.1.4 | Στρατηγικές Κατανομής και Baselines .....                | 69 |
| 7.1.5 | Μετρικές Αξιολόγησης.....                                | 70 |
| 7.2   | Αξιολόγηση υπό Ρεαλιστικές Συνθήκες Λειτουργίας.....     | 71 |
| 7.3   | Σύγκριση Στρατηγικών Κατανομής .....                     | 72 |
| 7.4   | Αποτελεσματικότητα της Κβαντοποίησης.....                | 78 |
| 7.4.1 | Ανάλυση Μεγέθους του Payload.....                        | 79 |
| 7.4.2 | Ανάλυση Μείωσης του Χρόνου Μετάδοσης.....                | 79 |
| 7.5   | Πολλαπλοί Clients.....                                   | 81 |
| 7.6   | Ευκαιριακή Ανάλυση και Επισκευή Αλυσίδας .....           | 83 |
| 7.6.1 | Αποτελεσματικότητα Ευκαιριακής Ανάλυσης .....            | 83 |
| 7.6.2 | Χρόνος Επισκευής Αλυσίδας .....                          | 84 |
| 7.7   | Σύνοψη Αποτελεσμάτων.....                                | 87 |
| 8     | Συμπεράσματα και Προοπτικές .....                        | 90 |
| 8.1   | Σύνοψη Συστήματος.....                                   | 90 |
| 8.2   | Περιορισμοί της Εργασίας .....                           | 90 |
| 8.3   | Μελλοντική Εργασία .....                                 | 92 |
|       | Βιβλιογραφία.....                                        | 93 |

# Λίστα Εξισώσεων

|                                                                                                                                |    |
|--------------------------------------------------------------------------------------------------------------------------------|----|
| Equation 1. Τύπος υπολογισμού του μητρώου των εξόδων του μηχανισμού attention. Πηγή: [7]                                       | 21 |
| Equation 2. Τύπος υπολογισμού του μητρώου εξόδου του multi-head attention. Πηγή: [7]...                                        | 23 |
| Equation 3. Μηχανισμός attention με εφαρμογή causal attention mask. ....                                                       | 24 |
| Equation 4. Κωδικοποίηση της απόλυτης θέσης εφαρμόζοντας έναν πίνακα περιστροφής στα διανύσματα query και key. Πηγή: [14]..... | 27 |
| Equation 5. Το similarity score μεταξύ query και key ενσωματώνοντας τη σχετική απόστασή τους (n-m). Πηγή: [14] .....           | 27 |
| Equation 6. Συνάρτηση ενεργοποίησης Swish.....                                                                                 | 30 |
| Equation 7. Μαθηματική διατύπωση του FFN με ενεργοποίηση SwiGLU. Πηγή: [19] .....                                              | 30 |
| Equation 8. Τύπος υπολογισμού του processing rate για έναν server node. ....                                                   | 60 |
| Equation 9. Τύπος υπολογισμού του αναλογικότερου αριθμού από transformer layers που αντιστοιχούν σε έναν ενεργό server. ....   | 62 |

# Λίστα Σχημάτων και Εικόνων

|                                                                                                                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1. Διάγραμμα αρχιτεκτονικής του Transformer. Πηγή: [7] .....                                                                                                                                                  | 19 |
| Figure 2 . Διάγραμμα του μηχανισμού Scaled Dot Product Attention. Πηγή: [7].....                                                                                                                                     | 21 |
| Figure 3. Διάγραμμα του μηχανισμού Multi-Head Attention με $h$ παράλληλα attention heads. Πηγή: [7].....                                                                                                             | 22 |
| Figure 4. Διάγραμμα αρχιτεκτονικής του Llama 3 με $L$ transformer layers. Πηγή: [από τον συγγραφέα].....                                                                                                             | 24 |
| Figure 5. Διάγραμμα των βημάτων του input layer. Πηγή: [από τον συγγραφέα] .....                                                                                                                                     | 25 |
| Figure 6. Απεικόνιση των σταδίων prefill και decoding (autoregressive generation). Πηγή: [15] .....                                                                                                                  | 27 |
| Figure 7. Διαγραμματική απεικόνιση του prefill και decoding stage με kv caching. Στο decoding stage φαίνεται η επαναχρησιμοποίηση των μητρώων $K$ και $V$ από το προηγούμενο στάδιο. Πηγή: [από τον συγγραφέα] ..... | 28 |
| Figure 8. Διαγραμματική επισκόπηση των μεθόδων MHA, GQA και MQA. Πηγή: [18] .....                                                                                                                                    | 29 |
| Figure 9. GQA στο Llama 3 - 8B με 32 query heads και 8 key-value heads. Πηγή: [από τον συγγραφέα].....                                                                                                               | 29 |
| Figure 10. Διάγραμμα αρχιτεκτονικής του FFN με ενεργοποίηση SwiGLU. Πηγή: [από τον συγγραφέα].....                                                                                                                   | 30 |
| Figure 11. Διάγραμμα της αρχιτεκτονικής του output layer με το παράδειγμα της πρόβλεψης του επόμενου token της ακολουθίας: “The capital of France is”. Πηγή: [από τον συγγραφέα] .....                               | 31 |
| Figure 12. Διάγραμμα απεικόνισης του tensor και pipeline parallelism με δύο GPUs. Πηγή: [33] .....                                                                                                                   | 35 |
| Figure 13. Recurrent pipeline parallelism του MDI-LLM framework. Πηγή: [46].....                                                                                                                                     | 40 |
| Figure 14. Παράδειγμα τοπολογίας του συστήματος με δυο client nodes, τρεις active server nodes και τρεις Backup nodes. ....                                                                                          | 44 |
| Figure 15. Διάγραμμα λογικής αρχιτεκτονικής ενός server node. ....                                                                                                                                                   | 45 |
| Figure 16. Διάγραμμα λογικής αρχιτεκτονικής ενός client node. ....                                                                                                                                                   | 47 |
| Figure 17. Παράδειγμα ανάθεσης επιπέδων του μοντέλου Llama 3.2 1B. ....                                                                                                                                              | 53 |
| Figure 18. Τοπολογία κατανεμημένης εκτέλεσης του μοντέλου Llama 3.2 1B, με δύο server nodes.....                                                                                                                     | 54 |
| Figure 19. Εκτέλεση αιτημάτων gRPC κατά την διαδικασία του κατανεμημένου συμπερασμού. ....                                                                                                                           | 54 |
| Figure 20. Παράδειγμα κβαντοποίησης με την τεχνική 8-bit dynamic block-wise quantization. Πηγή: [από τον συγγραφέα] .....                                                                                            | 56 |
| Figure 21. Αποκβαντοποίηση με την τεχνική 8-bit dynamic block-wise quantization. Πηγή: [από τον συγγραφέα] .....                                                                                                     | 57 |

|                                                                                                                                                                                        |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 22. Στιγμιότυπο οθόνης από την διεπαφή χρήστη. ....                                                                                                                             | 58 |
| Figure 23. Στιγμιότυπο οθόνης από την διεπαφή χρήστη. Ανάπτυξη της καρτέλας του server “worker-B”. ....                                                                                | 59 |
| Figure 24. Παράδειγμα μη εξισορροπημένης αλυσίδας εκτέλεσης πριν την εφαρμογή του μηχανισμού της Δυναμικής Ανακατανομής Επιπέδων. ....                                                 | 61 |
| Figure 25. Εξισορροπημένη αλυσίδα εκτέλεσης μετά την εφαρμογή Δυναμικής Ανακατανομής Επιπέδων.....                                                                                     | 62 |
| Figure 26. Τοπολογία της ενεργής αλυσίδας πριν (πάνω) και μετά (κάτω) την εφαρμογή Ευκαιριακής Ανάληψης από τον εφεδρικό κόμβο server-D. ....                                          | 63 |
| Figure 27. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανάληψης των επιπέδων του από τον server-A. ....                           | 65 |
| Figure 28. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανάληψης των επιπέδων του από τον εφεδρικό κόμβο server-D. ....            | 65 |
| Figure 29. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανακατανομής επιπέδων. ....                                                | 66 |
| Figure 30. Συνολικός αριθμός παραγόμενων tokens συναρτήσει του χρόνου για τις στρατηγικές Single (CPU/GPU) Node, Greedy, Balanced. ....                                                | 74 |
| Figure 31. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για τις στρατηγικές Single CPU Node (αριστερά) και Single GPU Node (δεξιά) allocation. ....                                      | 76 |
| Figure 32. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για την στρατηγική Greedy allocation. ....                                                                                       | 77 |
| Figure 33. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για την στρατηγική Balanced Allocation. ....                                                                                     | 77 |
| Figure 34. Κατανομή των μέσων καθυστερήσεων ανά κόμβο σε αλυσίδα εκτέλεσης με 9 server nodes με περιορισμένο bandwidth (5 Mbps). ....                                                  | 78 |
| Figure 35. Σύγκριση μέσης καθυστέρησης δικτύου συναρτήσει του bandwidth (1-1000 Mbps) με και χωρίς κβαντοποίηση για τα μοντέλα Llama 3.2 1B, 3B και Llama 3.1 8B. ....                 | 80 |
| Figure 36. Επίδραση της κβαντοποίησης στο throughput υπό συνθήκες απρόβλεπτης πτώσης της ταχύτητας δικτύου στα 10 Mbps. ....                                                           | 81 |
| Figure 37. Χρονοδιάγραμμα παραγωγής των συνολικών tokens του συστήματος κατά την ταυτόχρονη εξυπηρέτηση 1 έως 3 clients από αλυσίδα 3 server nodes. ....                               | 82 |
| Figure 38. Throughput του συστήματος κατά την ταυτόχρονη εξυπηρέτηση 1 έως 5 clients από αλυσίδα 3 server nodes. ....                                                                  | 82 |
| Figure 39. Επίδραση του μηχανισμού της Ευκαιριακής Ανάληψης στο throughput κατά την παραγωγή 250 tokens, με χρόνο αντικατάστασης του αδύναμου κόμβου στα 2.64 δευτερόλεπτα. ....       | 83 |
| Figure 40. Σύγκριση εξέλιξης της παραγωγής tokens κατά την αποτυχία του worker-B (στα 100 tokens) και την επισκευή της αλυσίδας χρησιμοποιώντας εφεδρικό κόμβο έναντι προκατόχου. .... | 85 |

Figure 41. Εξέλιξη παραγωγής tokens κατά την επιδιόρθωση αλυσίδας για το μοντέλο Llama 3.1 8B, αναδεικνύοντας τον αυξημένο χρόνο επισκευής (29.84 δευτερόλεπτα) λόγω του μεγέθους των παραμέτρων. .... 86

# Λίστα Πινάκων

|                                                                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Table 1. Συγκριτική αποτίμηση βασικών προσεγγίσεων κατανεμημένης εκτέλεσης LLMs...                                                                           | 42 |
| Table 2. DHT Global Keys.....                                                                                                                                | 48 |
| Table 3. DHT Server Node Subkeys. ....                                                                                                                       | 49 |
| Table 6. Ανάθεση επιπέδων του Llama 3.2 3B για τις στρατηγικές ανάθεσης Single (CPU/GPU) Node, Greedy, Balanced.....                                         | 73 |
| Table 7. Σύγκριση μετρικών Time-To-First-Token (TTFT), Throughput και Inter-Token Latency (ITL) για τις τέσσερις στρατηγικές.....                            | 73 |
| Table 8. Κατανάλωση μνήμης (RAM και VRAM) ανά κόμβο για τις διαφορετικές στρατηγικές ανάθεσης.....                                                           | 75 |
| Table 9. Μέγεθος ωφέλιμου φορτίου (payload size) ενδιάμεσων ενεργοποιήσεων ανά στάδιο παραγωγής με και χωρίς κβαντοποίηση 8-bit για τα μοντέλα Llama 3. .... | 79 |

# Συντομογραφίες

|      |                              |
|------|------------------------------|
| AI   | Artificial Intelligence      |
| CPU  | Central Processing Unit      |
| DHT  | Distributed Hash Table       |
| FFN  | FeedForward Network          |
| GPU  | Graphics Processing Unit     |
| GQA  | Grouped-Query Attention      |
| GUI  | Graphical User Interface     |
| ITL  | Inter-Token Latency          |
| KV   | Key-Value                    |
| LLM  | Large Language Model         |
| MHA  | Multi-Head Attention         |
| MQA  | Multi-Query Attention        |
| NLP  | Natural Language Processing  |
| OOM  | Out-Of-Memory                |
| P2P  | Peer-to-Peer                 |
| PP   | Pipeline Parallelism         |
| RAM  | Random Access Memory         |
| RoPE | Rotary Positional Embeddings |
| RPC  | Remote Procedure Calls       |
| TLE  | Transformer Layer Equivalent |
| TP   | Tensor Parallelism           |
| TPS  | Tokens Per Second            |
| TTFT | Time-To-First-Token          |
| VRAM | Video Random Access Memory   |

# Απόδοση Όρων

|                                                                      |                                                                                                                                                                                                                 |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ανθεκτικότητα σε Σφάλματα (Fault Tolerance)                          | Η ικανότητα του συστήματος να επιδιορθώνεται και να διατηρεί την σταθερότητά του σε περίπτωση αποτυχίας μεμονωμένων κόμβων.                                                                                     |
| Δυναμική Ανακατανομή (Dynamic Reallocation)                          | Η διαδικασία ανακατανομής των επιπέδων του μοντέλου που αναλαμβάνει κάθε κόμβος βάσει του ρυθμού επεξεργασίας του και των περιορισμών μνήμης του, με σκοπό την εξισορρόπηση της καθυστέρησης συμπερασμού.       |
| Ευκαιριακή Ανάλληψη (Opportunistic Takeover)                         | Ο μηχανισμός μέσω του οποίου ένας εφεδρικός ή ενεργός κόμβος υποκαθιστά έναν πιο αδύναμο κόμβο της αλυσίδας εκτέλεσης για την μεγιστοποίηση της απόδοσης του συστήματος.                                        |
| Καθυστέρηση Συμπερασμού (Inference Delay)                            | Ο χρόνος επεξεργασίας των επιπέδων του μοντέλου που έχει αναλάβει ένας ενεργός κόμβος.                                                                                                                          |
| Ρυθμός Επεξεργασίας (Processing Rate)                                | Ο αριθμός των transformer layers ενός κόμβου δια την καθυστέρηση συμπερασμού. Αποτελεί ένδειξη της υπολογιστικής ισχύος ενός κόμβου.                                                                            |
| Αλυσίδα Εκτέλεσης (Inference Chain)                                  | Η ακολουθία των κόμβων που έχουν φορτώσει τα επίπεδα του μοντέλου.                                                                                                                                              |
| Κόμβος (Node)                                                        | Μια συσκευή που είναι μέρος του συστήματος και του υφιστάμενου DHT.                                                                                                                                             |
| Ενεργός Κόμβος (Active Node)                                         | Ένας κόμβος που είναι μέλος της αλυσίδας εκτέλεσης.                                                                                                                                                             |
| Εφεδρικός Κόμβος (Backup Node)                                       | Ένας κόμβος που βρίσκεται σε αναμονή και είναι διαθέσιμος για τους μηχανισμούς Ανθεκτικότητας σε Σφάλματα και Ευκαιριακής Ανάλληψης.                                                                            |
| Κατανεμημένος Πίνακας Κατακερματισμού (Distributed Hash Table - DHT) | Αποκεντρωμένη δομή αποθήκευσης που μοιράζει τα δεδομένα σε μορφή κλειδιού-τιμής σε ένα δίκτυο πολλαπλών κόμβων, επιτρέποντας την εύρεση και ανάκτησή τους χωρίς την ανάγκη κάποιου κεντρικού κόμβου-συντονιστή. |
| Κβαντοποίηση 8-bit ανά Συστάδες (8-bit Block-wise Quantization)      | Η κβαντοποίηση σε μπλοκ συμπιέζει τις ενδιάμεσες ενεργοποιήσεις του μοντέλου σε μορφή χαμηλότερης ακρίβειας ανά ομάδες, απομονώνοντας τοπικά τις ακραίες τιμές.                                                 |
| Παραλληλισμός Διοχέτευσης (Pipeline Parallelism)                     | Η μέθοδος κατανομής του υπολογιστικού φόρτου, όπου τα επίπεδα ενός μοντέλου διαμοιράζονται και εκτελούνται σειριακά από πολλαπλούς κόμβους.                                                                     |

|                                            |                                                                                                                                                                                                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pipeline Bubbles                           | Περίοδοι αδράνειας κατά τις οποίες κόμβοι που βρίσκονται χαμηλότερα στην αλυσίδα εκτέλεσης περιμένουν τους προηγούμενους κόμβους να τελειώσουν την επεξεργασία τους, ή το αντίστροφο.                                                     |
| Επίπεδο Μετασχηματιστή (Transformer Layer) | Το βασικό επαναλαμβανόμενο δομικό στοιχείο ενός LLM που περιλαμβάνει τον μηχανισμό Self-Attention και ένα εμπρόσθιο δίκτυο τροφοδότησης (FFN).                                                                                            |
| Τελικό Επίπεδο Εξόδου (Final Output Layer) | Το τελικό επίπεδο ενός LLM που μετατρέπει τις εσωτερικές διανυσματικές αναπαραστάσεις του τελευταίου transformer layer σε σκορ πιθανοτήτων (logits) για κάθε token του λεξιλογίου, επιτρέποντας στο μοντέλο να επιλέξει το επόμενο token. |
| KV Caching                                 | Τεχνική μνήμης που αποθηκεύει τους προϋπολογισμένους πίνακες Key και Value των προηγούμενων tokens, ώστε να μην χρειάζεται να επανυπολογίζονται σε κάθε νέο βήμα παραγωγής κειμένου.                                                      |
| Token                                      | Η βασική μονάδα δεδομένων στην οποία ένα LLM χωρίζει το κείμενο.                                                                                                                                                                          |
| Logits                                     | Οι μη κανονικοποιημένες τιμές πρόβλεψης που εξάγονται από το τελικό επίπεδο εξόδου για όλα τα tokens του λεξιλογίου, πριν μετατραπούν σε κατανομή πιθανοτήτων μέσω της συνάρτησης softmax.                                                |
| Διεκπεραιωτική Ικανότητα (Throughput)      | Ο ρυθμός παραγωγής tokens του συστήματος.                                                                                                                                                                                                 |
| Prefill Stage                              | Το αρχικό στάδιο του συμπερασμού στην αρχή της εκτέλεσης, όπου αρχικοποιείται η KV cache.                                                                                                                                                 |
| Decoding Stage                             | Η φάση της αποκωδικοποίησης (ή παραγωγής) κατά τον συμπερασμό, όπου τα tokens του κειμένου παράγονται διαδοχικά αξιοποιώντας την KV Cache των προηγούμενων tokens.                                                                        |

# 1 Εισαγωγή

## 1.1 Κίνητρο

Η εξέλιξη των Μεγάλων Γλωσσικών Μοντέλων (Large Language Model - LLMs) έχει επιφέρει σημαντικές δυνατότητες στην επεξεργασία φυσικής γλώσσας. Ωστόσο, η λειτουργία αυτών των μοντέλων χαρακτηρίζεται από εξαιρετικά υψηλές απαιτήσεις σε μνήμη (RAM/VRAM) και υπολογιστική ισχύ. Βάσει πειραματικών μετρήσεων, ένα μοντέλο βελτιστοποιημένο για περιβάλλοντα άκρου (edge), όπως το Llama3.2 3B [1], δεσμεύει ~8GB μνήμης, χωρίς την εφαρμογή τεχνικών συμπίεσης, ενώ το ισχυρότερο Llama3.1 8B αγγίζει τα ~16GB. Συνεπώς η πλήρης φόρτωση και εκτέλεση ενός σύγχρονου LLM ακόμα και μικρού σχετικά μεγέθους σε έναν μεμονωμένο υπολογιστικό κόμβο ευρείας κατανάλωσης είναι συχνά ανέφικτη.

Αυτός ο περιορισμός οδηγεί παραδοσιακά σε δύο λύσεις: την αγορά εξειδικευμένου εξοπλισμού υψηλού κόστους ή την χρήση υπηρεσιών νέφους. Η προσέγγιση του νέφους αν και προσβάσιμη, εγείρει κρίσιμα ζητήματα ιδιωτικότητας, καθώς τα δεδομένα του χρήστη μεταφέρονται και επεξεργάζονται σε διακομιστές τρίτων. Ως εκ τούτου, η διατήρηση της πλήρους κυριότητας των δεδομένων καθιστά την τοπική εκτέλεση αναγκαία, ειδικά σε τομείς διαχείρισης ευαίσθητων πληροφοριών όπως η υγειονομική περίθαλψη [2] και ο χρηματοοικονομικός κλάδος [3].

## 1.2 Ερευνητικό Πρόβλημα και Ερευνητικά Ερωτήματα

Η εκτέλεση Μεγάλων Γλωσσικών Μοντέλων σε συσκευές ευρείας κατανάλωσης περιορίζεται σημαντικά από τις υψηλές απαιτήσεις τους σε υπολογιστική ισχύ και διαθέσιμη μνήμη. Η αξιοποίηση πολλαπλών συσκευών σε ένα συνεργατικό περιβάλλον μπορεί να αντιμετωπίσει μέρος αυτών των περιορισμών, μέσω της κατανομής των επιπέδων του μοντέλου και του αντίστοιχου υπολογιστικού φόρτου μεταξύ διαφορετικών κόμβων. Ωστόσο, η κατανομημένη εκτέλεση εισάγει νέα προβλήματα που σχετίζονται με την ετερογένεια του υλικού, τις διαφορετικές δυνατότητες επεξεργασίας και μνήμης των κόμβων, τις καθυστερήσεις επικοινωνίας, την εξισορρόπηση του υπολογιστικού φορτίου και τη διατήρηση της λειτουργίας του συστήματος σε περίπτωση αποτυχίας ή αποσύνδεσης κάποιου συμμετέχοντος κόμβου. Οι δυσκολίες αυτές γίνονται ακόμη σημαντικότερες όταν επιδιώκεται η δημιουργία ενός αποκεντρωμένου συστήματος. Η απουσία ενός μόνιμου κεντρικού κόμβου-συντονιστή προϋποθέτει ότι οι συμμετέχοντες κόμβοι πρέπει να μπορούν να διαχειρίζονται συλλογικά την κατάσταση και την τοπολογία του δικτύου, να αξιολογούν τις διαθέσιμες υπολογιστικές δυνατότητες και να προσαρμόζουν αυτόνομα την κατανομή των επιπέδων του μοντέλου. Παράλληλα, το σύστημα πρέπει να μπορεί να ανταποκρίνεται σε μεταβολές της απόδοσης των κόμβων και να αναδιοργανώνεται όταν κάποιος ενεργός κόμβος αποτυγχάνει, χωρίς να απαιτείται χειροκίνητη παρέμβαση.

Επιπλέον, η χρήση pipeline parallelism για την κατανομή των επιπέδων ενός autoregressive γλωσσικού μοντέλου δημιουργεί επιπρόσθετες προκλήσεις. Η συνολική απόδοση της αλυσίδας εκτέλεσης εξαρτάται σε μεγάλο βαθμό από τον βραδύτερο κόμβο, ενώ η μετάδοση των ενδιάμεσων ενεργοποιήσεων μεταξύ των διαδοχικών σταδίων μπορεί να αποτελέσει σημαντικό σημείο συμφόρησης όταν το διαθέσιμο εύρος ζώνης του δικτύου είναι περιορισμένο. Παράλληλα, η αποτελεσματική υποστήριξη πολλαπλών ταυτόχρονων χρηστών απαιτεί κατάλληλη διαχείριση των αιτημάτων και των αντίστοιχων KV caches, ώστε να αξιοποιούνται καλύτερα οι διαθέσιμοι υπολογιστικοί πόροι.

Με βάση τα παραπάνω, το βασικό ερευνητικό πρόβλημα της παρούσας διπλωματικής αφορά το κατά πόσο μπορεί να αναπτυχθεί ένα δυναμικό, ανθεκτικό σε σφάλματα σύστημα συνεργατικής εκτέλεσης LLMs με αποκεντρωμένη διαχείριση, το οποίο να αξιοποιεί αποτελεσματικά ετερογενείς συσκευές ενός τοπικού δικτύου και να προσαρμόζει αυτόματα τη λειτουργία του στις μεταβαλλόμενες υπολογιστικές και δικτυακές συνθήκες.

Για τη διερεύνηση του παραπάνω προβλήματος διατυπώνονται τα ακόλουθα ερευνητικά ερωτήματα – research questions:

- **RQ1:** Σε ποιο βαθμό είναι εφικτή η αποτελεσματική συνεργατική εκτέλεση ενός Μεγάλου Γλωσσικού Μοντέλου σε ετερογενείς συσκευές ενός τοπικού δικτύου, χωρίς την ανάγκη ενός μόνιμου κεντρικού κόμβου-συντονιστή;
- **RQ2:** Σε ποιο βαθμό η δυναμική ανακατανομή των επιπέδων του μοντέλου, με βάση τις πραγματικές υπολογιστικές δυνατότητες και τους περιορισμούς μνήμης των κόμβων, μπορεί να βελτιώσει την απόδοση του συστήματος σε σύγκριση με μια αρχική ή στατική κατανομή του φόρτου;
- **RQ3:** Πώς επηρεάζουν οι περιορισμοί και οι μεταβολές του διαθέσιμου εύρους ζώνης του δικτύου την απόδοση της κατανεμημένης εκτέλεσης και σε ποιο βαθμό η 8-bit dynamic block-wise quantization των ενδιάμεσων ενεργοποιήσεων μπορεί να περιορίσει το communication overhead;
- **RQ4:** Σε ποιο βαθμό μπορεί η προτεινόμενη αρχιτεκτονική να διατηρεί τη συνέχεια της εκτέλεσης και να αναδιοργανώνει αυτόματα την αλυσίδα σε περίπτωση αποτυχίας ή υποβάθμισης της απόδοσης ενός ενεργού κόμβου;
- **RQ5:** Σε ποιο βαθμό μπορεί η διοχέτευση παράλληλων αιτημάτων και η χρήση ανεξάρτητων KV caches ανά client να βελτιώσει την αξιοποίηση της αλυσίδας εκτέλεσης και τη συνολική διεκπεραιωτική ικανότητα κατά την ταυτόχρονη εξυπηρέτηση πολλαπλών χρηστών;

Τα παραπάνω ερευνητικά ερωτήματα καθοδηγούν τόσο τον σχεδιασμό της προτεινόμενης αρχιτεκτονικής όσο και την πειραματική αξιολόγησή της. Η διερεύνησή τους πραγματοποιείται μέσω της σύγκρισης διαφορετικών στρατηγικών κατανομής επιπέδων, της αξιολόγησης της απόδοσης υπό διαφορετικές συνθήκες δικτύου, της ταυτόχρονης εξυπηρέτησης πολλαπλών clients και της ελεγχόμενης πρόκλησης υποβάθμισης ή αποτυχίας ενεργών κόμβων.

### 1.3 Στόχοι και Συνεισφορές

Ο κύριος στόχος της διπλωματικής εργασίας είναι η ανάπτυξη ενός συστήματος κατανεμημένης εκτέλεσης LLMs, το οποίο αξιοποιεί τον διαθέσιμο εξοπλισμό των χρηστών με ελάχιστη ανάγκη χειροκίνητης διαμόρφωσης και επίβλεψης. Ειδικότερα, στόχος είναι, κατά την προσθήκη υπολογιστικών κόμβων, να συγκροτείται αυτόματα μια ισορροπημένη ως προς το φορτίο, δυναμική και ανθεκτική σε σφάλματα αλυσίδα εκτέλεσης. Επιπλέον, επιδιώκεται αποκεντρωμένος συντονισμός της αρχιτεκτονικής, χωρίς κάποιον κεντρικό κόμβο-συντονιστή, ενσωματώνοντας αποδοτικά ετερογενείς επιταχυντές (CPU και GPU).

Στο πλαίσιο της επίτευξης των παραπάνω στόχων οι κύριες συνεισφορές της εργασίας συνοψίζονται στα εξής:

- **Προσαρμοσμένη Αρχιτεκτονική Μοντέλου:** Αναπτύχθηκε μια προσαρμοσμένη υλοποίηση της αρχιτεκτονικής των μοντέλων Llama 3, μέσω της βιβλιοθήκης PyTorch. Η υλοποίηση αυτή επιτρέπει τη δυναμική φόρτωση συγκεκριμένων επιπέδων του μοντέλου και ενσωματώνει έναν μηχανισμό εναλλασσόμενων KV caches, υποστηρίζοντας την ταυτόχρονη παραγωγή κειμένου για πολλαπλούς χρήστες.
- **Αποκεντρωμένη Διαχείριση Δικτύου:** Υλοποιήθηκε ένα σύστημα διαχείρισης του δικτύου των συμμετεχόντων κόμβων, βασισμένο σε ένα DHT. Μέσω αυτού, νέοι κόμβοι εντάσσονται στο δίκτυο και αναλαμβάνουν τα επίπεδα του μοντέλου που δεν έχουν εξυπηρετηθεί, συνθέτοντας μια πλήρη αλυσίδα εκτέλεσης. Παράλληλα, οι πληροφορίες που δημοσιεύουν οι κόμβοι στο DHT, επιτρέπουν την παρακολούθηση της κατάστασης του δικτύου και καθιστούν εφικτή την λειτουργία των μηχανισμών ανακατανομής επιπέδων και επισκευής.
- **Ανθεκτικότητα σε Σφάλματα (Fault Tolerance):** Ενσωματώθηκαν μηχανισμοί για την επιδιόρθωση της αλυσίδας σε περίπτωση αποτυχίας κάποιου κόμβου. Η αποκατάσταση του δικτύου πραγματοποιείται αξιοποιώντας είτε τους υφιστάμενους ενεργούς κόμβους είτε τους διαθέσιμους εφεδρικούς.
- **Εξισορρόπηση Φορτίου (Load Balancing):** Αναπτύχθηκε μηχανισμός δυναμικής ανακατανομής των επιπέδων του μοντέλου, που εξισορροπεί την αλυσίδα εκτέλεσης ως προς την καθυστέρηση συμπερασμού (inference delay) κάθε ενεργού κόμβου, συνυπολογίζοντας τον ατομικό ρυθμό επεξεργασίας και τη διαθέσιμη μνήμη του.
- **Ευκαιριακή Ανάλυση (Opportunistic Takeover):** Προστέθηκε η δυνατότητα ανάληψης φορτίου, όπου ένας κόμβος με επαρκή μνήμη μπορεί να αναλάβει τα επίπεδα ενός πιο αδύναμου κόμβου της ενεργής αλυσίδας (βάσει ρυθμού επεξεργασίας), με στόχο την μεγιστοποίηση της συνολικής απόδοσης.
- **Υποστήριξη Ετερογένειας:** Το σύστημα επιτρέπει την ταυτόχρονη συμμετοχή κόμβων εξοπλισμένων με CPUs και GPUs (συγκεκριμένα κάρτες γραφικών της Nvidia με υποστήριξη CUDA), αξιοποιώντας τις εγγενείς δυνατότητες της PyTorch.
- **Βελτιστοποίηση Μετάδοσης:** Για την μείωση της καθυστέρησης δικτύου κατά την μεταφορά ενδιάμεσων ενεργοποιήσεων μεταξύ ενεργών κόμβων, εφαρμόζεται 8-bit dynamic block-wise quantization.
- **Αρχιτεκτονική Διοχέτευσης (Pipelining):** Εφαρμόστηκε αποδοτικός μηχανισμός pipelining στα αιτήματα συμπερασμού (inference requests) της ενεργής αλυσίδας. Ο μηχανισμός αυτός αυξάνει την συνολική διεκπεραιωτική ικανότητα του συστήματος, εξασφαλίζοντας υψηλή απόδοση κατά την ταυτόχρονη εξυπηρέτηση πολλαπλών χρηστών.

- **Γραφική Διεπαφή Χρήστη (GUI):** Υλοποιήθηκε διεπαφή χρήστη, που προσφέρει ένα περιβάλλον chatbot για την αλληλεπίδραση με το σύστημα και την παρακολούθηση της κατάστασης και απόδοσης του δικτύου, αντλώντας πληροφορίες από το DHT.

## 1.4 Δομή της Διπλωματικής Εργασίας

Η παρούσα διπλωματική εργασία οργανώνεται σε οκτώ κεφάλαια. Μετά την εισαγωγή, όπου παρουσιάζονται το κίνητρο, το ερευνητικό πρόβλημα, τα ερευνητικά ερωτήματα, οι στόχοι και οι κύριες συνεισφορές της εργασίας, η δομή των επόμενων κεφαλαίων διαμορφώνεται ως εξής:

Στο **Κεφάλαιο 2** παρουσιάζεται το απαραίτητο θεωρητικό υπόβαθρο για την κατανόηση της προτεινόμενης αρχιτεκτονικής. Εξετάζονται οι βασικές αρχές των Μεγάλων Γλωσσικών Μοντέλων και των Transformers, η αρχιτεκτονική της οικογένειας Llama 3, η λειτουργία των KV caches, καθώς και οι τεχνολογίες που αξιοποιούνται για την καταναεμημένη επικοινωνία και διαχείριση του συστήματος, όπως τα RPCs και τα DHTs.

Στο **Κεφάλαιο 3** πραγματοποιείται η βιβλιογραφική ανασκόπηση των κυριότερων προσεγγίσεων βελτιστοποίησης και καταναεμημένης εκτέλεσης LLMs. Παρουσιάζονται τεχνικές βελτιστοποίησης σε επίπεδο μεμονωμένου κόμβου, μέθοδοι παραλληλισμού και σύγχρονες αρχιτεκτονικές καταναεμημένου inference. Ιδιαίτερη έμφαση δίνεται στα συστήματα Petals, AR-MDI και MDI-LLM, ενώ στο τέλος του κεφαλαίου πραγματοποιείται συγκριτική αποτίμηση των υφιστάμενων προσεγγίσεων και προσδιορίζεται το ερευνητικό κενό που επιχειρεί να καλύψει η παρούσα εργασία.

Στο **Κεφάλαιο 4** περιγράφεται η αρχιτεκτονική του προτεινόμενου συστήματος. Αναλύονται η αποκεντρωμένη διαχείριση των συμμετεχόντων κόμβων μέσω του DHT, η συγκρότηση της ενεργής αλυσίδας εκτέλεσης, η κατανομή των transformer layers και οι βασικοί μηχανισμοί επικοινωνίας και συνεργασίας μεταξύ των κόμβων.

Στο **Κεφάλαιο 5** παρουσιάζονται οι επιμέρους λεπτομέρειες της υλοποίησης. Περιγράφονται η προσαρμοσμένη υλοποίηση της αρχιτεκτονικής Llama 3, η διαχείριση των KV caches, η διαδικασία του καταναεμημένου inference, η επικοινωνία μέσω RPC, η κβαντοποίηση των ενδιάμεσων ενεργοποιήσεων, η υποστήριξη πολλαπλών ταυτόχρονων clients και η γραφική διεπαφή χρήστη.

Στο **Κεφάλαιο 6** αναλύονται οι μηχανισμοί δυναμικής προσαρμογής και ανθεκτικότητας της αρχιτεκτονικής. Ειδικότερα, παρουσιάζονται η Δυναμική Ανακατανομή Επιπέδων για την εξισορρόπηση του υπολογιστικού φορτίου, ο μηχανισμός Ευκαιριακής Ανάληψης για την αντικατάσταση κόμβων χαμηλότερης απόδοσης και οι διαδικασίες Επισκευής της Αλυσίδας σε περίπτωση αποτυχίας ενεργών κόμβων.

Στο **Κεφάλαιο 7** πραγματοποιείται η πειραματική αξιολόγηση της προτεινόμενης αρχιτεκτονικής. Αρχικά παρουσιάζονται η πειραματική μεθοδολογία, το υπολογιστικό περιβάλλον, οι δικτυακές συνθήκες και οι μετρικές αξιολόγησης. Στη συνέχεια συγκρίνονται διαφορετικές στρατηγικές κατανομής των επιπέδων, αξιολογείται η αποτελεσματικότητα της κβαντοποίησης, εξετάζεται η ταυτόχρονη εξυπηρέτηση πολλαπλών clients και αξιολογούνται οι μηχανισμοί δυναμικής εξισορρόπησης φορτίου και ανθεκτικότητας σε σφάλματα. Το κεφάλαιο ολοκληρώνεται με τη συζήτηση και τη συνολική αποτίμηση των αποτελεσμάτων σε συνάρτηση με τα ερευνητικά ερωτήματα της εργασίας.

Τέλος, στο **Κεφάλαιο 8** συνοψίζονται τα βασικά συμπεράσματα και τα κυριότερα ευρήματα της εργασίας. Παρουσιάζονται οι περιορισμοί της υλοποίησης και της πειραματικής αξιολόγησης και διατυπώνονται κατευθύνσεις για μελλοντική επέκταση και περαιτέρω αξιολόγηση της προτεινόμενης αρχιτεκτονικής.

## 2 Θεωρητικό Υπόβαθρο

### 2.1 Large Language Models

Τα Μεγάλα Γλωσσικά Μοντέλα (Large Language Models - LLMs) είναι μια κατηγορία μοντέλων βαθιάς μάθησης (deep learning models), εκπαιδευμένα σε τεράστιο όγκο δεδομένων, επιτρέποντάς τους να κατανοούν και να παράγουν φυσική γλώσσα.

Στον πυρήνα τους είναι μηχανές στατιστικής πρόβλεψης, που προβλέπουν επαναληπτικά την επόμενη λέξη ή token σε μια ακολουθία. Εντοπίζουν πρότυπα σε ένα κείμενο και παράγουν γλώσσα που ακολουθεί αυτά τα πρότυπα.

Τα LLMs είναι το αποκορύφωμα εξέλιξης δεκαετιών στην έρευνα της Επεξεργασίας Φυσικής Γλώσσας (Natural Language Processing - NLP) και η ανάπτυξή τους είναι σε μεγάλο βαθμό υπεύθυνη για την εκρηκτική πρόοδο της Τεχνητής Νοημοσύνης (Artificial Intelligence - AI) από τα τέλη της δεκαετίας του 2010 και καθ' όλη τη διάρκεια της δεκαετίας του 2020. Δημοφιλή LLMs και οι διεπαφές τους (ChatGPT [4], Gemini [5], Claude [6]) έχουν γίνει ευρέως γνωστά, φέρνοντας την παραγωγική τεχνητή νοημοσύνη (generative AI) στο προσκήνιο του δημόσιου ενδιαφέροντος.

#### 2.1.1 Transformer

Τα LLMs βασίζονται σε ένα είδος αρχιτεκτονικής νευρωνικού δικτύου που ονομάζεται Transformer, η οποία διαπρέπει στην επεξεργασία ακολουθιακών δεδομένων, όπως ένα κείμενο. Η αρχιτεκτονική αυτή εισήχθη το 2017 με το επιστημονικό άρθρο “Attention is All You Need” [7] και θεωρείται στιγμή – ορόσημο στην ιστορία του deep learning.

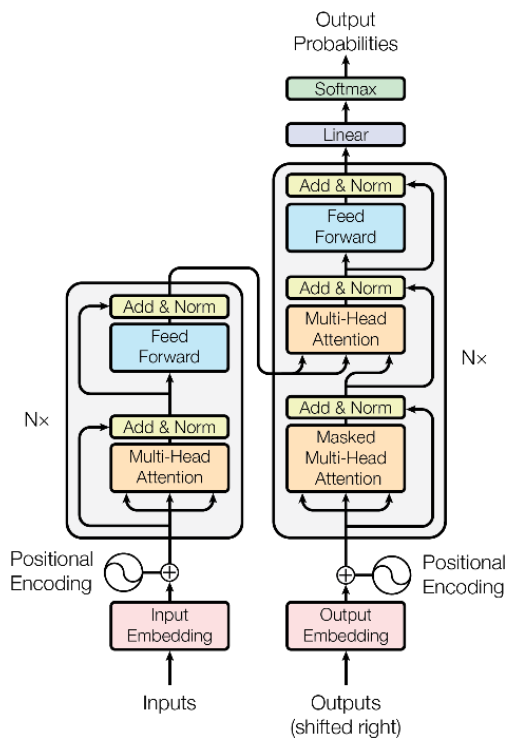


Figure 1. Διάγραμμα αρχιτεκτονικής του Transformer. Πηγή: [7]

### 2.1.2 Self-Attention

Ο πυρήνας του Transformer είναι ο μηχανισμός self-attention, που του επιτρέπει να «δίνει προσοχή» σε διαφορετικά tokens σε διαφορετικές στιγμές. Το self-attention είναι χρήσιμο από την μία επειδή επιτρέπει στο μοντέλο να υπολογίζει σχέσεις και εξαρτήσεις μεταξύ των tokens, ακόμα και αν έχουν μεγάλη απόσταση μεταξύ τους, ενώ ταυτόχρονα, η αρχιτεκτονική του, επιτρέπει την παραλληλοποίηση, καθιστώντας το πολύ πιο αποδοτικό από προηγούμενες μεθόδους.

Εσωτερικά η αρχιτεκτονική του Transformer αποτελείται από πολλαπλά διαδοχικά επίπεδα επεξεργασίας. Αρχικά το κείμενο εισόδου χωρίζεται σε βασικές μονάδες που ονομάζονται tokens. Στη συνέχεια, κάθε token αντιστοιχίζεται σε μια διανυσματική αναπαράσταση γνωστή ως embedding, η οποία αναπαριστά την βασική του σημασία. Καθώς τα embeddings, διατρέχουν τα διαδοχικά επίπεδα του μοντέλου, η αναπαράστασή τους αναπροσαρμόζεται και εμπλουτίζεται με βάση τα συμφραζόμενα και το θεματικό πλαίσιο του κειμένου.

Ο στόχος αυτής της διαδικασίας είναι το μοντέλο να μάθει σημασιολογικές συσχετίσεις μεταξύ λέξεων, έτσι ώστε λέξεις όπως «φύλλο» και «δέντρο» να βρεθούν πιο κοντά στον διανυσματικό χώρο σε μια έκθεση για ένα δάσος απ' ό,τι οι λέξεις «φύλλο» και «πίτα».

Ο μηχανισμός self-attention βοηθά το μοντέλο να κατανοήσει ποια μέρη του κειμένου είναι σημαντικά όταν κάνει προβλέψεις. Αυτό επιτυγχάνεται προβάλλοντας αρχικά το κάθε embedding σε τρία είδη διανυσμάτων, πολλαπλασιάζοντάς τα με τα κατάλληλα μητρώα βαρών.

Αυτά είναι:

- Το διάνυσμα **Query** –  $Q_i$ , που αναπαριστά αυτό που «αναζητά» το τρέχον token, δηλαδή κάνει την ερώτηση: «Ποια άλλα tokens είναι σημαντικά για εμένα;».
- Το διάνυσμα **Key** –  $K_i$ , που αναπαριστά την ταυτότητα του token στην πρόταση και δείχνει το πόσο σχετίζεται το token με ένα query.
- Το διάνυσμα **Value** –  $V_i$ , που περιέχει την πληροφορία ή σημασία του token, η οποία κλιμακώνεται βάσει των attention weights που υπολογίζονται, για να δημιουργηθεί η τελική, συμφραζομενικά εμπλουτισμένη αναπαράσταση του token.

Έστω ότι τα διανύσματα αυτά είναι διανύσματα-γραμμές και έχουμε μια ακολουθία από  $n$  tokens. Τα διανύσματα ομαδοποιούνται στα μητρώα  $Q \in \mathbb{R}^{n \times d_k}$ ,  $K \in \mathbb{R}^{n \times d_k}$ ,  $V \in \mathbb{R}^{n \times d_v}$ :

$$Q = \begin{bmatrix} Q_1 \\ Q_2 \\ \vdots \\ Q_n \end{bmatrix} \quad K = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_n \end{bmatrix} \quad V = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_n \end{bmatrix}$$

Σκοπός είναι ο υπολογισμός των attention weights, τα οποία μας δείχνουν το πόσο βάρος πρέπει να δώσουμε στα υπόλοιπα tokens του κειμένου όταν υπολογίζουμε την νέα αναπαράσταση του τρέχοντος token και ο πολλαπλασιασμός αυτών των attention weights με το μητρώο  $V$ , για να πάρουμε τις νέες αναπαραστάσεις.

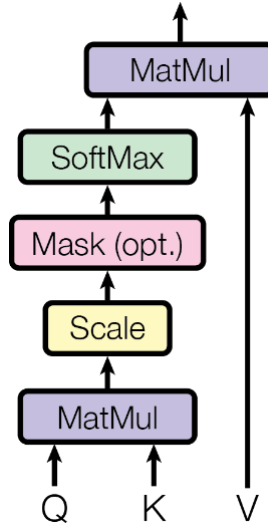


Figure 2 . Διάγραμμα του μηχανισμού Scaled Dot Product Attention. Πηγή: [7]

Μαθηματικά έχουμε:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Equation 1. Τύπος υπολογισμού του μητρώου των εξόδων του μηχανισμού attention. Πηγή: [7]

Αρχικά υπολογίζεται το εσωτερικό γινόμενο των  $Q$  και  $K$ , το οποίο αναπαριστά την ομοιότητα των tokens μεταξύ τους και κάθε τιμή κλιμακώνεται με έναν παράγοντα  $\frac{1}{\sqrt{d_k}}$ , για να μην εμφανιστούν πολύ μεγάλες τιμές και το  $softmax$  φτάσει σε περιοχές που έχει πολύ μικρά gradients.

Υπολογισμός των similarity scores:

$$QK^T = \begin{bmatrix} Q_1 \\ Q_2 \\ \vdots \\ Q_n \end{bmatrix} \begin{bmatrix} K_1^T & K_2^T & \dots & K_n^T \end{bmatrix} = \begin{bmatrix} Q_1 K_1^T & Q_1 K_2^T & \dots & Q_1 K_n^T \\ Q_2 K_1^T & Q_2 K_2^T & \dots & Q_2 K_n^T \\ \vdots & \vdots & \ddots & \vdots \\ Q_n K_1^T & Q_n K_2^T & \dots & Q_n K_n^T \end{bmatrix} = \begin{bmatrix} S_{11} & S_{12} & \dots & S_{1n} \\ S_{21} & S_{22} & \dots & S_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ S_{n1} & S_{n2} & \dots & S_{nn} \end{bmatrix}$$

Το αποτέλεσμα αποτελεί το μητρώο των similarity scores. Για παράδειγμα η γραμμή 1 του αποτελέσματος δείχνει τα σκορ ομοιότητας του token 1 με όλα τα υπόλοιπα.

Κλιμάκωση και κανονικοποίηση:

$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) = \begin{bmatrix} softmax\left(\left(\frac{S_{11}}{\sqrt{d_k}} \quad \frac{S_{12}}{\sqrt{d_k}} \quad \dots \quad \frac{S_{1n}}{\sqrt{d_k}}\right)\right) \\ softmax\left(\left(\frac{S_{21}}{\sqrt{d_k}} \quad \frac{S_{22}}{\sqrt{d_k}} \quad \dots \quad \frac{S_{2n}}{\sqrt{d_k}}\right)\right) \\ \vdots \\ softmax\left(\left(\frac{S_{n1}}{\sqrt{d_k}} \quad \frac{S_{n2}}{\sqrt{d_k}} \quad \dots \quad \frac{S_{nn}}{\sqrt{d_k}}\right)\right) \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} & \dots & W_{1n} \\ W_{21} & W_{22} & \dots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{n1} & W_{n2} & \dots & W_{nn} \end{bmatrix}$$

Η εφαρμογή του *softmax* σε κάθε γραμμή μετατρέπει τα similarity scores των tokens σε κατανομές πιθανότητας και δίνει τα attention weights του κάθε token. Τα attention weights αναπαριστούν πόσο βάρος έχει κάθε άλλο token όταν υπολογίζεται η νέα αναπαράσταση του τρέχοντος token.

Τέλος, υπολογίζονται οι νέες συμφραζομενικά εμπλουτισμένες αναπαραστάσεις των tokens, πολλαπλασιάζοντας το μητρώο των attention weights με το μητρώο των values.

Υπολογισμός των νέων αναπαραστάσεων:

$$\begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1n} \\ W_{21} & W_{22} & \cdots & W_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ W_{n1} & W_{n2} & \cdots & W_{nn} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_n \end{bmatrix} = \begin{bmatrix} W_{11}V_1 + W_{12}V_2 + \cdots + W_{1n}V_n \\ W_{21}V_1 + W_{22}V_2 + \cdots + W_{2n}V_n \\ \vdots \\ W_{n1}V_1 + W_{n2}V_2 + \cdots + W_{nn}V_n \end{bmatrix} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}$$

Φαίνεται ότι η νέα αναπαράσταση  $Y_i$  ενός token αποτελεί ένα weighted sum όλων των διανυσμάτων value, σταθμισμένων με τα attention weights του token. Αυτό το attention weighted sum καθορίζει κατά πόσο ένα διάνυσμα value θα περάσει στην νέα αναπαράσταση του token και επιτρέπει στο μοντέλο να επικεντρωθεί στα σχετικά συμφραζόμενα, ενώ αγνοεί λιγότερο σημαντικά tokens.

Τέλος, οι νέες αυτές διανυσματικές αναπαραστάσεις των tokens προστίθενται στο αρχικό τους embedding. Με αυτόν τον τρόπο το embedding του κάθε token ενημερώνεται ώστε να εκφράζει καλύτερα το πλαίσιο στο οποίο βρίσκεται σε σχέση με τα υπόλοιπα.

### 2.1.3 Multi – Head Attention

Στην γλώσσα, οι λέξεις ενός κειμένου παρουσιάζουν πολύπλευρες συσχετίσεις (π.χ. συντακτική συσχέτιση, σημασιολογία, συσχέτιση προσδιορισμού). Για να μπορέσει το Transformer να εντοπίσει αυτές τις διαφορετικές συσχετίσεις, υλοποιείται το multi – head attention με πολλαπλά παράλληλα attention heads.

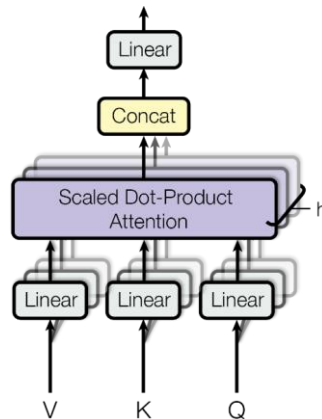


Figure 3. Διάγραμμα του μηχανισμού Multi-Head Attention με  $h$  παράλληλα attention heads. Πηγή: [7]

Το embedding  $E_i$  του κάθε token προβάλλεται γραμμικά σε  $h$  διαφορετικά διανύσματα  $Q_i^k, K_i^k, V_i^k$  όπου  $k \in [1, 2, \dots, h]$ . Τα διανύσματα αυτά περνούν παράλληλα από  $h$  attention heads και καταλήγουμε με  $h$  διαφορετικές διανυσματικές αναπαραστάσεις για κάθε token του κειμένου. Τέλος, όλες οι αναπαραστάσεις ενώνονται και περνούν από ένα τελικό επίπεδο εξόδου όπου προβάλλονται στην αρχική τους διάσταση.

Μαθηματικά έχουμε:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}^O$$

$$\text{where } \text{head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V)$$

Equation 2. Τύπος υπολογισμού του μητρώου εξόδου του multi-head attention. Πηγή: [7]

Στην πράξη, κατά την εκπαίδευση του μοντέλου τα διαφορετικά attention heads μαθαίνουν να εντοπίζουν τις πολύπλευρες συσχετίσεις της γλώσσας.

#### 2.1.4 Autoregressive Decoder-Only Transformers

Η αρχιτεκτονική του αυθεντικού Transformer [7] περιλάμβανε δομή encoder – decoder. Ο encoder επιτρέπει την αμφίδρομη κατανόηση ενός κειμένου επεξεργάζοντας παράλληλα όλα τα tokens, ενώ η παραγωγή κειμένου του decoder περιορίζεται στην επεξεργασία από αριστερά προς τα δεξιά. Σύγχρονα μοντέλα transformer βασίζονται είτε μόνο στον encoder είτε μόνο στον decoder είτε και στα δύο, ανάλογα με τους στόχους της εκπαίδευσής τους (pre-training objectives).

Autoencoding (AE) μοντέλα, όπως τα BERT [8] και BART [9], τα οποία αξιοποιούν είτε μόνο encoders, είτε encoder και decoder, εκπαιδεύονται στην ανάκτηση ενός κειμένου από αλλοιωμένα δεδομένα και διαπρέπουν στην κατανόηση της γλώσσας.

Autoregressive (AR) μοντέλα, όπως τα GPT [10], Llama, που αξιοποιούν decoders, εκπαιδεύονται στην αναπαραγωγή κειμένων και προορίζονται για εφαρμογές παραγωγής κειμένου, όπως μετάφραση, περίληψη και απάντηση ερωτήσεων.

Η οικογένεια των μοντέλων Llama 3, στην οποία βασίζεται η παρούσα διπλωματική εργασία, όπως και τα περισσότερα σύγχρονα LLMs αποτελούν autoregressive decoder-only transformers.

##### 2.1.4.1 Causal Attention Mask

Τα autoregressive decoder-only transformers εκπαιδεύονται για την παραγωγή κειμένου ένα token την φορά. Συνεπώς, για ένα δεδομένο token πρέπει να «δίνουν προσοχή» μόνο στα tokens που έχουν προηγηθεί στην πρόταση, χωρίς να κοιτάζουν μελλοντικά tokens.

Για να επιτευχθεί αυτό χρησιμοποιείται causal – attention masking. Δηλαδή, πριν την εφαρμογή του *softmax* στον μηχανισμό attention προστίθεται στο μητρώο των similarity scores, ένα κάτω τριγωνικό μητρώο της μορφής  $\mathbf{M}_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}$ , όπου όλα τα στοιχεία πάνω από την διαγώνιο είναι  $-\infty$ , και τα υπόλοιπα είναι 0. Εφόσον  $\text{softmax}(-\infty) = 0$ , τα similarity scores ενός token για μελλοντικά tokens θα είναι 0.

$$\begin{aligned}
 \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{softmax} \left( \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V} = \\
 &= \text{softmax} \left( \begin{bmatrix} S_{11} & S_{12} & \cdots & S_{1n} \\ S_{21} & S_{22} & \cdots & S_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ S_{n1} & S_{n2} & \cdots & S_{nn} \end{bmatrix} + \begin{bmatrix} 0 & -\infty & \cdots & -\infty \\ 0 & 0 & \cdots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \right) \mathbf{V} = \\
 &= \begin{bmatrix} W_{11} & 0 & \cdots & 0 \\ W_{21} & W_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ W_{n1} & W_{n2} & \cdots & W_{nn} \end{bmatrix} \mathbf{V}
 \end{aligned}$$

Equation 3. Μηχανισμός attention με εφαρμογή causal attention mask.

## 2.2 Llama 3

Η υλοποίηση του συστήματος της παρούσας διπλωματικής εργασίας βασίστηκε στην αρχιτεκτονική των μοντέλων Llama 3 της Meta [11]. Με στόχο την πληρέστερη κατανόηση του συστήματος, στις ενότητες που ακολουθούν αναλύονται τα επιμέρους αρχιτεκτονικά στοιχεία της συγκεκριμένης οικογένειας μοντέλων.

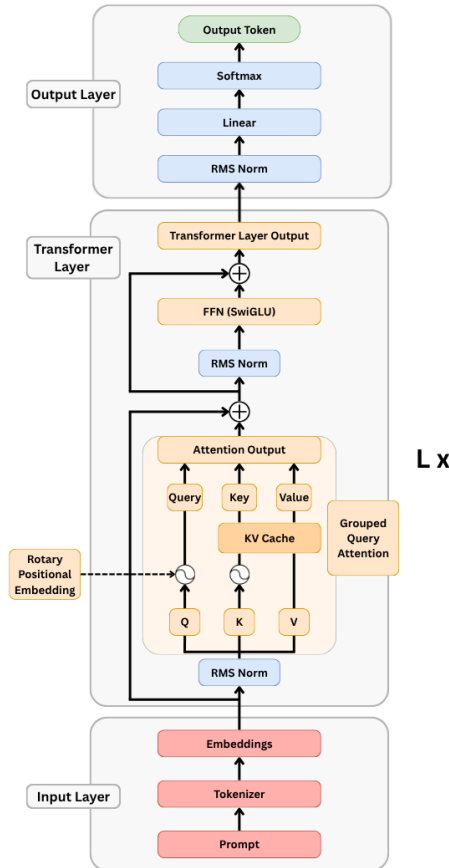


Figure 4. Διάγραμμα αρχιτεκτονικής του Llama 3 με L transformer layers. Πηγή: [από τον συγγραφέα].

## 2.2.1 Input Layer

Αρχικά το κείμενο εισόδου του χρήστη χωρίζεται σε subwords που ονομάζονται tokens μέσω ενός tokenizer. Τα tokens αυτά αντιστοιχίζονται στα token-ids τους με βάση το λεξιλόγιο που χρησιμοποιείται και τέλος κάθε token-id μετατρέπεται σε διάνυσμα embedding διάστασης  $d_{model}$ .

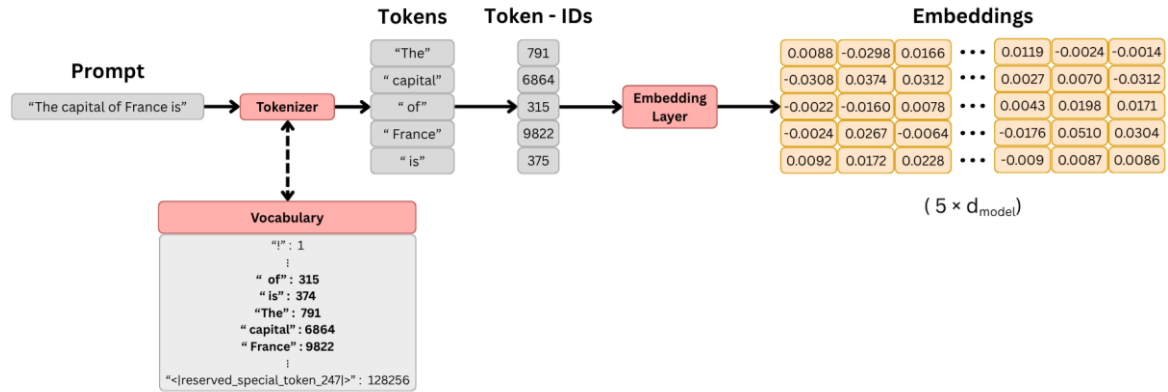


Figure 5. Διάγραμμα των βημάτων του input layer. Πηγή: [από τον συγγραφέα]

## 2.2.2 Transformer Layer

Στη συνέχεια, τα embedding μετασχηματίζονται και εμπλουτίζονται πληροφοριακά καθώς διέρχονται από  $L$  διαδοχικά επίπεδα transformer. Ο αριθμός των επιπέδων είναι ανάλογος της πολυπλοκότητας του μοντέλου και αυξάνεται στα δίκτυα με περισσότερες παραμέτρους. Ενδεικτικά, το μοντέλο Llama 3.2-1B αποτελείται από 16 επίπεδα, ενώ το Llama 3.1-8B διαθέτει 32.

### 2.2.2.1 Root Mean Square Normalization (RMS Norm)

Όπως απεικονίζεται στο Figure 4, η κανονικοποίηση RMS (RMS Norm) εφαρμόζεται στις ενδιάμεσες ενεργοποιήσεις σε πολλαπλά στάδια του μοντέλου. Η κανονικοποίηση είναι απαραίτητη καθώς τα διανύσματα των ενεργοποιήσεων διαθέτουν μεγάλο αριθμό διαστάσεων (4096 στην περίπτωση του Llama 3.1 8B), γεγονός που αυξάνει την πιθανότητα εμφάνισης ακραίων τιμών. Κατά την εκπαίδευση, αυτό το φαινόμενο μπορεί να οδηγήσει σε έκρηξη ή μηδενισμό των gradients, προκαλώντας καθυστέρηση στην σύγκλιση ή και απόκλιση. Μέσω της κανονικοποίησης, οι τιμές των ενεργοποιήσεων προσαρμόζονται σε ένα ελεγχόμενο εύρος, σταθεροποιώντας και επιταχύνοντας την διαδικασία εκπαίδευσης.

Η συγκεκριμένη μέθοδος εισήχθη το 2019 στο άρθρο [12] ως μια εναλλακτική της μεθόδου LayerNorm [13]. Οι συγγραφείς της μελέτης υπέθεσαν πως η επιτυχία της μεθόδου LayerNorm δεν οφείλεται στην αμεταβλητότητα ως προς την επανακέντρωση (re-centering invariance), αλλά στην αμεταβλητότητα ως προς την κλιμάκωση (re-scaling invariance). Βάσει αυτού, πρότειναν την κανονικοποίηση RMS, η οποία επικεντρώνεται στο re-scaling invariance και κανονικοποιεί τις ενεργοποιήσεις αποκλειστικά μέσω της στατιστικής του root mean square.

Η κανονικοποίηση RMS είναι υπολογιστικά πιο αποδοτική από την μέθοδο LayerNorm, καθώς εξαλείφει την ανάγκη υπολογισμού της μέσης τιμής και της επανακέντρωσης (re-centering) των ενεργοποιήσεων. Διατηρώντας μόνο τον υπολογισμό της κλιμάκωσης με την στατιστική RMS, μειώνει τον υπολογιστικό φόρτο, επιτυγχάνοντας ταυτόχρονα παρόμοιες επιδόσεις.

Έστω  $\mathbf{a} \in \mathbb{R}^{1 \times d_{model}}$  ένα διάνυσμα ενδιάμεσης ενεργοποίησης και  $a_i, i \in [1, 2, \dots, d_{model}]$  η  $i$ -οστή συνιστώσα του  $\mathbf{a}$ .

Για την κανονικοποίηση του διανύσματος υπολογίζεται:

$$a'_i = \frac{a_i}{RMS(\mathbf{a})} \gamma_i, \text{ όπου } RMS(\mathbf{a}) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} (a_i)^2 + \epsilon}$$

Η τιμή  $\epsilon$ , είναι μια πολύ μικρή σταθερά που προστίθεται για την αποφυγή διαίρεσης με το μηδέν. Η εκπαιδευσιμη παράμετρος  $\gamma_i$ , που είναι ξεχωριστή για κάθε διάσταση του διανύσματος, κλιμακώνει τις κανονικοποιημένες τιμές, παρέχοντας περαιτέρω σταθερότητα.

### 2.2.2.2 Rotary Positional Embeddings (RoPE)

Από μόνα τους τα embeddings δεν περιέχουν καμία πληροφορία σχετικά με την θέση του token στην ακολουθία. Ωστόσο, η επίγνωση της απόλυτης θέσης τους και κυρίως της σχετικής τους απόστασης είναι κρίσιμη για την αποτελεσματική εκπαίδευση ενός LLM στην κατανόηση της γλώσσας.

Τα μοντέλα Llama 3, όπως και τα περισσότερα LLMs, έχουν υιοθετήσει την μέθοδο των Rotary Positional Embeddings RoPE [14] για την κωδικοποίηση της πληροφορίας θέσης. Το RoPE κωδικοποιεί την απόλυτη θέση ενός token πολλαπλασιάζοντάς το με ένα μητρώο περιστροφής, ενώ η εξάρτηση της σχετικής θέσης ενσωματώνεται στον μηχανισμό του self-attention.

Συγκριτικά με προηγούμενες μεθόδους, η μέθοδος RoPE πλεονεκτεί, επειδή παρέχει ευελιξία στο μέγεθος της ακολουθίας και επιτρέπει την μείωση της συσχέτισης των tokens καθώς αυξάνεται η σχετική τους απόσταση (decaying dependency).

Παρακάτω ακολουθεί η μαθηματική διατύπωση της μεθόδου.

Έστω  $\mathbb{S}_N = \{w_i\}_{i=1}^N$  είναι μια ακολουθία με  $N$  tokens, με το  $w_i$  να είναι το  $i$ -οστό στοιχείο. Τα αντίστοιχα embeddings του  $\mathbb{S}_N$  είναι τα  $\mathbb{E}_N = \{\mathbf{x}_i\}_{i=1}^N$ , όπου  $\mathbf{x}_i \in \mathbb{R}^d$  είναι το  $d$ -διάστατο embedding του  $w_i$ .

Στόχος είναι η κωδικοποίηση της πληροφορίας της απόλυτης θέσης του token στα αντίστοιχα διανύσματα query και key, έτσι ώστε το εσωτερικό τους γινόμενο, που αποτελεί τον πυρήνα του μηχανισμού attention, να εξαρτάται αποκλειστικά από τις αναπαραστάσεις των tokens και την σχετική τους απόσταση.

Συγκεκριμένα, αναζητούνται τα εξής: η συνάρτηση  $f_q$ , που αντιστοιχίζει το embedding στην θέση  $m$  ( $\mathbf{x}_m$ ) στο αντίστοιχο διάνυσμα  $\mathbf{q}$  με την θέση  $m$  κωδικοποιημένη ( $\mathbf{q}_m$ ), η συνάρτηση  $f_k$  που κάνει το ίδιο για την εύρεση του διανύσματος  $\mathbf{k}$  της θέσης  $n$  ( $\mathbf{x}_n$ ) και ένα εσωτερικό γινόμενο μεταξύ των  $\mathbf{q}_m$  και  $\mathbf{k}_n$ , τέτοιο ώστε να εξαρτάται μόνο από τα  $\mathbf{x}_m, \mathbf{x}_n$  και την απόσταση  $m - n$ .

Δηλαδή αναζητούνται τα εξής:

$$\begin{aligned} \mathbf{q}_m &= f_q(\mathbf{x}_m, m) \\ \mathbf{k}_n &= f_k(\mathbf{x}_n, n) \\ \mathbf{q}_m^T \mathbf{k}_n &= \langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, m - n) \end{aligned}$$

Το RoPE πραγματοποιεί την κωδικοποίηση της απόλυτης θέσης με την εφαρμογή ενός συγκεκριμένου μητρώου περιστροφής  $\mathbf{R}_{\theta,m}^d$  στις προβολές  $\mathbf{W}_q \mathbf{x}_m$  και  $\mathbf{W}_k \mathbf{x}_n$ . Το μητρώο αυτό περιστρέφει διαδοχικά ζεύγη συνιστωσών των προβολών κατά γωνίες  $m\theta_i$  και  $n\theta_i$  αντίστοιχα. Οι γωνίες περιστροφής καθορίζονται από τις σταθερές  $\theta_i = 1000^{-\frac{2(i-1)}{d}}, i \in [1, 2, \dots, \frac{d}{2}]$ .

Μαθηματικά η συνάρτηση ορίζεται ως:

$$\mathbf{f}_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\theta,m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

Equation 4. Κωδικοποίηση της απόλυτης θέσης εφαρμόζοντας έναν πίνακα περιστροφής στα διανύσματα query και key. Πηγή: [14]

όπου:

$$\mathbf{R}_{\theta,m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{\frac{d}{2}} & -\sin m\theta_{\frac{d}{2}} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{\frac{d}{2}} & \cos m\theta_{\frac{d}{2}} \end{bmatrix}$$

Χρησιμοποιώντας αυτά τα κωδικοποιημένα ως προς την θέση διανύσματα  $\mathbf{q}_m$  και  $\mathbf{k}_n$ , το εσωτερικό γινόμενο  $\mathbf{q}_m^T \mathbf{k}_n$  υπολογίζεται συναρτήσει μόνο των  $\mathbf{x}_m, \mathbf{x}_n, n - m$ , ως εξής:

$$\mathbf{q}_m^T \mathbf{k}_n = (\mathbf{R}_{\theta,m}^d \mathbf{W}_q \mathbf{x}_m)^T (\mathbf{R}_{\theta,n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}_m^T \mathbf{W}_q^T \mathbf{R}_{\theta,n-m}^d \mathbf{W}_k \mathbf{x}_n$$

Equation 5. Το similarity score μεταξύ query και key ενσωματώνοντας τη σχετική απόστασή τους ( $n-m$ ). Πηγή: [14]

Συνεπώς, εφαρμόζοντας την κωδικοποίηση απόλυτης θέσης της εξίσωσης (4) στα queries και keys και υπολογίζοντας τα similarity scores του μηχανισμού attention μέσω της εξίσωσης (5), το μοντέλο αποκτά επίγνωση της σειράς των tokens στην ακολουθία και της σχετικής απόστασής τους, διασφαλίζοντας ότι tokens με μεγάλη απόσταση εμφανίζουν μειωμένη συσχέτιση.

### 2.2.2.3 KV Caching

Το LLM inference χωρίζεται σε δύο στάδια: το στάδιο prefill και το στάδιο decoding (ή generation).

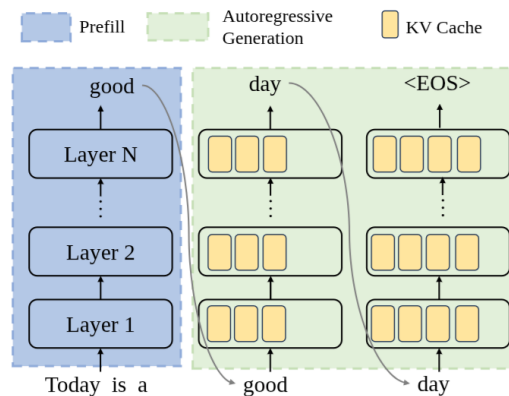


Figure 6. Απεικόνιση των σταδίων prefill και decoding (autoregressive generation). Πηγή: [15]

Το στάδιο prefill, λαμβάνει χώρα στην αρχή της εκτέλεσης. Κατά την διάρκεια αυτού του σταδίου, όλα τα tokens του αρχικού prompt προβάλλονται στα αντίστοιχα διανύσματα query, key και value, εκτελείται ο μηχανισμός attention και προβλέπεται το πρώτο νέο token της ακολουθίας.

Το στάδιο decoding εκτελείται σε κάθε επόμενο βήμα inference για την διαδοχική παραγωγή των υπόλοιπων tokens. Σε αυτή την φάση, μόνο το τελευταίο παραχθέν token προβάλλεται στα διανύσματα query, key και value και ο μηχανισμός attention εκτελείται εκ νέου με τα keys και values όλων των προηγούμενων tokens.

Δεδομένου ότι τα keys και values των προηγούμενων tokens δεν αλλάζουν, ο επανυπολογισμός τους αποτελεί περιττό υπολογιστικό φόρτο. Μια προφανής βελτιστοποίηση είναι η προσωρινή αποθήκευση (caching) τους, ώστε να επαναχρησιμοποιούνται άμεσα σε κάθε inference step. Η τεχνική αυτή είναι γνωστή ως Key-Value (KV) Caching [16] και αποτρέπει τις επαναλαμβανόμενες πράξεις, επιταχύνοντας την φάση της αποκωδικοποίησης.

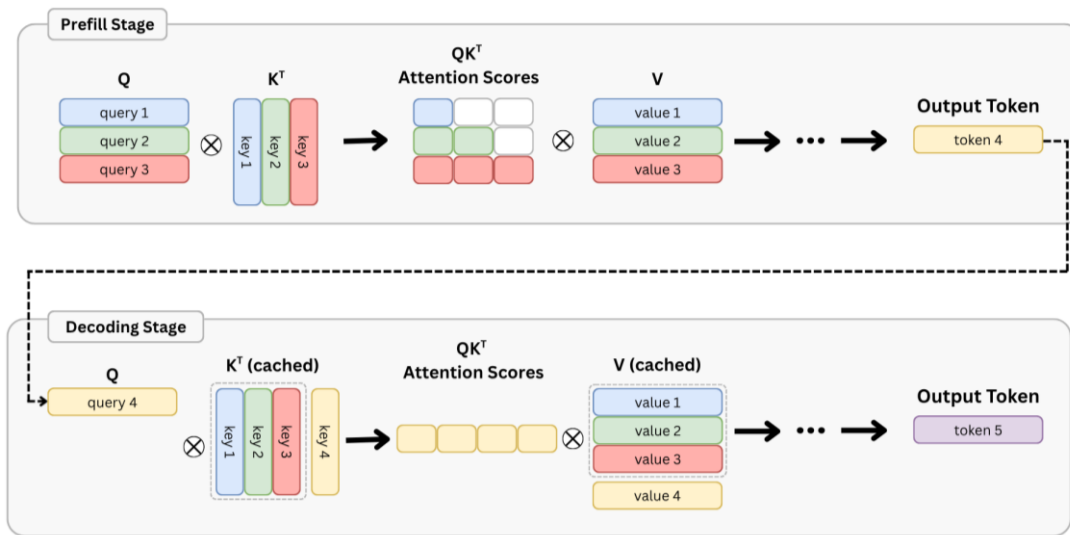


Figure 7. Διαγραμματική απεικόνιση του prefill και decoding stage με kv caching. Στο decoding stage φαίνεται η επαναχρησιμοποίηση των μητρώων  $K$  και  $V$  από το προηγούμενο στάδιο. Πηγή: [από τον συγγραφέα]

#### 2.2.2.4 Grouped – Query Attention (GQA)

Το αυθεντικό μοντέλο Transformer εφαρμόζει την τεχνική του multi-head attention (MHA), όπως αναφέρθηκε στην υποενότητα 2.1.3, όπου τα embeddings προβάλλονται σε  $h$  διαφορετικά μητρώα  $Q^i$ ,  $K^i$ ,  $V^i$  και ο μηχανισμός attention εκτελείται  $h$  φορές παράλληλα. Αυτή η μέθοδος όμως προκαλούσε σημαντικό bottleneck στην εκτέλεση του μοντέλου, λόγω του memory bandwidth overhead της φόρτωσης των attention keys και values για κάθε decoder block.

Το πρόβλημα της μνήμης μπορεί να αντιμετωπιστεί σημαντικά εφαρμόζοντας multi-query attention (MQA) [17], το οποίο χρησιμοποιεί πολλαπλά query heads αλλά ένα key και value head.

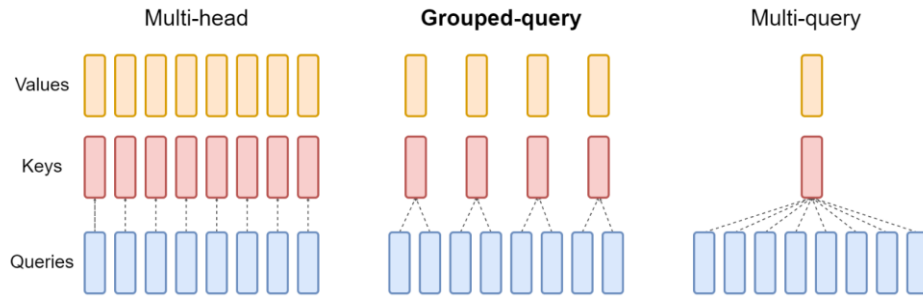


Figure 8. Διαγραμματική επισκόπηση των μεθόδων MHA, GQA και MQA. Πηγή: [18]

Το MQA όμως, μπορεί να οδηγήσει σε υποβάθμιση της ποιότητας παραγωγής και αστάθεια κατά την εκπαίδευση. Συνεπώς, το Llama 3, όπως και πολλά άλλα LLMs, έχουν υιοθετήσει την μέθοδο του grouped-query attention (GQA) [18], η οποία αποτελεί μια μέση λύση ανάμεσα στις δύο προηγούμενες μεθόδους. Το GQA επιτυγχάνει ποιότητα κοντά σε αυτή του MHA, ενώ παράλληλα είναι σχεδόν τόσο γρήγορο όσο το MQA.

Συγκεκριμένα στο GQA έχουμε  $h_q$  query heads και  $h_{kv}$  key-value heads. Τα query heads χωρίζονται σε  $G = h_{kv}$  groups με κάθε group να μοιράζεται ένα key-value head.

Για παράδειγμα στο μοντέλο Llama 3.1-8B:  $h_q = 32$  και  $h_{kv} = 8$ , οπότε έχουμε 8 groups με το κάθε group να αποτελείται από  $r = \frac{h_q}{h_{kv}} = 4$  query heads.

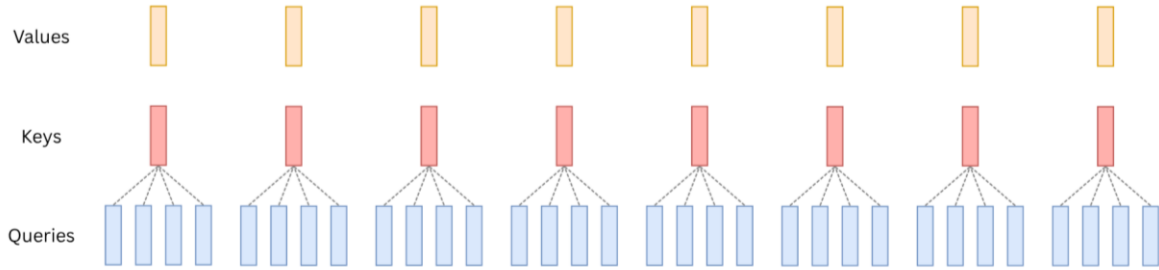


Figure 9. GQA στο Llama 3 - 8B με 32 query heads και 8 key-value heads. Πηγή: [από τον συγγραφέα].

Σε όλα τα transformer layers και σε κάθε inference step προστίθεται ένα νέο διάνυσμα key και value για κάθε key-value head στην KV cache. Έτσι, με το να μοιράζονται πολλαπλά query heads ενός group το ίδιο key και value, το μέγεθος της KV cache παραμένει διαχειρίσιμο και η προσπέλασή της γίνεται πιο γρήγορα.

### 2.2.2.5 FeedForward Network (Δίκτυο Εμπρόσθιας Τροφοδότησης)

Μετά από το GQA block τα embeddings που είναι πλέον εμπλουτισμένα με το context της ακολουθίας, κανονικοποιούνται μέσω RMSNorm και περνούν από ένα δίκτυο εμπρόσθιας τροφοδότησης (FeedForward Network - FFN). Στο FFN τα attention output embedding επεκτείνονται σε υψηλότερη διάσταση, ώστε να μπορέσουν να μάθουν πιο σύνθετα χαρακτηριστικά για τα tokens.

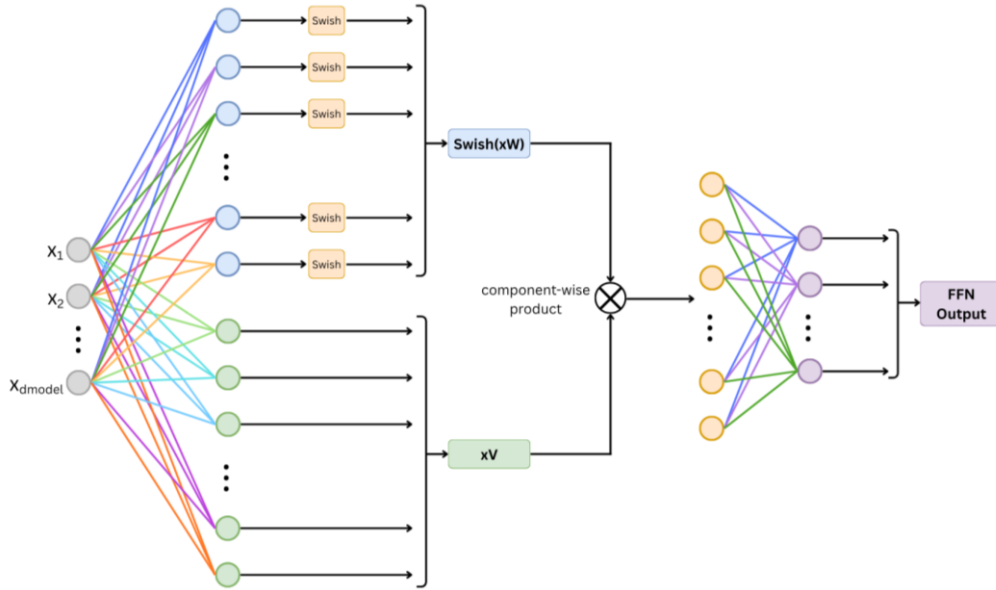


Figure 10. Διάγραμμα αρχιτεκτονικής του FFN με ενεργοποίηση SwiGLU. Πηγή: [από τον συγγραφέα].

Στα μοντέλα Llama 3, το επίπεδο FFN χρησιμοποιεί την τεχνική SwiGLU, η οποία εισήχθη το 2020 στο επιστημονικό άρθρο [19] και παρουσιάζει καλύτερες επιδόσεις από την τεχνική που εφαρμόζε το αυθεντικό Transformer στο επίπεδο FFN.

Συγκεκριμένα, το FFN εφαρμόζει δύο παράλληλους γραμμικούς μετασχηματισμούς στην είσοδο, ένας εκ των οποίων περνά από την συνάρτηση ενεργοποίησης  $Swish_1$ . Οι δύο μετασχηματισμοί πολλαπλασιάζονται στοιχείο-προς-στοιχείο και το αποτέλεσμα περνά από έναν τρίτο γραμμικό μετασχηματισμό που επιστρέφει την έξοδο στις αρχικές διαστάσεις της εισόδου.

$$Swish_\beta = x\sigma(\beta x)$$

Equation 6. Συνάρτηση ενεργοποίησης Swish.

Έστω,  $\mathbf{x} \in (\mathbb{R}^{1 \times d_{model}}$  η είσοδος του FFN,  $\mathbf{W} \in \mathbb{R}^{d_{model} \times d_{ff}}$ ,  $\mathbf{V} \in \mathbb{R}^{d_{model} \times d_{ff}}$  τα μητρώα των δύο πρώτων παράλληλων γραμμικών μετασχηματισμών και  $\mathbf{W}_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$  το μητρώο του τρίτου μετασχηματισμού.

Τότε:

$$FFN_{SwiGLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{W}_2) = (Swish_1(\mathbf{x}\mathbf{W}) \otimes \mathbf{x}\mathbf{V})\mathbf{W}_2$$

Equation 7. Μαθηματική διατύπωση του FFN με ενεργοποίηση SwiGLU. Πηγή: [19]

### 2.2.3 Output Layer

Η έξοδος του τελευταίου transformer layer οδηγείται στο output layer. Αρχικά, περνά από RMS normalization. Έπειτα, η έξοδος περνά από έναν γραμμικό μετασχηματισμό, ο οποίος αυξάνει την διάσταση του διανύσματος στο μέγεθος του λεξιλογίου του μοντέλου. Οι έξοδοι του γραμμικού μετασχηματισμού ονομάζονται logits και αποτελούν μη κανονικοποιημένες τιμές πρόβλεψης για όλα τα tokens του λεξιλογίου. Τέλος, τα logits μετατρέπονται σε πιθανότητες μέσω της συνάρτησης *softmax* και γίνεται η επιλογή του επόμενου token.

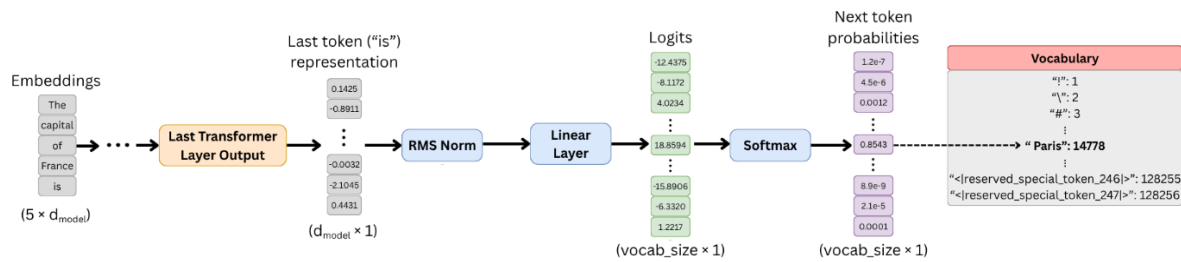


Figure 11. Διάγραμμα της αρχιτεκτονικής του output layer με το παράδειγμα της πρόβλεψης του επόμενου token της ακολουθίας: "The capital of France is". Πηγή: [από τον συγγραφέα]

## 2.3 Distributed Hash Table (DHT)

Το σύστημα που παρουσιάζεται στην διπλωματική αποτελεί ένα κατακεντρωμένο σύστημα, στο οποίο η διαχείριση της τοπολογίας πραγματοποιείται με αποκεντρωμένο τρόπο. Για να επιτευχθεί αυτό, χρησιμοποιείται ένας Κατακεντρωμένος Πίνακας Κατακερματισμού (Distributed Hash Table - DHT). Συγκεκριμένα, αξιοποιείται η βιβλιοθήκη *hivemind* [20] της Python, η οποία είναι βασισμένη στο *Kademlia* DHT [21] και είναι ειδικά βελτιστοποιημένη για την γρήγορη προσπέλαση και αποθήκευση μεγάλου όγκου ελαφρών μεταδεδομένων.

Ένα DHT είναι μια αποκεντρωμένη δομή αποθήκευσης που μοιράζεται δεδομένα σε μορφή κλειδιού-τιμής (key-value) σε ένα δίκτυο πολλαπλών κόμβων. Η δομή αυτή επιτρέπει την ανάκτηση και αποθήκευση πληροφοριών χωρίς την ανάγκη κάποιου κεντρικού κόμβου-συντονιστή.

Το *Kademlia* είναι ένα αποκεντρωμένο peer-to-peer σύστημα που παρέχει αποδεδειγμένη συνέπεια και απόδοση σε περιβάλλοντα που υπόκεινται σε συχνά σφάλματα κόμβων. Η βασική καινοτομία του *Kademlia* είναι η χρήση της συνάρτησης XOR για τον υπολογισμό της «απόστασης» μεταξύ των αναγνωριστικών των κόμβων (node IDs) και των κλειδιών. Κάθε κόμβος του δικτύου διατηρεί πληροφορίες δρομολόγησης για άλλους κόμβους σε λίστες που ονομάζονται k-buckets. Αυτή η τοπολογία επιτρέπει στο σύστημα να εντοπίζει τους κατάλληλους κόμβους δρομολογώντας παράλληλα και ασύγχρονα ερωτήματα (asynchronous queries), ελαχιστοποιώντας τις καθυστερήσεις.

## 2.4 Remote Procedure Calling (RPC)

Το Remote Procedure Calling (RPC) αποτελεί ένα μοντέλο επικοινωνίας πελάτη/διακομιστή (client/server), το οποίο επιτρέπει την εκτέλεση απομακρυσμένων κλήσεων με τέτοιο τρόπο ώστε να φαίνονται και να λειτουργούν ως τοπικές κλήσεις. Αν και το RPC αποτελεί μια παλαιότερη τεχνική, τα σύγχρονα κατακεντρωμένα περιβάλλοντα απαιτούν υψηλότερες αποδόσεις, με χαμηλότερη καθυστέρηση.

Για την κάλυψη αυτών των αναγκών αναπτύχθηκε το gRPC [22]. Το gRPC είναι ένα RPC framework ανοιχτού κώδικα, το οποίο χρησιμοποιεί το πρωτόκολλο επικοινωνίας HTTP/2. Αναπτύχθηκε αρχικά από την Google και πλέον διαχειρίζεται από την Cloud Native Computing Foundation (CNCF). Σε αντίθεση με frameworks όπως το REST που χρησιμοποιούν μορφές

απλού κειμένου (π.χ. JSON ή XML), το gRPC χρησιμοποιεί τα Protocol Buffers (Protobuf) ως γλώσσα ορισμού διεπαφής (Interface Design Language - IDL) για την σειριοποίηση δεδομένων σε δυαδική μορφή.

Στο σύστημα της διπλωματικής, η βιβλιοθήκη gRPC αποτελεί τον κεντρικό μηχανισμό για την υλοποίηση απομακρυσμένων κλήσεων, προσφέροντας κρίσιμα πλεονεκτήματα απόδοσης. Συγκεκριμένα, η χρήση των protocol buffers για την σειριοποίηση των δεδομένων σε δυαδική μορφή, εξασφαλίζει μειωμένο μέγεθος μηνυμάτων καθώς και ταχύτερη ανάλυση και μετάδοση των ενδιάμεσων ενεργοποιήσεων. Παράλληλα, αξιοποιείται η εγγενής δυνατότητα εφαρμογής συμπίεσης Gzip του gRPC στα κανάλια επικοινωνίας, η οποία μειώνει δραστικά τον όγκο των δεδομένων που διακινούνται στο δίκτυο. Τέλος, η χρήση του πρωτοκόλλου μεταφοράς HTTP/2 ενισχύει την ταχύτητα και την αποδοτικότητα του συστήματος έναντι του προκατόχου του (HTTP/1.1), καθώς χρησιμοποιεί δυαδική δομή για την αποστολή μηνυμάτων, ελαττώνει τις απαιτούμενες συνδέσεις TCP και εφαρμόζει header compression, περιορίζοντας έτσι τη στατική καθυστέρηση και βελτιστοποιώντας τη συνολική διεκπεραιωτική ικανότητα.

## 3 Βιβλιογραφική Ανασκόπηση

Παραδοσιακά, η εκτέλεση των LLMs βασίζεται σε εξειδικευμένες ομογενείς συστοιχίες GPU (GPU clusters) υψηλού κόστους ή σε υποδομές νέφους (cloud infrastructure). Παρόλο που τα cloud-based APIs παρέχουν προσβάσιμη και ταχεία εκτέλεση, επιβάλλουν υψηλά λειτουργικά κόστη και εισάγουν κρίσιμα ζητήματα ιδιωτικότητας και κυριότητας δεδομένων. Για να γεφυρωθεί το χάσμα μεταξύ των απαγορευτικών απαιτήσεων σε υλικό των LLMs και των περιορισμένων πόρων των συσκευών ευρείας κατανάλωσης, η έρευνα έχει στραφεί προς τεχνικές βελτιστοποίησης και κατανομημένα, συνεργατικά πλαίσια εκτέλεσης. Αυτές οι μέθοδοι επιδιώκουν να συγκεντρώσουν ετερογενείς, παγκοσμίως κατανομημένους ή τοπικούς κόμβους υπολογισμού σε έναν ενιαίο ιστό εκτέλεσης.

Στο κεφάλαιο αυτό, αναλύεται η σχετική βιβλιογραφία γύρω από την κατανομημένη εκτέλεση των LLMs. Αρχικά περιγράφονται συστημικές και αλγοριθμικές βελτιστοποιήσεις, που επιτρέπουν την φόρτωση μοντέλων σε συσκευές με περιορισμένους πόρους. Στη συνέχεια, αναλύονται τοπολογίες παραλληλοποίησης διαφορετικών αρχιτεκτονικών (edge-cloud, local edge, heterogeneous GPU networks). Τέλος, αναλύονται τα θεμελιώδη συστήματα και οι αλγόριθμοι που επιτρέπουν την ανάπτυξη αυτόνομων, ανθεκτικών σε σφάλματα αλυσίδων εκτέλεσης και ευθυγραμμίζονται άμεσα με τους στόχους της διπλωματικής εργασίας.

### 3.1 *Single-Node Βελτιστοποιήσεις Συμπερασμού*

Για τον μετριασμό των απαιτήσεων μνήμης και υπολογιστικής ισχύος κατά την εκτέλεση LLMs σε συσκευές περιορισμένων πόρων, έχουν προταθεί πολυάριθμες βελτιστοποιήσεις.

#### 3.1.1 *Αρχιτεκτονικές Βελτιστοποιήσεις*

Σε επίπεδο αρχιτεκτονικής του ίδιου του μοντέλου, συστήματα όπως το MobileLLM [23] προτείνουν μια “deep and thin” αρχιτεκτονική στοχευμένη σε κινητές συσκευές, υλοποιώντας ένα sub-billion parameter μοντέλο. Παράλληλα, αρχιτεκτονικές Mixture-of-Experts (MoE) προσφέρουν καινοτόμες λύσεις για την εκτέλεση LLMs σε edge devices [24], [25], [26]. Αυτές οι προσεγγίσεις εκμεταλλεύονται τεχνικές όπως το sparse activation και dynamic routing, με σκοπό την μείωση των υπολογιστικών απαιτήσεων.

#### 3.1.2 *Αλγοριθμικές Βελτιστοποιήσεις*

Σε αλγοριθμικό επίπεδο μέθοδοι όπως το Speculative Decoding (SD) [27], [28] και τεχνικές συμπίεσης (quantization, pruning, knowledge distillation) επιτυγχάνουν σημαντική επιτάχυνση στην παραγωγή κειμένου και μειώνουν τις απαιτήσεις μνήμης αντίστοιχα. Συγκεκριμένα, το SD επιχειρεί να παρακάμψει την καθυστέρηση της σειριακής παραγωγής χρησιμοποιώντας ένα ελαφρύ μοντέλο (draft model) για την πρόβλεψη υποψήφιων tokens, τα οποία επαληθεύει παράλληλα ένα μεγαλύτερο μοντέλο. Ωστόσο, σε αποκεντρωμένα περιβάλλοντα το network overhead της μεταφοράς αυτών των υποψήφιων tokens μέσω low-bandwidth διασυνδέσεων αναιρεί τον χρόνο υπολογισμού που εξοικονομεί.

### 3.1.2.1 Τεχνικές Συμπίεσης

Η Κβαντοποίηση (Quantization) αναφέρεται στην διαδικασία μετατροπής βαρών και ενεργοποιήσεων υψηλής ακρίβειας (floating-point) σε αναπαραστάσεις χαμηλότερης ακρίβειας. Η τεχνική αυτή μειώνει δραστικά το μέγεθος του μοντέλου και τις υπολογιστικές απαιτήσεις, ενώ διατηρεί σχετικά υψηλή ακρίβεια. Χωρίζεται σε δύο είδη: post-training quantization (PTQ) και quantization-aware training (QAT). Το PTQ χρησιμοποιείται αφού εκπαιδευτεί το μοντέλο, οπότε εφαρμόζεται πιο γρήγορα και είναι λιγότερο απαιτητικό από το QAT. Μερικές αξιόλογες μέθοδοι PTQ είναι το GPTQ [29] και AWQ [30]. Το GPTQ αξιοποιεί πληροφορία δεύτερης τάξης (second-order information) για την αντιστάθμιση του σφάλματος, μειώνοντας αποτελεσματικά την ακρίβεια σε 3 ή 4 bit ανά βάρος. Το Activation-aware Quantization (AWQ) εφαρμόζει κβαντοποίηση στα βάρη και στις ενεργοποιήσεις και βασίζεται στην παρατήρηση ότι ένα μικρό ποσοστό (0.1% - 1%) των βαρών είναι κρίσιμα για την απόδοση του LLM.

Παρά την βελτιστοποίηση που μπορεί να προσφέρει, η έντονη κβαντοποίηση μπορεί να οδηγήσει σε υποβάθμιση της ακρίβειας του μοντέλου και σε αύξηση του perplexity. Συμπληρωματικά, foundation models (70B+ parameters) παραμένουν πολύ μεγάλα για υλικό ευρείας κατανόησης ακόμα και με συμπίεση INT4. Συγκεκριμένα, ένα μοντέλο με 70B παραμέτρους σε ακρίβεια των 4 bit απαιτεί 35-40 GB μνήμης μόνο για τα στατικά βάρη.

Το Pruning αφορά την επιλεκτική αφαίρεση βαρών, με σκοπό την μείωση της πολυπλοκότητας του μοντέλου και την αύξηση της υπολογιστικής απόδοσης. Χωρίζεται κυρίως σε structured και unstructured pruning. Στο structured pruning, αφαιρούνται ολόκληρα υποσύνολα παραμέτρων όπως layers, channels ή φίλτρα. Αυτό είναι ιδιαίτερα ωφέλιμο για βελτιστοποιήσεις υλικού, μιας και οδηγεί σε πιο κανονικά memory access patterns και απλοποιεί τους υπολογισμούς. Το unstructured pruning, αφαιρεί ατομικά βάρη, προσφέροντας υψηλότερα ποσοστά συμπίεσης. Η μέθοδος αυτή όμως οδηγεί στη δημιουργία αραιών μητρώων, που μπορεί να είναι λιγότερο συμβατά με παραδοσιακές αρχιτεκτονικές υλικού [31].

Το Knowledge Distillation (KD) είναι μια τεχνική στην οποία μεταφέρεται γνώση από ένα μεγάλο μοντέλο (teacher) σε ένα μικρότερο πιο αποδοτικό μοντέλο (student). Χωρίζεται σε δύο είδη: το black-box και το white-box knowledge distillation. Στην πρώτη προσέγγιση το student model μαθαίνει μόνο από τις εξόδους του teacher model και χρησιμοποιείται όταν η εσωτερική δομή του teacher model είναι κλειστή (proprietary) ή όταν οι αρχιτεκτονικές των δύο μοντέλων διαφέρουν σημαντικά. Αντιθέτως, στο white-box distillation το student model έχει πρόσβαση στις κατανομές των εσωτερικών καταστάσεων του teacher model, επιτρέποντας μια βαθύτερη και πιο ακριβή διαδικασία εκπαίδευσης. Για παράδειγμα τα 1B και 3B Llama 3.2 μοντέλα [32] ενσωματώνουν τα logits του μοντέλου Llama 3.1 70B στην εκπαίδευσή τους για την ανάκτηση της επίδοσής τους μετά από pruning.

## 3.2 Τεχνικές Παραλληλοποίησης

Για την κατανομή μεγάλων νευρωνικών δικτύων σε πολλαπλούς υπολογιστικούς κόμβους απαιτούνται συγκεκριμένες μέθοδοι παραλληλοποίησης. Οι δύο κυριότερες μέθοδοι είναι το Tensor Parallelism (TP) και Model/Pipeline Parallelism (PP). Η δυνατότητα εφαρμογής τους εξαρτάται από τους περιορισμούς εύρους ζώνης και καθυστέρησης της δικτυακής υποδομής στην οποία εφαρμόζονται.

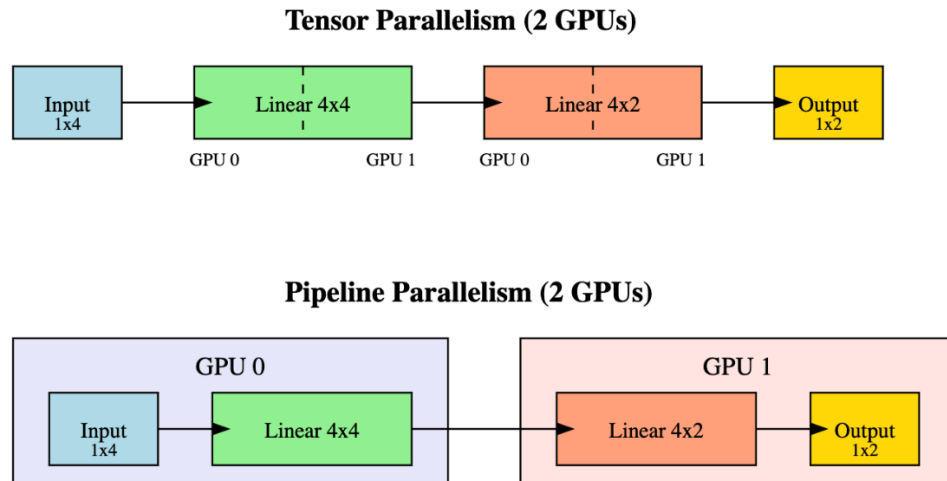


Figure 12. Διάγραμμα απεικόνισης του tensor και pipeline parallelism με δύο GPUs. Πηγή: [33]

### 3.2.1 Tensor Parallelism (TP)

Στο tensor parallelism μοιράζονται μαθηματικές πράξεις, όπως μεγάλοι πολλαπλασιασμοί μητρώων εντός ενός επιπέδου του μοντέλου, σε πολλαπλές GPUs ταυτόχρονα. Τα μερικά αποτελέσματα από τις GPUs πρέπει να συγχρονιστούν και να ενωθούν χρησιμοποιώντας collective communication primitives, όπως το *AllReduce*, πριν ξεκινήσει η επεξεργασία του επόμενου επιπέδου.

Παρόλο που το TP προσφέρει εξαιρετικό load balancing και σχεδόν μηδενικό idle time μεταξύ των συμμετεχόντων κόμβων, απαιτεί συνεχείς, high-bandwidth και ultra-low latency διασυνδέσεις (π.χ. NVLink ή InfiniBand) για να λειτουργήσει αποδοτικά. Σε αποκεντρωμένα περιβάλλοντα edge, όπου οι κόμβοι συνδέονται με τυπικό Ethernet ή Wifi με ταχύτητες από 20 Mbps έως 1 Gbps, οι υψηλές απαιτήσεις επικοινωνίας του TP το καθιστούν μη βιώσιμο.

Η θεμελιώδης υλοποίηση του tensor parallelism για transformer μοντέλα εισήχθη από το Megatron-LM [34], το οποίο διαχωρίζει τους πίνακες βαρών των επιπέδων self-attention και feed-forward είτε κατά γραμμές (row-parallel) είτε κατά στήλες (column-parallel), ελαχιστοποιώντας τον απαιτούμενο αριθμό synchronization points ανά transformer block σε δύο all-reduce operations (ένα στο forward και ένα στο backward pass).

Μια βασική πρόκληση του TP είναι η αυξημένη κατανάλωση μνήμης λόγω αποθήκευσης ενδιάμεσων ενεργοποιήσεων για το backward pass. Η εργασία "Reducing Activation Recomputation in Large Transformer Models" [35] αντιμετωπίζει αυτό το πρόβλημα εισάγοντας sequence parallelism (παραλληλισμός στη διάσταση της ακολουθίας για τα layer normalization και dropout στάδια, που δεν παραλληλοποιούνται από το tensor parallelism) σε συνδυασμό με selective activation recomputation, μειώνοντας δραστικά το memory footprint χωρίς τον υπολογιστικό αντίκτυπο πλήρους recomputation. Παρόλο που αυτές οι τεχνικές στοχεύουν κυρίως σε training clusters με NVLink, υπογραμμίζουν το γεγονός ότι ακόμα και σε ιδανικές, high-bandwidth συνθήκες το TP εισάγει σημαντικό memory overhead.

### 3.2.2 Pipeline Parallelism (PP)

Το model parallelism με την μορφή του pipeline parallelism, χωρίζει ένα LLM στο βάθος του. Για παράδειγμα, ένα LLM με  $L$  transformer layers συνολικά, χωρίζεται σε διακριτά, ακολουθιακά υποσύνολα με το κάθε υποσύνολο να ανατίθεται σε έναν κόμβο μιας ακολουθιακής αλυσίδας. Κατά την μεταφορά δεδομένων μεταξύ pipeline stages, ο όγκος επικοινωνίας καθορίζεται από τις διαστάσεις του διανύσματος ενεργοποίησης και απαιτεί πολύ λιγότερο bandwidth συγκριτικά με την πυκνή συλλογική επικοινωνία του tensor parallelism. Έτσι, το pipeline parallelism αποτελεί το πρότυπο για γεωγραφικά καταναμημένες ή τοπικές edge-network αρχιτεκτονικές inference.

Παρόλα αυτά, το κλασικό pipeline parallelism προσβάλλεται από “pipeline bubbles”, δηλαδή περιόδους όπου downstream κόμβοι περιμένουν τους upstream κόμβους να τελειώσουν την επεξεργασία και upstream κόμβοι παραμένουν ανενεργοί περιμένοντας το επόμενο inference step. Συμπληρωματικά, το συνολικό throughput του ακολουθιακού pipeline περιορίζεται από την ταχύτητα επεξεργασίας του πιο αργού κόμβου. Η βελτιστοποίηση του PP σε ετερογενή περιβάλλοντα απαιτεί λοιπόν ειδικούς αλγορίθμους ανάθεσης επιπέδων, dynamic routing και πρωτόκολλα micro-batching. Οι τεχνικές αυτές είναι απαραίτητες για την εξισορρόπηση του υπολογιστικού φορτίου και την απόκρυψη της καθυστέρησης επικοινωνίας μέσω της επικάλυψής της με υπολογισμούς.

## 3.3 Αρχιτεκτονικές Καταναμημένου Συμπερασμού

Όταν οι φυσικοί περιορισμοί των συσκευών δεν μπορούν να ξεπεραστούν από on-device τεχνικές βελτιστοποίησης, απαιτείται συνεργατική καταναμημένη εκτέλεση των μοντέλων σε πολλαπλές συσκευές. Η υλοποίηση της καταναμημένης εκτέλεσης των LLMs σε διαφορετικές τοπολογίες δικτύου και σε συσκευές με διαφορετικές δυνατότητες έχει αποφέρει πολλαπλά πλαίσια καταναμημένου συμπερασμού. Αυτά τα πλαίσια παρουσιάζουν θεμελιώδεις διαφορές στον βαθμό που βασίζονται σε υποδομές νέφους, στην παρουσία ενός κεντρικού κόμβου – συντονιστή και στην ανεκτικότητα που δείχνουν απέναντι στην ετερογένεια των συμμετεχουσών συσκευών.

### 3.3.1 Edge-Cloud Split Computing

Αρχιτεκτονικές split computing προσπαθούν να λύσουν το πρόβλημα των περιορισμένων πόρων των edge devices μεταφέροντας το βαρύτερο υπολογιστικό φορτίο σε cloud servers, διατηρώντας κάποια αρχικά στάδια επεξεργασίας, embedding layers ή κρίσιμες για την ιδιωτικότητα διαδικασίες sanitization σε edge devices.

Στο framework Memory and Latency Constrained Inference of Large Language Models via Adaptive Split Computing [36], παρουσιάζεται μια autoregressive-aware split αρχιτεκτονική. Χρησιμοποιείται One-Point Split Compression (OPSC), όπου εφαρμόζεται mixed-precision κβαντοποίηση για να χωριστεί το μοντέλο σε δύο μέρη. Το ένα μέρος είναι βαριά συμπιεσμένο, low-precision και εκτελείται στο edge device και το άλλο μέρος είναι full-precision και εκτελείται στο cloud. Για την διαχείριση του communication overhead μέσω ασύρματου ή κινητού δικτύου, εφαρμόζεται συμπίεση δύο σταδίων των ενδιάμεσων ενεργοποιήσεων, μέσω Threshold Splitting (TS) και Token-wise Adaptive Bit Quantization (TAB-Q).

Παρομοίως, το framework EdgeShard [15] χωρίζει το LLM σε διακριτά υπολογιστικά shards που κατανέμονται μη-συμμετρικά μεταξύ πολλαπλών edge devices ή/και cloud servers. Πραγματοποιείται offline profiling με σκοπό τη δημιουργία ενός μαθηματικού cost model που αναπαριστά τις ταχύτητες υπολογισμού και τους δικτυακούς περιορισμούς των συσκευών. Στη συνέχεια, χρησιμοποιώντας αλγορίθμους δυναμικού προγραμματισμού επιλέγει με βέλτιστο τρόπο τα τμήματα του μοντέλου που θα εκτελέσει κάθε συσκευή. Με αυτόν τον τρόπο αποφεύγεται η συχνά αργή και ασταθής συνεργασία μεταξύ edge devices και cloud σε περιπτώσεις όπου η υψηλότερη υπολογιστική ισχύς του cloud server δεν μπορεί να αξιοποιηθεί πλήρως, λόγω ασθενούς σύνδεσης δικτύου.

### **3.3.2 Κατανεμημένος Συμπερασμός Άκρου σε Τοπικά Δίκτυα**

Με στόχο την διατήρηση ιδιωτικότητας δεδομένων και την εξάλειψη της εξάρτησης από το νέφος, διάφορα πλαίσια αξιοποιούν μόνο edge devices και λειτουργούν εντός ενός τοπικού δικτύου όπως εταιρικά intranets και smart homes.

Το framework Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling [37], προσπαθεί να αντιμετωπίσει τις δοκιμασίες του task allocation και dynamic scheduling εντός τοπικών δικτύων, υπό αυστηρά deadline constraints. Εισάγει τον μηχανισμό Fair Cost-Efficient Incentive Mechanism (FCIM) για την ανάθεση εργασιών και επιπέδων, ο οποίος λειτουργεί ως ένα αποκεντρωμένο, auction-based πρωτόκολλο. Ο μηχανισμός FCIM επιλέγει cost-effective και memory-feasible edge devices, ενώ παράλληλα παρακινεί δίκαιη κατανομή και ειλικρινές bidding, διασφαλίζοντας ότι δεν θα διαταράξει κάποια συσκευή το pipeline, δημιουργώντας λανθασμένες δυνατότητες. Συμπληρωματικά, το framework εφαρμόζει έναν Adaptive Dynamic Scheduling Algorithm (ADSA) για τον χρονοπρογραμματισμό της εκτέλεσης στις συσκευές. Ο ADSA είναι μια preemption-aware, deadline-driven στρατηγική, η οποία αναδιατάσσει δυναμικά τις εργασίες με βάση τα χαρακτηριστικά του φόρτου και τους χρόνους άφιξης, μειώνοντας δραστικά τις καθυστερήσεις αναμονής και τον συνολικό χρόνο ολοκλήρωσης των εργασιών.

Το LOIP [38] είναι ένα framework συνεργατικής lossless εκτέλεσης LLMs για συσκευές με περιορισμένους πόρους. Λόγω περιορισμών μνήμης στο edge hardware απαιτείται offloading κάποιων βαρών στον εξωτερικό αποθηκευτικό χώρο, το οποίο εισάγει καθυστερήσεις στην εκτέλεση, γνωστές ως “loading bubbles”. Το LOIP μετριάζει αυτό το bottleneck αξιοποιώντας μια προσέγγιση offloading-based interleaved pipeline parallelism, η οποία πραγματοποιεί στρατηγική επικάλυψη της φόρτωσης των βαρών από τον δίσκο με τους τρέχοντες υπολογισμούς και την επικοινωνία μεταξύ συσκευών. Το σύστημα συνδυάζει ένα offloading-aware cost model με έναν fine-grained layer allocation scheduler, βελτιστοποιεί μεταφορές δεδομένων χρησιμοποιώντας custom CUDA operators που προορίζονται για Unified Memory Architectures (UMA) και υλοποιεί μια στρατηγική δυναμικής προσαρμογής μνήμης για την δυναμική διαχείριση προβλέπτων network bandwidths και της αύξησης της KV cache.

### **3.3.3 Ετερογενείς και Εθελοντικές GPUs**

Το μεγαλύτερο μέρος της έρευνας στην κατανεμημένη εκτέλεση LLMs αφορά ετερογενείς συσκευές σε ασταθή δίκτυα και επικεντρώνεται σε εθελοντικές GPUs και GPU clusters. Τα frameworks αυτής της κατηγορίας απαιτούν προηγμένους αλγορίθμους ανάθεσης εργασιών και ειδικές μεθόδους διαχείρισης των καθυστερήσεων λόγω δικτύου.

Σε μη-αποκεντρωμένες μεθοδολογίες, συστήματα όπως το Helix [39] λειτουργούν σε ετερογενείς GPU clusters, εφαρμόζοντας κεντροποιημένους αλγορίθμους mixed-integer linear programming (MILP). Το Helix διατυπώνει την ανάθεση των LLM layers ως πρόβλημα max-flow

σε ένα directed weighted graph. Παρόλο που επιτυγχάνει μαθηματικά βέλτιστη ανάθεση, ο επιλυτής MILP είναι υπολογιστικά δαπανηρός και βασίζεται σε κεντροποιημένη ενορχήστρωση.

Για την χαλάρωση της κεντροποιημένης φύσης του scheduler, με την ταυτόχρονη διαχείριση της υψηλής δικτυακής καθυστέρησης, έχουν αναπτυχθεί διάφορα ημι-αποκεντρωμένα frameworks.

Το BloomBee [40] προορίζεται για internet-scale inference και αντιμετωπίζει την επικοινωνία μεγάλων αποστάσεων ως το κυριότερο bottleneck. Επικεντρώνεται σε τρεις βασικές βελτιστοποιήσεις: ανάθεση επιπέδων μέσω δυναμικού προγραμματισμού, τοπικό tensor offloading (μετακίνηση ανενεργών KV caches στην CPU RAM για την μεγιστοποίηση της VRAM) και εντατικό micro-batching. Για την μείωση του communication overhead, το BloomBee υλοποιεί έναν προσαρμοσμένο αλγόριθμο byte-split lossless compression ειδικά σχεδιασμένο για ενδιάμεσες ενεργοποιήσεις. Συγκεκριμένα, χωρίζει κάθε FP16 στοιχείο σε δύο 8-bit byte streams, φανερώνοντας επαναλαμβανόμενα δυαδικά μοτίβα τα οποία μπορούν να αξιοποιηθούν αποτελεσματικά από αλγορίθμους κωδικοποίησης εντροπίας. Συμπληρωματικά, υλοποιείται ένας μηχανισμός pruned speculative decoding, χρησιμοποιώντας έναν ελαφρύ classifier που αφαιρεί draft tokens χαμηλής πιθανότητας πριν την μετάδοσή τους.

Το Parallax [41] παρουσιάζει έναν ανθεκτικό scheduler δύο φάσεων, ειδικά σχεδιασμένο για ασταθή περιβάλλοντα εθελοντικών πόρων. Η πρώτη φάση αποτελεί μία latency-dominant στρατηγική ανάθεσης που χρησιμοποιεί δυναμικό προγραμματισμό και έναν fractional water-filling αλγόριθμο για να χωρίσει το μοντέλο σε συνεχή στάδια και να τα καταναείμει δίκαια σε GPUs με διαφορετικές υπολογιστικές δυνατότητες. Η δεύτερη φάση υλοποιεί την επιλογή της αλυσίδας κατά τον χρόνο εκτέλεσης, αντιμετωπίζοντας την ανάθεση των επιπέδων ως ένα μεγάλο Directed Acyclic Graph (DAG). Μέσω ενός DHT, όλες οι συμμετέχουσες GPUs δημοσιεύουν συνεχώς τα ζωντανά προφίλ των καθυστερήσεων επεξεργασίας τους και των χρόνων μετ' επιστροφής (one-way round-trip timers - RTTs). Για κάθε client request, μια σάρωση δυναμικού προγραμματισμού του DAG κατασκευάζει δυναμικά μια βέλτιστη, ισορροπημένη ως προς τον φόρτο αλυσίδα εκτέλεσης, αποφεύγοντας υποβαθμισμένους, συμφορημένους ή ξαφνικά εκτός σύνδεσης κόμβους.

Το HexGen [42], προορίζεται για ετερογενή και cross-datacenter περιβάλλοντα, υποστηρίζοντας inference με tensor και pipeline parallelism. Εφαρμόζει αλγορίθμους non-uniform splitting για να αντιστοιχίσει sub-graphs του μοντέλου ανάλογα με τους περιορισμούς του εκάστοτε υλικού. Το ThunderServe [43] προορίζεται για κατακερματισμένα περιβάλλοντα νέφους, ενσωματώνοντας phase splitting (χωρίζοντας τα στάδια prefill και decoding) σε συνδυασμό με ετερογενή ενορχήστρωση, επιτυγχάνοντας σταθερά τα Service Level Objective (SLO) μέσω ενός ελαφρού μηχανισμού rescheduling, ο οποίος προσαρμόζει δυναμικά τους φόρτους εργασίας χωρίς εκτεταμένη διακοπή της λειτουργίας του συστήματος.

Ενώ τα συστήματα όπως Parallax, BloomBee και HexGen υποστηρίζουν την ετερογένεια των συμμετεχόντων κόμβων, πολλά από αυτά βασίζονται, σε κάποιον βαθμό, σε υπολογιστικά απαιτητικούς αλγορίθμους συντονισμού ή σε GPUs υψηλών επιδόσεων. Για την επίτευξη ενός αυτόνομου, ανθεκτικού σε σφάλματα συστήματος, το οποίο απευθύνεται σε συσκευές περιορισμένων πόρων πρέπει να συνδυαστούν συγκεκριμένοι θεμελιώδεις μηχανισμοί.

### 3.3.4 Θεμελιώδης Έμπνευση Αυτόνομου Κατανεμημένου Συμπερασμού

Οι στόχοι της διπλωματικής εργασίας απαιτούν μια δυναμική και ανθεκτική σε σφάλματα αλυσίδα εκτέλεσης ενός LLM, αξιοποιώντας ετερογενείς συσκευές ευρείας κατανάλωσης. Το σύστημα δεν επιτρέπει την παρουσία ενός κεντρικού συντονιστή και απαιτεί δυνατότητες αυτόνομης οργάνωσης και επισκευής. Η φιλοσοφία σχεδιασμού και η αρχιτεκτονική του συστήματος εμπνέονται από τρία βασικά frameworks: Petals [44], AR-MDI [45] και MDI-LLM [46].

#### 3.3.4.1 Petals

Το framework Petals, ήταν πρωτοπόρο με την εφαρμογή του swarm parallelism για LLMs, επιτρέποντας σε εθελοντές να φιλοξενήσουν συνεργατικά μέσω του διαδικτύου, μεγάλα μοντέλα (100B+ παραμέτρων) όπως το BLOOM και Llama.

Η βασικότερη δομική έμπνευση που αντλείται από το Petals είναι η χρήση ενός Distributed Hash Table (DHT) για την αποκεντρωμένη ανακάλυψη κόμβων και την αποθήκευση μεταδεδωμένων του δικτύου. Στο Petals, οι κόμβοι ανακοινώνουν περιοδικά τα ενεργά transformer blocks που φιλοξενούν και τοπικά χαρακτηριστικά απόδοσης σε ένα παγκοσμίως προσβάσιμο DHT, χρησιμοποιώντας την βιβλιοθήκη *hivemind* της Python. Όταν ένας client αρχικοποιεί μια συνεδρία inference, υποβάλλει ένα query στο DHT για να εντοπίσει διαθέσιμους κόμβους που εξυπηρετούν συλλογικά όλα τα απαραίτητα επίπεδα του μοντέλου. Για τη δημιουργία ενός βέλτιστου μονοπατιού, ο client κάνει ring κοντινούς server για να μετρήσει το network latency και χρησιμοποιεί beam search, ώστε να βρει την ακολουθία από servers με τον ελάχιστο συνολικό χρόνο επεξεργασίας.

Το DHT προσφέρει στο δίκτυο ανθεκτικότητα σε σφάλματα εγγενώς. Εάν ένας ενεργός κόμβος αποτύχει ή αποσυνδεθεί τα μεταδεδωμένα του, όπως η ακριβής ακολουθία από transformer layers που φιλοξενούσε, παραμένουν διαθέσιμα στο DHT και μπορούν να αξιοποιηθούν για την επισκευή του δικτύου.

Συμπληρωματικά, για την μείωση του communication overhead κατά την μεταφορά μεγάλων activation tensors μέσω αργών διασυνδέσεων, το Petals υλοποιεί 8-bit dynamic block-wise quantization στα hidden states πριν τη μετάδοση. Με αυτόν τον τρόπο μειώνεται το μέγεθος του μεταφερόμενου φορτίου κατά το ήμισυ με αμελητέα υποβάθμιση στην απόδοση του μοντέλου.

#### 3.3.4.2 AR-MDI

Το framework Adaptive and Resilient Model-Distributed Inference (AR-MDI), παρέχει τα αλγοριθμικά θεμέλια για την εξάλειψη του κεντρικού συντονιστή κατά την εξισορρόπηση φορτίου. Το AR-MDI διατυπώνει την κατανομή του μοντέλου ως ένα τοπικό πρόβλημα βελτιστοποίησης min-max.

$$\min_{A(k) \in S} \max_{\forall n} \{\max\{d_n(k), \tau_n(k)\}\}$$

Το framework προσπαθεί να εξισορροπήσει το combined computing delay  $d_n(k)$  και το transmission delay  $\tau_n(k)$  όλων των συμμετεχόντων κόμβων για την αποφυγή της στασιμότητας του pipeline.

Αντί να βασίζεται σε έναν κεντρικό κόμβο που να εκτελεί έναν αλγόριθμο MILP ή καθολικού δυναμικού προγραμματισμού, το AR-MDI παρουσιάζει έναν αποκεντρωμένο αλγόριθμο. Κάθε εργάτης υπολογίζει μια τοπική μετρική καθυστέρησης  $\gamma_n$  (per-parameter computing delay) με

βάση το προφίλ του υλικού του. Κατά την διάρκεια της εκτέλεσης, οι κόμβοι διαδίδουν αναδρομικά αυτές τις μετρικές σε γειτονικούς κόμβους στην αλυσίδα. Κάθε κόμβος υπολογίζει το βέλτιστο αριθμό παραμέτρων που πρέπει να επεξεργαστεί με τον τύπο:

$$\rho_n^*(k) = W \frac{1/\gamma_n(K)}{\sum 1/\gamma_n(K)}, \text{ όπου το } W \text{ είναι ο συνολικός αριθμός παραμέτρων του μοντέλου.}$$

Η μέθοδος αυτή προσαρμόζεται κατάλληλα στην διπλωματική για την εξισορρόπηση του αριθμού των transformer layers που φορτώνονται ανάμεσα στους συμμετέχοντες ενεργούς κόμβους.

### 3.3.4.3 MDI-LLM

Το Model-Distributed Inference for Large Language Models (MDI-LLM), γεφυρώνει το χάσμα ανάμεσα στο παραδοσιακό pipeline parallelism και την autoregressive φύση ενός LLM. Στο τυπικό pipeline parallelism οι downstream κόμβοι παραμένουν ανενεργοί όσο οι upstream κόμβοι επεξεργάζονται τα επίπεδα τους.

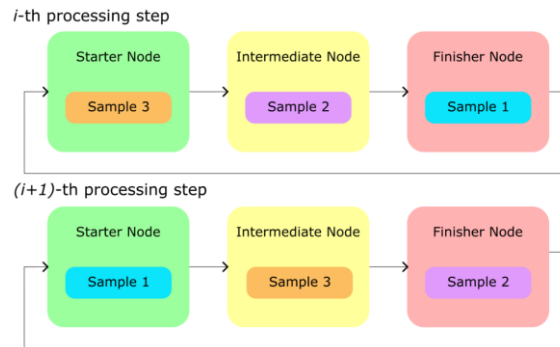


Figure 13. Recurrent pipeline parallelism του MDI-LLM framework. Πηγή: [46]

Το MDI-LLM εισάγει το “recurrent pipeline parallelism” για να επιλύσει αυτό το πρόβλημα. Σε αυτό το μοντέλο, όταν ένας κόμβος ολοκληρώσει την επεξεργασία του ξεκινάει αμέσως την επεξεργασία ενός νέου αιτήματος από διαφορετικό χρήστη. Διοχετεύοντας πολλαπλά αιτήματα από ταυτόχρονους χρήστες, οι κόμβοι δεν παραμένουν σε αδράνεια και ενισχύεται το συνολικό throughput.

Για την διαχείριση της KV cache των διαφορετικών ακολουθιών, το MDI-LLM χρησιμοποιεί Rotating KV Caches. Δηλαδή αποθηκεύει ξεχωριστά μητρώα Key και Value και εναλλάσσει τα «ενεργά» ανάλογα με την ακολουθία που επεξεργάζεται. Το σύστημα της διπλωματικής προσαρμόζει αυτή την τεχνική διαφοροποιώντας τις KV caches ανάλογα με τις διευθύνσεις των clients που έχουν πραγματοποιήσει τα αιτήματα συμπερασμού.

Παρά την αποτελεσματική εφαρμογή του recurrent pipeline parallelism, η υλοποίηση του MDI-LLM βασίζεται σε ένα “Starter Node” (κεντρικό συντονιστή), ο οποίος δέχεται τα prompts, διαχειρίζεται το δίκτυο των υπόλοιπων κόμβων και επιστρέφει το παραγόμενο κείμενο. Η διπλωματική εργασία αποκλίνει από αυτή την αρχιτεκτονική απόφαση για να εξασφαλίσει αποκεντρωμένη διαχείριση της τοπολογίας, διατηρώντας ωστόσο ένα μοναδικό σημείο εισόδου της ροής του inference.

### 3.4 Συγκριτική Αποτίμηση Προσεγγίσεων και Ερευνητικό Κενό

Η βιβλιογραφική ανασκόπηση των προηγούμενων ενοτήτων καταδεικνύει ότι η κατανεμημένη εκτέλεση Μεγάλων Γλωσσικών Μοντέλων έχει προσεγγιστεί μέσα από διαφορετικές αρχιτεκτονικές, οι οποίες επιχειρούν να αντιμετωπίσουν προβλήματα όπως οι υψηλές απαιτήσεις μνήμης, η ετερογένεια των διαθέσιμων υπολογιστικών πόρων, οι καθυστερήσεις επικοινωνίας και η αποτελεσματική κατανομή του υπολογιστικού φορτίου. Ωστόσο, οι υφιστάμενες προσεγγίσεις διαφοροποιούνται σημαντικά ως προς τον βαθμό αποκέντρωσης, τη δυναμική προσαρμογή της τοπολογίας και του φόρτου εργασίας, καθώς και την ικανότητά τους να διατηρούν τη λειτουργία του συστήματος σε περίπτωση μεταβολής ή αποτυχίας των συμμετεχόντων κόμβων.

Οι αρχιτεκτονικές edge-cloud split computing, όπως το EdgeShard [15], αξιοποιούν την υπολογιστική ισχύ υποδομών νέφους σε συνδυασμό με συσκευές άκρου, εφαρμόζοντας profiling και τεχνικές βελτιστοποίησης για την επιλογή του κατάλληλου σημείου κατάτμησης του μοντέλου. Παρότι οι προσεγγίσεις αυτές επιτρέπουν την εκτέλεση μεγάλων μοντέλων από συσκευές περιορισμένων δυνατοτήτων, εξακολουθούν να εξαρτώνται, πλήρως ή μερικώς, από εξωτερική υπολογιστική υποδομή και κατά συνέπεια δεν εξαλείφουν το cloud dependency [15]. Αντίθετα, προσεγγίσεις κατανεμημένης εκτέλεσης αποκλειστικά σε edge devices, όπως το Edge-LLM [37] και το LOIP [38], μεταφέρουν το σύνολο της διαδικασίας σε τοπικούς πόρους και επικεντρώνονται περισσότερο στη δυναμική ανάθεση εργασιών, στον προγραμματισμό της εκτέλεσης και στη διαχείριση περιορισμών μνήμης και δικτύου.

Παράλληλα, συστήματα που έχουν σχεδιαστεί για ετερογενείς συστοιχίες GPUs, όπως το Helix [39], επιτυγχάνουν αποτελεσματική κατανομή των επιπέδων του μοντέλου λαμβάνοντας υπόψη τις διαφορετικές υπολογιστικές και δικτυακές δυνατότητες των κόμβων. Ωστόσο, η βελτιστοποίηση της κατανομής βασίζεται σε κεντρικούς μηχανισμούς ενορχήστρωσης και σε υπολογιστικά απαιτητικές διαδικασίες βελτιστοποίησης. Η ύπαρξη ενός κεντρικού μηχανισμού διαχείρισης περιορίζει τον βαθμό αυτονομίας του δικτύου και μπορεί να δημιουργήσει ένα μοναδικό σημείο συντονισμού, γεγονός που διαφοροποιεί τέτοιες προσεγγίσεις από ένα περιβάλλον συνεργατικής εκτέλεσης με αποκεντρωμένο έλεγχο.

Ιδιαίτερα σημαντικές για την παρούσα εργασία είναι οι προσεγγίσεις Petals [44], AR-MDI [45] και MDI-LLM [46], καθώς παρέχουν συμπληρωματικούς μηχανισμούς που αξιοποιούνται και επεκτείνονται στην προτεινόμενη αρχιτεκτονική. Το Petals [44] αποτελεί χαρακτηριστικό παράδειγμα αποκεντρωμένης συνεργατικής εκτέλεσης μεγάλων μοντέλων, χρησιμοποιώντας ένα Distributed Hash Table για τη δημοσίευση και αναζήτηση των διαθέσιμων transformer blocks και των χαρακτηριστικών απόδοσης των συμμετεχόντων κόμβων. Η χρήση του DHT διευκολύνει τη λειτουργία σε δυναμικά περιβάλλοντα και παρέχει δυνατότητες ανάκαμψης σε περίπτωση αποσύνδεσης κόμβων. Επιπλέον, το Petals εφαρμόζει 8-bit dynamic block-wise quantization στις ενδιάμεσες ενεργοποιήσεις, μειώνοντας το communication overhead κατά τη μετάδοσή τους μεταξύ των κόμβων.

Το AR-MDI [45] προσεγγίζει διαφορετικά το πρόβλημα της ετερογένειας, εισάγοντας έναν αποκεντρωμένο μηχανισμό εξισορρόπησης του φόρτου. Αντί της εφαρμογής ενός κεντρικού

αλγορίθμου κατανομής, κάθε κόμβος υπολογίζει τοπικά δείκτες καθυστέρησης βάσει του προφίλ του υλικού του και ανταλλάσσει τις σχετικές πληροφορίες με γειτονικούς κόμβους. Με αυτόν τον τρόπο, η κατανομή του υπολογιστικού φορτίου μπορεί να προσαρμόζεται στις διαφορετικές επιδόσεις των συμμετεχουσών συσκευών χωρίς την ανάγκη ενός κεντρικού συντονιστή.

Τέλος, το MDI-LLM [46] αντιμετωπίζει την υποχρησιμοποίηση των κόμβων που εμφανίζεται στο συμβατικό pipeline parallelism μέσω της τεχνικής του recurrent pipeline parallelism. Με την ταυτόχρονη διοχέτευση αιτημάτων από διαφορετικούς χρήστες και τη χρήση Rotating KV Caches, επιτυγχάνεται καλύτερη αξιοποίηση των διαδοχικών σταδίων της αλυσίδας και αύξηση του συνολικού throughput. Ωστόσο, η αρχιτεκτονική του MDI-LLM εξακολουθεί να χρησιμοποιεί έναν Starter Node ως κεντρικό σημείο διαχείρισης των αιτημάτων και συντονισμού των συμμετεχόντων κόμβων [46].

Ο παρακάτω πίνακας συνοψίζει τις βασικές διαφοροποιήσεις μεταξύ των κυριότερων προσεγγίσεων που συνδέονται άμεσα με την αρχιτεκτονική της παρούσας εργασίας.

Table 1. Συγκριτική αποτίμηση βασικών προσεγγίσεων κατανεμημένης εκτέλεσης LLMs.

| Χαρακτηριστικό                                            | Petals [44]  | AR-MDI [45] | MDI-LLM [46] | Προτεινόμενο Σύστημα |
|-----------------------------------------------------------|--------------|-------------|--------------|----------------------|
| Κατανεμημένη εκτέλεση επιπέδων μοντέλου                   | ✓            | ✓           | ✓            | ✓                    |
| Αποκεντρωμένος συντονισμός                                | ✓            | ✓           | ✗            | ✓                    |
| Απουσία μόνιμου κεντρικού συντονιστή                      | ✓            | ✓           | ✗            | ✓                    |
| Υποστήριξη ετερογενών υπολογιστικών πόρων                 | ✓            | ✓           | ✓            | ✓                    |
| Δυναμική εξισορρόπηση φορτίου βάσει απόδοσης κόμβων       | Περιορισμένη | ✓           | Περιορισμένη | ✓                    |
| Δυναμική ανακατανομή transformer layers κατά την εκτέλεση | Περιορισμένη | ✓           | ✗            | ✓                    |
| Αυτόματη ανάκαμψη από αποτυχία κόμβου                     | ✓            | ✓           | Περιορισμένη | ✓                    |
| Χρήση εφεδρικών κόμβων για αντικατάσταση ενεργών κόμβων   | ✗            | –           | ✗            | ✓                    |

|                                                          |                  |   |   |   |
|----------------------------------------------------------|------------------|---|---|---|
| Ευκαιριακή αντικατάσταση αδύναμου ενεργού κόμβου         | X                | X | X | ✓ |
| Ταυτόχρονη εξυπηρέτηση πολλαπλών clients μέσω pipelining | –                | – | ✓ | ✓ |
| Rotating / ξεχωριστές KV caches ανά client               | –                | – | ✓ | ✓ |
| 8-bit quantization ενδιάμεσων ενεργοποιήσεων             | ✓                | X | X | ✓ |
| Σχεδιασμός για αυτόνομη λειτουργία σε τοπικό δίκτυο      | Όχι αποκλειστικά | ✓ | ✓ | ✓ |

Όπως προκύπτει από τη συγκριτική αποτίμηση, η υφιστάμενη βιβλιογραφία παρέχει επιμέρους λύσεις για τα βασικά προβλήματα της κατανεμημένης εκτέλεσης LLMs, χωρίς όμως να ενοποιεί πλήρως τις απαιτήσεις της αποκέντρωσης, της δυναμικής εξισορρόπησης φορτίου, της ανθεκτικότητας σε σφάλματα, της ετερογένειας του υλικού και της ταυτόχρονης εξυπηρέτησης πολλαπλών χρηστών σε μία ενιαία αρχιτεκτονική προσανατολισμένη σε τοπικά δίκτυα.

Η παρούσα εργασία επιχειρεί να καλύψει αυτό το κενό συνδυάζοντας και επεκτείνοντας επιμέρους ιδέες των παραπάνω προσεγγίσεων. Από το Petals υιοθετείται η χρήση ενός DHT για αποκεντρωμένη διαχείριση της κατάστασης του δικτύου και η κβαντοποίηση των ενδιάμεσων ενεργοποιήσεων. Από το AR-MDI προσαρμόζεται η λογική της κατανεμημένης αξιολόγησης της απόδοσης των κόμβων και της δυναμικής εξισορρόπησης του φόρτου. Από το MDI-LLM αξιοποιείται η λογική του recurrent pipelining και των ξεχωριστών KV caches για την ταυτόχρονη εξυπηρέτηση πολλαπλών clients.

Πέρα από τον συνδυασμό αυτών των χαρακτηριστικών, η προτεινόμενη αρχιτεκτονική εισάγει επιπλέον μηχανισμούς Δυναμικής Ανακατανομής των transformer layers, αυτόματης Επισκευής της Αλυσίδας εκτέλεσης και Ευκαιριακής Ανάληψης, μέσω της οποίας ένας διαθέσιμος εφεδρικός κόμβος μπορεί να αντικαταστήσει έναν ενεργό κόμβο χαμηλότερης απόδοσης. Με αυτόν τον τρόπο, το σύστημα δεν προσαρμόζεται μόνο σε μόνιμες διαφορές μεταξύ των συσκευών, αλλά και σε μεταβολές της διαθέσιμης υπολογιστικής ισχύος και σε απρόβλεπτες αποτυχίες κατά τη διάρκεια της εκτέλεσης.

Συνεπώς, το ερευνητικό κενό που αντιμετωπίζεται στην παρούσα διπλωματική αφορά την ανάπτυξη μιας ενιαίας, αυτοπροσαρμοζόμενης αρχιτεκτονικής συνεργατικού LLM inference με αποκεντρωμένη διαχείριση, ικανής να αξιοποιεί ετερογενείς συσκευές ευρείας κατανάλωσης σε τοπικό δίκτυο, να ανακατανέμει δυναμικά το υπολογιστικό φορτίο, να εξυπηρετεί παράλληλα πολλαπλούς χρήστες και να διατηρεί τη λειτουργία της παρά την υποβάθμιση της απόδοσης ή την αποτυχία μεμονωμένων κόμβων.

## 4 Αρχιτεκτονική Συστήματος

### 4.1 Επισκόπηση Αρχιτεκτονικής

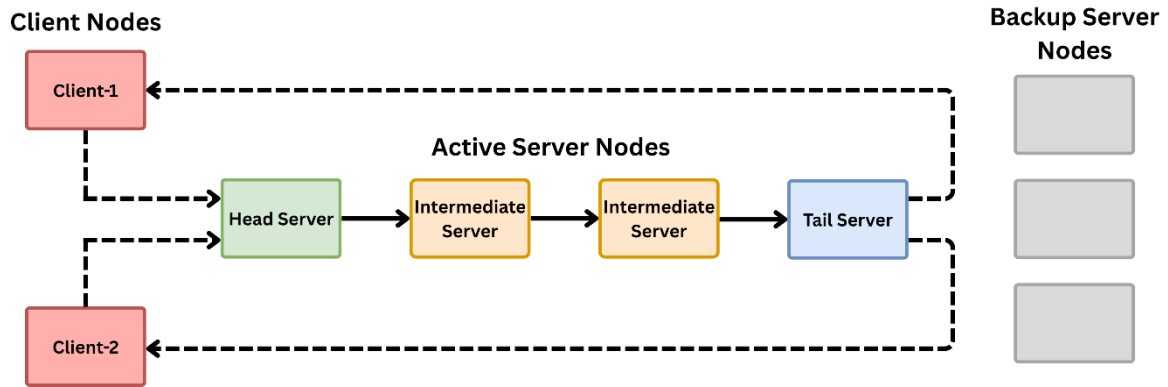


Figure 14. Παράδειγμα τοπολογίας του συστήματος με δυο client nodes, τρεις active server nodes και τρεις Backup nodes.

Στην αρχιτεκτονική του συστήματος της διπλωματικής συμμετέχουν δύο είδη κόμβων: servers nodes και client nodes. Τα επίπεδα του LLM μοιράζονται στους server nodes, δημιουργώντας μια αλυσίδα συμπερασμού (inference chain). Οι client nodes στέλνουν αιτήματα παραγωγής κειμένου στην αλυσίδα και λαμβάνουν τα παραγόμενα tokens. Κάθε κόμβος συμμετέχει σε ένα DHT, στο οποίο δημοσιεύει μεταδεδομένα σχετικά με την κατάστασή του, επιτρέποντας την αποκεντρωμένη παρακολούθηση, διαχείριση και βελτιστοποίηση του δικτύου.

Οι server nodes χωρίζονται σε ενεργούς (active) και εφεδρικούς (backups). Οι ενεργοί κόμβοι φορτώνουν μια ακολουθία από transformer layers ακολουθώντας την τεχνική του pipeline parallelism. Αφού φορτωθούν όλα τα transformer layers και το τελικό output layer στους ενεργούς κόμβους, δημιουργείται η αλυσίδα συμπερασμού και το σύστημα είναι έτοιμο για την παραγωγή κειμένου. Ανάλογα με την θέση τους στην αλυσίδα οι ενεργοί κόμβοι αποκτούν έναν από τρεις ρόλους: Head, Intermediate και Tail.

Ως Backup nodes ορίζονται οι κόμβοι που εισέρχονται στο δίκτυο αφού έχουν φορτωθεί όλα τα επίπεδα του μοντέλου. Δεν συμμετέχουν στην παραγωγή κειμένου και παραμένουν σε αναμονή για την επισκευή της αλυσίδας σε περίπτωση σφάλματος ή για την αντικατάσταση ενός αδύναμου υπολογιστικά ενεργού κόμβου.

Οι client nodes αποτελούν τους κόμβους-χρήστες του δικτύου, φορτώνουν το input layer του μοντέλου, στέλνουν αιτήματα συμπερασμού στην κεφαλή (Head) της αλυσίδας εκτέλεσης και λαμβάνουν τα παραγόμενα tokens από την ουρά (Tail) της αλυσίδας. Υπάρχει δυνατότητα παραγωγής κειμένου από πολλούς clients ταυτόχρονα, αυξάνοντας την συνολική διεκπεραιωτική ικανότητα του συστήματος μέσω της τεχνικής του recurrent pipeline parallelism.

## 4.2 Αρχιτεκτονική Server Node

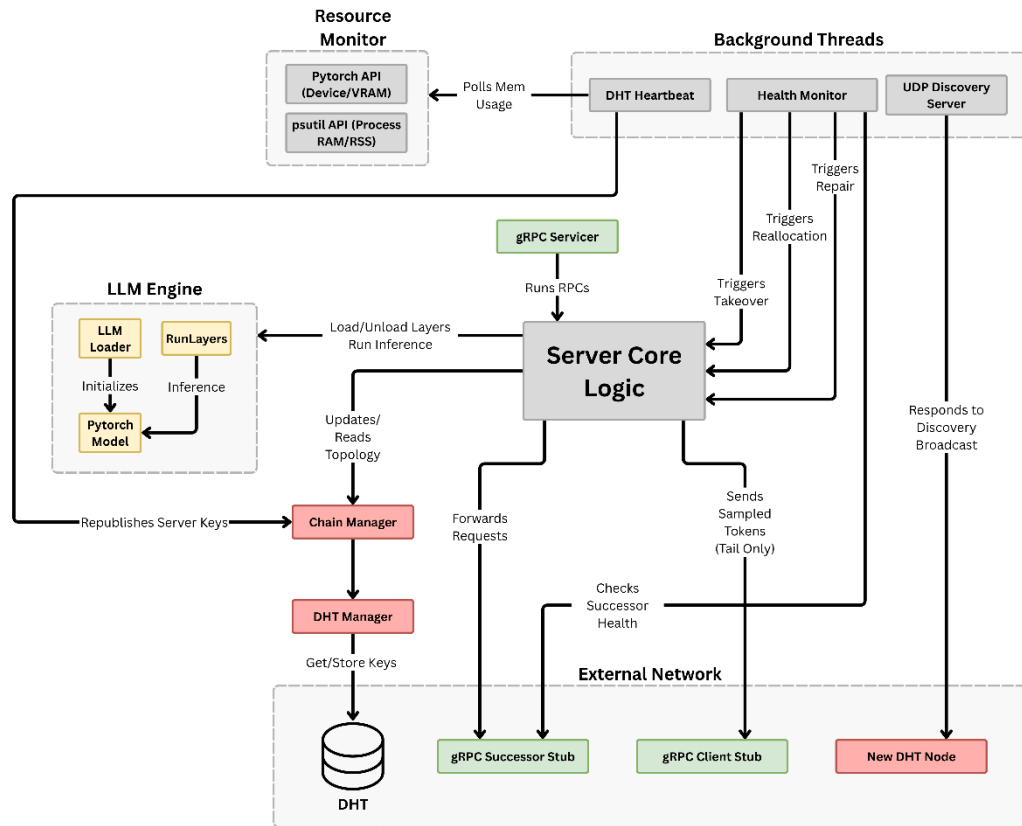


Figure 15. Διάγραμμα λογικής αρχιτεκτονικής ενός server node.

Ο server node αποτελεί την θεμελιώδη υπολογιστική μονάδα του συστήματος, υπεύθυνη για την εκτέλεση ενός μέρους του συμπερασμού και την διαχείριση της υποκείμενης τοπολογίας κόμβων. Κάθε κόμβος είναι σχεδιασμένος ως ένας αυτόνομος εργάτης που εκτελεί self-profiling και έχει την ικανότητα να προσαρμόζει δυναμικά τον φόρτο εργασίας του με βάση τους περιορισμούς του και τις εκάστοτε συνθήκες του δικτύου.

Η εσωτερική αρχιτεκτονική ενός server node, διαχωρίζεται στην διαχείριση της P2P τοπολογίας και την υποκείμενη μηχανή deep learning inference. Αυτό επιτυγχάνεται μέσω τεσσάρων κύριων υποσυστημάτων: της Διαχείρισης Κατανεμημένης Κατάστασης, της Δυναμικής Αξιολόγηση Πόρων, της Μηχανής Συμπερασμού και των Μηχανισμών Ανοχής Σφαλμάτων και Ανακατανομής Επιπέδων.

### 4.2.1 Διαχείριση Κατανεμημένης Κατάστασης και Τοπολογίας

Για την διατήρηση της σταθερότητας της αλυσίδας εκτέλεσης, χωρίς εξάρτηση από έναν κεντρικό συντονιστή, η αρχιτεκτονική αξιοποιεί ένα DHT.

Η μονάδα DHT Manager δρα ως διεπαφή με το P2P overlay, περικλείοντας την υποκείμενη υλοποίηση DHT που βασίζεται στο πρωτόκολλο Kademlia. Είναι υπεύθυνη για την ανακάλυψη κόμβων, την είσοδο του server στο P2P δίκτυο και την αποθήκευση και λήψη κλειδιών-τιμών στο DHT.

Η μονάδα Chain Manager υπαγορεύει την λογική τοπολογία του κόμβου. Κατά την αρχικοποίηση, ο Chain Manager αξιολογεί την καθολική κατάσταση του δικτύου, που είναι αποθηκευμένη στο DHT, για να καθορίσει την θέση του κόμβου στην αλυσίδα και τον ρόλο του (Head, Intermediate, Tail, ή Backup). Παρακολουθεί και ενημερώνει συνεχώς τόσο τα καθολικά κλειδιά, όσο και τα τοπικά υποκλειδιά (subkeys) του κόμβου.

#### 4.2.2 Δυναμική Αξιολόγηση Πόρων

Μια κρίσιμη απαίτηση για την λειτουργία με ετερογενές υλικό είναι η ικανότητα αυτόνομης αξιολόγησης της τοπικής υπολογιστικής ικανότητας. Ο server node υλοποιεί δυναμική αξιολόγηση για την αποφυγή σφαλμάτων out-of-memory (OOM) και τη βελτίωση της κατανομής των επιπέδων. Συγκεκριμένα ο κόμβος εξετάζει συνεχώς τη διαθέσιμη μνήμη του (RAM/VRAM) μέσω των psutil και torch.cuda APIs. Οι πληροφορίες αυτές αποθηκεύονται στο DHT και χρησιμοποιούνται για τον καθορισμό του μέγιστου φόρτου εργασίας που μπορεί να αναλάβει ένας κόμβος.

Συμπληρωματικά, με την ένταξή του στο δίκτυο, ο κόμβος εκτελεί μια τοπική διαδικασία benchmarking, χρησιμοποιώντας dummy input tensors. Με αυτόν τον τρόπο υπολογίζεται το inference delay του και εγκαθιδρύεται ένα βασικό processing rate (layers / sec), το οποίο στη συνέχεια δημοσιεύεται στο DHT και χρησιμοποιείται για την εξισορρόπηση του φόρτου εργασίας.

#### 4.2.3 Μηχανή Συμπερασμού

Η μονάδα αυτή αποτελεί το επίπεδο εκτέλεσης της αρχιτεκτονικής και γεφυρώνει τη στοίβα δικτύου με το PyTorch backend. Ο κόμβος φορτώνει δυναμικά μια συνεχόμενη ακολουθία από transformer layers που του έχουν ανατεθεί (και προαιρετικά το τελικό επίπεδο εξόδου) στην μνήμη. Η ανταλλαγή tensors μεταξύ των κόμβων διεκπεραιώνεται μέσω καναλιών gRPC. Μόλις ένας κόμβος ολοκληρώσει ένα τοπικό forward pass, τα παραγόμενα intermediate tensors σειριοποιούνται και μεταδίδονται απευθείας στο gRPC stub του επόμενου κόμβου.

Για την υποστήριξη ταυτόχρονων αιτημάτων η μονάδα αυτή διατηρεί ξεχωριστές KV caches που αντιστοιχίζονται σε μοναδικά αναγνωριστικά πελατών, διασφαλίζοντας την διατήρηση του context μεταξύ αιτημάτων από διαφορετικές πηγές.

#### 4.2.4 Ανοχή Σφαλμάτων και Ανακατανομή Επιπέδων

Για να διασφαλιστεί υψηλή διαθεσιμότητα εν μέσω αποτυχιών των κόμβων, η αρχιτεκτονική ενσωματώνει πρωτόκολλα συνεχούς ασύγχρονης παρακολούθησης και αυτοεπιδιόρθωσης.

Μια διεργασία παρασκηνίου εκτελεί στοχευμένα gRPC αιτήματα *Check* προς τον άμεσο επόμενο κόμβο. Εάν επιβεβαιωθεί ότι ο επόμενος κόμβος είναι ανενεργός, ξεκινά ένα τοπικό πρωτόκολλο επιδιόρθωσης χρησιμοποιώντας είτε τον εαυτό του είτε είναι έναν εφεδρικό κόμβο για την ανάληψη των «ορφανών» επιπέδων.

Παράλληλα, ο κόμβος παρακολουθεί ενεργά την υπολογιστική ισορροπία της αλυσίδας. Εάν εντοπιστούν σοβαρά σημεία συμφόρησης, ή εάν ένας εφεδρικός κόμβος εντοπίσει έναν ενεργό κόμβο με σημαντικά ασθενέστερο ρυθμό επεξεργασίας, το σύστημα ενεργοποιεί μια καθολική Ανακατανομή (reallocation) ή μια Ευκαιριακή Ανάληψη (opportunistic takeover). Τα transformer layers ανακατανέμονται μαθηματικά, σε αναλογία με τον ρυθμό επεξεργασίας που έχει αξιολογηθεί για κάθε κόμβο σε σχέση με τον συνολικό ρυθμό επεξεργασίας του συστήματος, διασφαλίζοντας ότι οι ετερογενείς ταχύτητες επεξεργασίας δεν προκαλούν ανισορροπία στο pipeline.

### 4.3 Αρχιτεκτονική Client Node

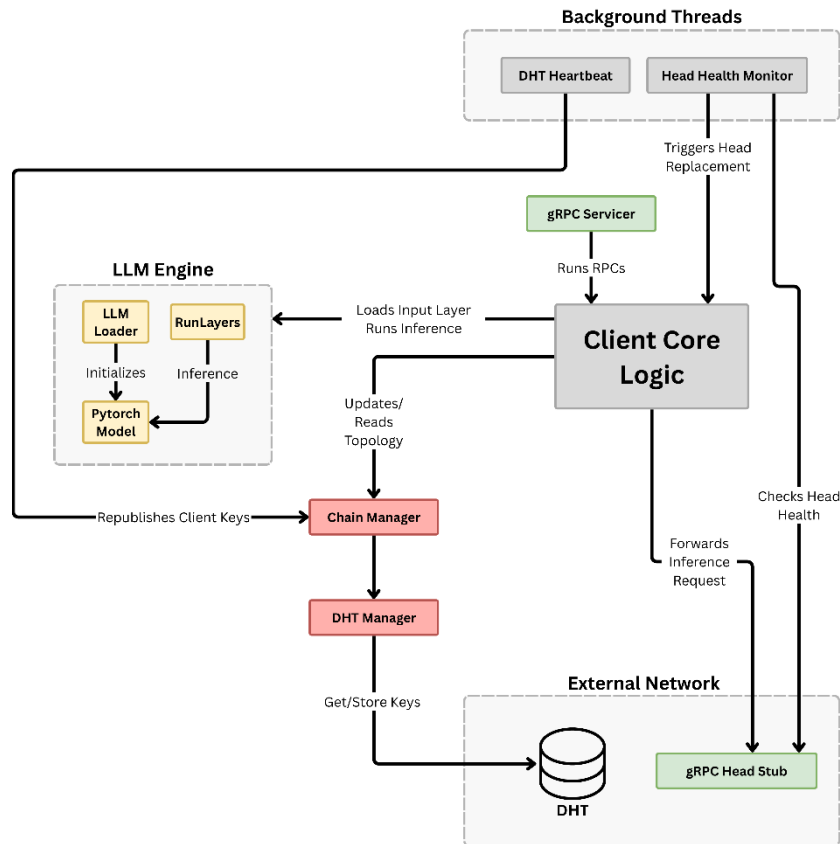


Figure 16. Διάγραμμα λογικής αρχιτεκτονικής ενός client node.

Ο client node λειτουργεί ως σημείο αλληλεπίδρασης μεταξύ του τελικού χρήστη και της κατανεμημένης αλυσίδας συμπερασμού. Σε αντίθεση με τους server nodes, ο client δεν αναλαμβάνει το βαρύ υπολογιστικό φόρτο των ενδιαμέσων transformer layers και του τελικού output layer. Είναι υπεύθυνος για την εκκίνηση της διαδικασίας παραγωγής κειμένου, αναλαμβάνοντας το επίπεδο εισόδου του μοντέλου και την επίβλεψη της ακεραιότητας του Head server.

Η αρχιτεκτονική του είναι σχεδιασμένη ώστε να είναι ελαφριά και αποδοτική, επιτρέποντας την εκτέλεση σε συσκευές με περιορισμένους πόρους. Αποτελείται από τρία κύρια υποσυστήματα: Τοπική Επεξεργασία, Διαχείριση Κατάστασης και Εποπτεία του Head Server.

#### 4.3.1 Τοπική Επεξεργασία και Δρομολόγηση Συμπερασμού

Ο client node διαχειρίζεται αποκλειστικά τον tokenizer και τα αρχικά επίπεδα του μοντέλου. Κατά την αρχικοποίηση φορτώνει μόνο το embedding layer του LLM στην μνήμη του, αφήνοντας τα transformer layers και το τελικό επίπεδο εξόδου στους server nodes.

Ο client λειτουργεί ως η αρχή και το τέλος του κατανεμημένου inference pipeline. Αρχικά, τα τοπικά παραγόμενα embeddings σειριοποιούνται και αποστέλλονται στον Head server. Στη συνέχεια, ο client δέχεται ασύγχρονα το παραγόμενο token από τον Tail server, το οποίο αποκωδικοποιείται και επιστρέφεται στον χρήστη.

### 4.3.2 Διαχείριση Κατάστασης

Ο client συνδέεται στο P2P δίκτυο μέσω του DHT Manager και καταχωρεί την παρουσία του στο καθολικό κλειδί “*client\_nodes*”, το οποίο ανανεώνεται τακτικά μέσω ενός background thread.

Κατά την εκκίνηση και τον τερματισμό του, ο client ενημερώνει το δίκτυο για τον συνολικό αριθμό των ενεργών clients. Αυτό επιτρέπει στους server nodes να αναπροσαρμόσουν δυναμικά το μέγιστο μήκος ακολουθίας ανά πελάτη, αποτρέποντας την υπερχειλίση μνήμης λόγω ανεξέλεγκτης αύξησης της KV cache.

### 4.3.3 Εποπτεία και Επιδιόρθωση Head Server

Ενώ οι server nodes επιβλέπουν τους successors τους, οι clients φέρουν την ευθύνη παρακολούθησης της υγείας του Head server. Αυτό επιτυγχάνεται μέσω μίας διεργασίας παρασκηνίου που πραγματοποιεί τακτικούς ελέγχους στο Head server gRPC stub. Για να αποφευχθεί ο ανταγωνισμός σε περίπτωση που πολλαπλοί clients εντοπίσουν την πτώση του Head, εκλέγεται ντετερμινιστικά ένας client - ηγέτης για να εκκινήσει την διαδικασία επιδιόρθωσης. Για την επιδιόρθωση, ο client - ηγέτης αναθέτει τα «ορφανά» επίπεδα στον επόμενο διαθέσιμο κόμβο της αλυσίδας ή σε κάποιο εφεδρικό κόμβο.

## 4.4 Διαχείριση Τοπολογίας - DHT

Στο πλαίσιο του συστήματος της διπλωματικής κάθε συσκευή αποτελεί και κόμβο ενός DHT. Το DHT διαδραματίζει τον ρόλο του κεντρικού μηχανισμού συντονισμού. Έτσι, αντί να υπάρχει ένας κεντρικός κόμβος ο οποίος ελέγχει ποιος server node αναλαμβάνει τα εκάστοτε επίπεδα του μοντέλου, οι κόμβοι χρησιμοποιούν το DHT για να παρακολουθούν και να ενημερώνουν συλλογικά την τοπολογία της αλυσίδας εκτέλεσης.

Στους παρακάτω πίνακες σημειώνονται τα ζεύγη key-value που χρησιμοποιούνται στο DHT.

Table 2. DHT Global Keys.

| Global Keys        |                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------|
| Keys               | Values                                                                                       |
| “chain_head”       | node_id (str)                                                                                |
| “chain_tail”       | node_id (str)                                                                                |
| “chain_leader”     | node_id (str)                                                                                |
| “chain_status”     | {“READY”   “UNREADY”   “RUNNING”   “PROFILING”   “REPAIRING”   “REAL-LOCATING”   “TAKEOVER”} |
| “num_total_layers” | int                                                                                          |
| “num_total_params” | int                                                                                          |

|                         |                                                          |
|-------------------------|----------------------------------------------------------|
| “all_layers_loaded”     | bool                                                     |
| “backup_nodes”          | {id1 (str): bool, id2 (str): bool, ..., idn (str): bool} |
| “client_nodes”          | {id1 (str): bool, id2 (str): bool, ..., idn (str): bool} |
| “server_info_(node_id)” | Dict[subkey (str), value (Any)]                          |

Table 3. DHT Server Node Subkeys.

| Server Node Subkeys<br>“server_info_(node_id)” |                              |
|------------------------------------------------|------------------------------|
| Subkey                                         | Value                        |
| “id”                                           | str                          |
| “is_backup”                                    | bool                         |
| “layers”                                       | tuple[int, int]              |
| “layers_loaded”                                | bool                         |
| “load_output_layer”                            | bool                         |
| “output_layer_loaded”                          | bool                         |
| “output_layer_temporal_tle”                    | float                        |
| “output_layer_memory_tle”                      | float                        |
| “hostname”                                     | str, π.χ. “head-server”      |
| “address”                                      | str, π.χ. “head-server:5001” |
| “device”                                       | {“cuda”   “cpu”}             |
| “memory_usage”                                 | float                        |
| “memory_limit”                                 | float                        |
| “available_memory”                             | float                        |
| “vram_usage”                                   | float                        |
| “vram_limit”                                   | float                        |
| “available_vram”                               | float                        |

|                     |                                                              |
|---------------------|--------------------------------------------------------------|
| “kv_cache_memories” | {client1_addr (str): float, ..., clientn_addr (str): float } |
| “successor”         | {“id”: node_id (str), “address”: succ_addr (str)}            |
| “processing_rate”   | float                                                        |
| “inference_delay”   | float                                                        |

#### 4.4.1 Keys-Values

Στο DHT αποθηκεύονται global keys και subkeys για κάθε ξεχωριστό server node. Τα global keys παρέχουν μεταδεδομένα για το συνολικό δίκτυο, όπως δείκτες στο node id της κεφαλής (“chain\_head”) και της ουράς (“chain\_tail”) της αλυσίδας, καθώς και για την τρέχουσα κατάσταση της (“chain\_status”). Για την αποφυγή race conditions, η κατάσταση της αλυσίδας ενημερώνεται από έναν κόμβο-ηγέτη, ο οποίος αλλάζει τακτικά και ορίζεται μέσω του κλειδιού “chain\_leader”. Επιπλέον, διατηρούνται dictionaries με τους client nodes και τους backup nodes, αξιοποιώντας την δυνατότητα αποθήκευσης subkeys της βιβλιοθήκης hivemind. Συγκεκριμένα τα global keys “backup\_nodes” και “client\_nodes” περιέχουν subkeys με τα ids των εφεδρικών και των κόμβων-χρηστών με λογικές τιμές True/False. Για παράδειγμα όταν ένας κόμβος ορίζεται ως εφεδρικός αποθηκεύει την παρακάτω τιμή στο DHT.

```
dht.store(key="backup_nodes", subkey=self.node_id, value=True)
```

Συμπληρωματικά, στο DHT αποθηκεύονται κλειδιά της μορφής “server\_info\_(node\_id)”, όπου οι αντίστοιχες τιμές είναι dictionaries με μεταδεδομένα για συγκεκριμένους server nodes. Με αυτόν τον τρόπο, αποθηκεύονται πληροφορίες όπως η ακολουθία των transformer layers που έχει φορτώσει ένας server node, η διαθέσιμη μνήμη του, ο ρυθμός επεξεργασίας του κ.α.

#### 4.4.2 Εισαγωγή Νέων DHT Κόμβων

Όταν ένας νέος server node επιχειρήσει να εισέλθει στο δίκτυο, ελέγχει αρχικά αν υφίσταται ήδη κάποιο ενεργό DHT, πραγματοποιώντας UDP broadcasting με σκοπό την εύρεση ενός bootstrap node. Αν δεν βρεθεί κάποιο DHT, δημιουργείται η αρχική αλυσίδα, ο κόμβος ορίζεται ως Head και Tail της αλυσίδας και φορτώνει όσα transformer layers του επιτρέπει η μνήμη του. Διαφορετικά ανακτώνται οι πληροφορίες του κόμβου Tail και ο νέος server προστίθεται στο τέλος της αλυσίδας ενημερώνοντας, το successor subkey του προηγούμενου Tail. Στη συνέχεια, φορτώνει όσα επίπεδα μπορεί, συνεχίζοντας από το τελευταίο επίπεδο του predecessor του. Σε περίπτωση που ένας νέος κόμβος προστίθεται στο τέλος της αλυσίδας και έχουν ήδη φορτωθεί όλα τα transformer layers στους υπόλοιπους ενεργούς κόμβους, φορτώνει το τελικό output layer. Αξίζει να σημειωθεί ότι η αρχιτεκτονική επιλογή να ανατεθεί το επίπεδο εξόδου του LLM σε server node αντί για τον client node, έγινε επειδή αποτελείται από υπολογιστικά βαριά στοιχεία και θα επιβάρυνε υπέρμετρα τους client nodes, σε αντίθεση με το επίπεδο εισόδου.

Αντίστοιχα, οι client nodes, αφού εισέλθουν στο DHT, αναζητούν την διεύθυνση IP του Head server, μέσω του DHT για να συνδεθούν μαζί του και να μπορέσουν να ξεκινήσουν την διαδικασία παραγωγής κειμένου.

#### **4.4.3 Ανθεκτικότητα σε Σφάλματα**

Κάθε φορά που αποθηκεύεται ένα key-value στο DHT, ορίζεται ένας χρόνος λήξης (expiration time). Επομένως, οι πληροφορίες του DHT πρέπει να ανανεώνονται τακτικά από τον κόμβο που τις δημοσίευσε. Μέχρι την παρέλευση αυτού του χρόνου λήξης, οι πληροφορίες παραμένουν διαθέσιμες στο DHT, ακόμα και αν ο κόμβος που τις αποθήκευσε αρχικά έχει τεθεί εκτός λειτουργίας. Αυτή η ιδιότητα επιτρέπει στους υπόλοιπους κόμβους να ανακτήσουν τα μεταδεδωμένα ενός αποτυχημένου κόμβου και να πυροδοτήσουν μηχανισμούς επισκευής της αλυσίδας.

## 5 Προσαρμοσμένη Υλοποίηση και Κατανεμημένη Εκτέλεση

### 5.1 Προσαρμοσμένη Υλοποίηση Llama3

Το κατανεμημένο σύστημα εκτέλεσης της διπλωματικής απαιτεί τροποποιήσεις στην Pytorch υλοποίηση της αρχιτεκτονικής των μοντέλων Llama3 (*model.py*), ώστε να επιτρέπεται η δυναμική φόρτωση και εκτέλεση συγκεκριμένων επιπέδων του μοντέλου και να διατηρούνται ξεχωριστές KV caches για κάθε client.

#### 5.1.1 Κατάτμηση Μοντέλου και Δυναμική Φόρτωση

Η κύρια τροποποίηση στην αρχιτεκτονική επιτρέπει σε μεμονωμένους κόμβους να φορτώνουν μόνο συγκεκριμένα τμήματα του νευρωνικού δικτύου. Συγκεκριμένα ο constructor της κλάσης *Llama3* εισάγει τις παραμέτρους *load\_initial\_layer*, *layers\_to\_load* και *load\_output\_layer*, επιτρέποντας σε έναν κόμβο να φορτώσει μόνο το embedding layer, μια συγκεκριμένη ακολουθία από transformer layers ή/και τα τελικά output layers (*RMSNorm* και *lm\_head*).

#### 5.1.2 Τμηματοποιημένη Εκτέλεση

Συμπληρωματικά, η τυπική μέθοδος *forward* της Pytorch χωρίζεται σε διακριτές φάσεις εκτέλεσης (*forward\_client\_initial* και *forward\_server*) ανάλογα με το είδος του κόμβου που εκτελεί το inference.

Η μέθοδος *forward\_client\_initial* εκτελείται σε client nodes και υπολογίζει τα αρχικά token embeddings από τα input IDs του prompt.

Η μέθοδος *forward\_server* εκτελείται από ενεργούς server nodes και επεξεργάζεται την ακολουθία από transformer layers που έχουν φορτώσει και προαιρετικά τα τελικά επίπεδα *RMSNorm* και *lm\_head* για την παραγωγή των logits.

#### 5.1.3 Διαχείριση KV Cache Πολλαπλών Χρηστών

Σε ένα αποκεντρωμένο περιβάλλον, οι ενεργοί servers πρέπει να επεξεργάζονται inference requests από πολλαπλούς clients ταυτόχρονα, χωρίς συγκρούσεις μνήμης. Η αρχιτεκτονική της KV cache έχει τροποποιηθεί για να χειρίζεται πολλαπλά contexts δυναμικά.

Συγκεκριμένα, το module *CausalSelfAttention* διατηρεί ένα λεξικό (*kv\_caches*) που αντιστοιχίζει ένα μοναδικό αναγνωριστικό του client (*client\_id*) σε ένα αποκλειστικό αντικείμενο *KVCache*. Η εναλλαγή της ενεργής KV cache για το επερχόμενο forward pass πραγματοποιείται μέσω της μεθόδου *set\_active\_client*, η οποία ορίζει το κλειδί του λεξικού ανάλογα με το αναγνωριστικό του client, του οποίου ο server επεξεργάζεται το αίτημα.

Για την αποφυγή σφαλμάτων out-of-memory, το συνολικό memory footprint των KV caches περιορίζεται από μια σταθερά *GLOBAL\_MAX\_SEQ\_LEN*, η οποία ορίζει το μέγιστο μήκος ακολουθίας που μπορούν να παράξουν οι server nodes για όλους τους clients. Μέθοδοι όπως οι *add\_client\_cache*, *remove\_client\_cache* και *reallocate\_caches*, μοιράζουν δυναμικά αυτό το παγκόσμιο όριο μεταξύ όλων των τρεχόντων συνδεδεμένων clients. Για παράδειγμα αν το

`GLOBAL_MAX_SEQ_LEN` είναι 8192 και έχουμε δύο συνδεδεμένους clients, ο καθένας μπορεί να παράξει έως και 4096 tokens και το μέγεθος της KV cache που αναλογεί στον καθένα τους αρχικοποιείται ανάλογα.

## 5.2 LLM Loader API

Το module `llm_loader.py` καθιερώνει το θεμελιώδες API για την αρχικοποίηση και την διαχείριση βαρών του μοντέλου Llama3. Η χρήση του διαφοροποιείται ανάλογα με τον ρόλο του εκάστοτε κόμβου (server ή client) στο συνεργατικό δίκτυο.

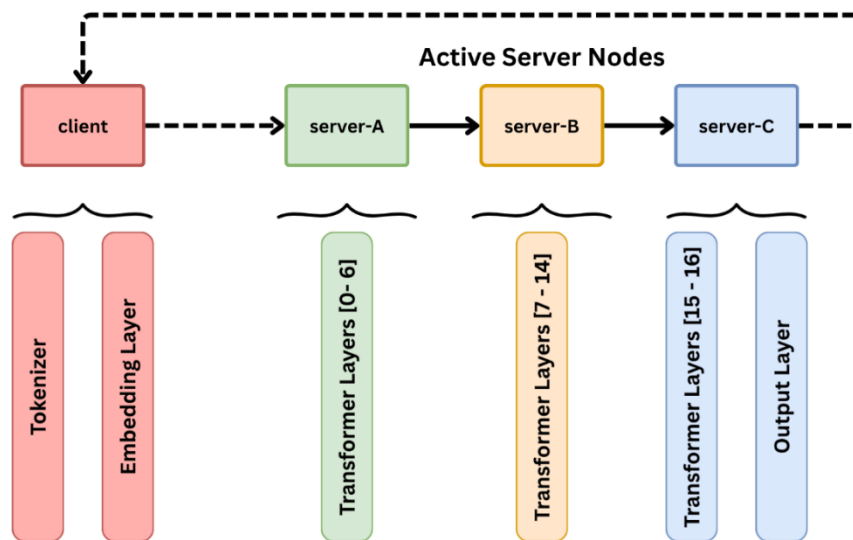


Figure 17. Παράδειγμα ανάθεσης επιπέδων του μοντέλου Llama 3.2 1B.

### 5.2.1 Χρήση από Server Nodes

Ένας server node χρησιμοποιεί την μέθοδο κλάσης `LLM.load` για να αρχικοποιήσει το μοντέλο του στον καλύτερο διαθέσιμο επιταχυντή (*cuda* ή *cpu*) και να φορτώσει τα βάρη των τμημάτων που του έχουν ανατεθεί, με βάση τα ορίσματα `layers_to_load` και `load_output_layer`. Για να ανακτήσει τα απαραίτητα βάρη, ο loader αναλύει το αρχείο `model.safetensor.index.json`. Το αρχείο αυτό λειτουργεί ως ευρετήριο που αντιστοιχίζει τα ονόματα των παραμέτρων του μοντέλου στα αντίστοιχα δεδομένα τους, τα οποία είναι αποθηκευμένα σε ένα ή περισσότερα αρχεία *safetensor*. Η ανάλυση του ευρετηρίου αυτού επιτρέπει στον server node να εντοπίσει και να φορτώσει αποκλειστικά τα storage shards που περιέχουν τα transformer layers ή/και το τελικό output layer που του έχουν ανατεθεί. Σημειώνεται ότι κάθε κόμβος του δικτύου πρέπει να έχει όλα τα βάρη του μοντέλου αποθηκευμένα στην περιφερειακή του μνήμη.

Συμπληρωματικά, ο καθορισμένος Tail server, που φιλοξενεί το output layer, προωθεί τα τελικά υπολογισμένα logits στη μέθοδο `sample_logits`. Αυτή η μέθοδος εφαρμόζει top-p sampling [47] και temperature scaling για την τελική επιλογή του επόμενου token.

### 5.2.2 Χρήση από Client Nodes

Σε αντίθεση με τους server nodes, ο client node χρησιμοποιεί την μονάδα *llm\_loader* για την προεπεξεργασία του κειμένου και τη διαχείριση του αρχικού επιπέδου (embedding layer).

Κατά την αρχικοποίηση του μοντέλου μέσω της μεθόδου *LLM.load*, ο client θέτει την παράμετρο *load\_initial\_layer = True* και φορτώνει αποκλειστικά το embedding layer. Παράλληλα, αρχικοποιεί την κλάση *Preprocessor*, που είναι υπεύθυνη για τις διαδικασίες του tokenization και de-tokenization. Η μέθοδος *encode* του *Preprocessor* μετατρέπει το αρχικό κείμενο σε input IDs, ενώ η μέθοδος *decode* μετατρέπει τα παραγόμενα token IDs σε αναγνώσιμο κείμενο. Εάν φορτώνεται κάποιο instruction tuned μοντέλο εφαρμόζεται η μέθοδος *apply\_chat\_template*, η οποία μορφοποιεί τα δεδομένα εισόδου προσθέτοντας τα ειδικά tokens της αρχιτεκτονικής Llama3 (π.χ. `<|start_header_id|>` και `<|eot_id|>`)

## 5.3 Κατανεμημένη Εκτέλεση Μοντέλου

Με σκοπό την καλύτερη κατανόηση της κατανεμημένης αρχιτεκτονικής του συστήματος, στην υποενότητα αυτή θα γίνει μια αναλυτική περιγραφή της εκτέλεσης ενός inference request, ακολουθώντας το αίτημα μέσα από όλα τα στάδια του pipeline.

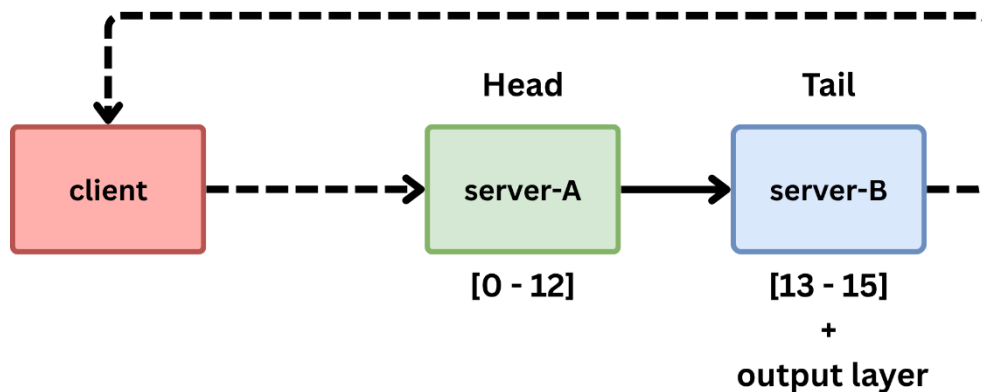


Figure 18. Τοπολογία κατανεμημένης εκτέλεσης του μοντέλου Llama 3.2 1B, με δύο server nodes.

Έστω η τοπολογία που απεικονίζεται στο παραπάνω διάγραμμα, με έναν client node και δύο server nodes. Ο server A έχει φορτώσει τα transformer layers [0 – 12] και ο server B τα transformer layers [13 - 15] και το τελικό output layer. Πριν ξεκινήσει η διαδικασία του κατανεμημένου inference ο client έχει δημιουργήσει ένα gRPC channel με τον Head server και ο Head server με τον successor του (Tail server).

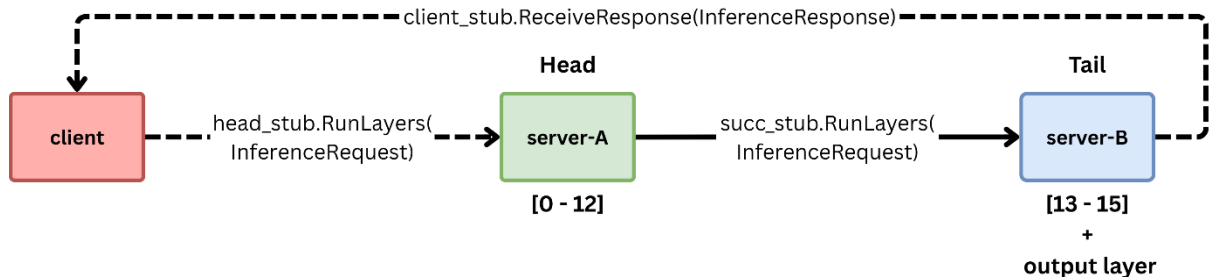


Figure 19. Εκτέλεση αιτημάτων gRPC κατά την διαδικασία του κατανεμημένου συμπερασμού.

Αρχικά, ο χρήστης του client node πληκτρολογεί ένα prompt, το οποίο κωδικοποιείται σε input IDs και μετατρέπεται σε embeddings. Ο πρώτος κύκλος παραγωγής αφορά το prefill stage, όπου τα embeddings των input IDs περνούν από 8-bit κβαντοποίηση και σειριοποιούνται, δημιουργώντας ένα gRPC μήνυμα *InferenceRequest* το οποίο περιλαμβάνει το tensor των embeddings (*tensor\_data*), πληροφορίες για το tensor αυτό (*tensor\_shape, dtype*), το μέγεθος της τρέχουσας ακολουθίας (*seq\_len*), τον μέγιστο επιτρεπτό αριθμό των επιστρεφόμενων tokens (*max\_returned\_tokens*), τις τιμές *absmax* για την αποκβαντοποίηση και τη διεύθυνση IP του client.

Στη συνέχεια ο client εκτελεί το RPC *RunLayers* μέσω του gRPC stub του Head server και αναμένει απάντηση από τον Tail server.

```
self.head_server_stub.RunLayers(inference_request)
```

Ο gRPC servicer του Head server λαμβάνει το inference request και εκτελεί την αποσειριοποίηση και αποκβαντοποίηση των embeddings. Τα δεδομένα που έλαβε από τον client οδηγούνται στην μέθοδο *run\_local\_layers*. Στην μέθοδο αυτή αρχικά δεσμεύεται μνήμη για την KV cache του νέου client και ορίζεται ως η ενεργή KV cache. Έπειτα ο Head server εκτελεί ένα forward pass μέσα από τα φορτωμένα επίπεδα του (transformer layers [0-12]) και ενημερώνει τις μετρικές inference delay και processing rate στο DHT. Οι ενδιάμεσες ενεργοποιήσεις κβαντοποιούνται και δημιουργείται ένα νέο *InferenceRequest* το οποίο αποστέλλεται στον αμέσως επόμενο κόμβο της αλυσίδας.

Τέλος, ο Tail server αποσειριοποιεί τα δεδομένα που έλαβε από τον Head server, δεσμεύει την KV cache του client, εκτελεί το forward pass στα επίπεδά του (transformer layers [13-15] και τελικό output layer) και υπολογίζει το επόμενο token μέσω της μεθόδου *sample\_logits*. Το επόμενο token που υπολογίστηκε σειριοποιείται σε ένα μήνυμα *InferenceResponse* και στέλνεται πίσω στον client μέσω του RPC *ReceiveResponse*.

```
self.client_stub.ReceiveResponse(response)
```

Σε αυτό το στάδιο, ο client αποκωδικοποιεί το token ID που έλαβε και το επιστρέφει στην διεπαφή χρήστη. Η παραγωγή κειμένου συνεχίζεται με αντίστοιχο τρόπο και στο decoding stage με την διαφορά ότι στον Head server θα αποσταλεί μόνο το embedding του καινούριου token και η θέση του στην ακολουθία (*input\_pos*).

## 5.4 Βελτιστοποίηση Δικτύου (8-bit Blockwise Quantization)

Στο πλαίσιο της συνεργατικής εκτέλεση LLMs, η μετάδοση των ενδιάμεσων ενεργοποιήσεων μεταξύ κόμβων εισάγει συχνά σημαντική καθυστέρηση λόγω δικτύου. Οι έξοδοι των transformer layers αποτελούνται από tensors μεγάλης κλίμακας, οι οποίοι αναπαριστούνται σε μορφή κινητής υποδιαστολής ακρίβειας 16-bit ή 32-bit, με αποτέλεσμα η μεταφορά τους στο δίκτυο να είναι χρονοβόρα. Για τον περιορισμό αυτής της καθυστέρησης, στην τρέχουσα διπλωματική εργασία υιοθετείται συμπίεση των ενδιάμεσων ενεργοποιήσεων πριν τη μετάδοση τους στον επόμενο κόμβο, μέσω της τεχνικής του 8-bit dynamic block-wise quantization. Η μέθοδος αυτή εισήχθη από τους Dettmers κ.ά. [48] και έχει εφαρμοστεί με επιτυχία σε παρόμοια καταναμημένα περιβάλλοντα όπως το framework Petals [44].

Το dynamic block-wise quantization επιτυγχάνει συμπίεση υψηλής ακρίβειας, λαμβάνοντας υπόψη το δυναμικό εύρος και την παρουσία ακραίων τιμών στις ενεργοποιήσεις νευρωνικών δικτύων. Ειδικότερα, η μέθοδος βασίζεται στην κατασκευή ενός μη-γραμμικού χάρτη κβαντοποίησης, ο οποίος αποτελείται από 256 διακριτές τιμές. Ο χάρτης αυτός αξιοποιεί μια εξειδικευμένη αναπαράσταση 8-bit, η οποία περιλαμβάνει ένα bit προσήμου, έναν εκθέτη μεταβλητού μήκους (αποτελούμενο από μηδενικά που τερματίζεται με «1») και ένα κλασματικό μέρος μεταβλητού μεγέθους. Η παραγόμενη μη-γραμμική κατανομή των περιοχών κβαντοποίησης (quantization bins) κατανέμει υψηλότερη ακρίβεια στις τιμές που βρίσκονται κοντά στο μηδέν ενώ διατηρεί την δυνατότητα αναπαράστασης αριθμών μεγαλύτερου μεγέθους. Η συγκεκριμένη προσέγγιση εξασφαλίζει την ελαχιστοποίηση των σφαλμάτων στρογγυλοποίησης, καθώς αντικατοπτρίζει την στατιστική κατανομή των ενεργοποιήσεων των LLMs.

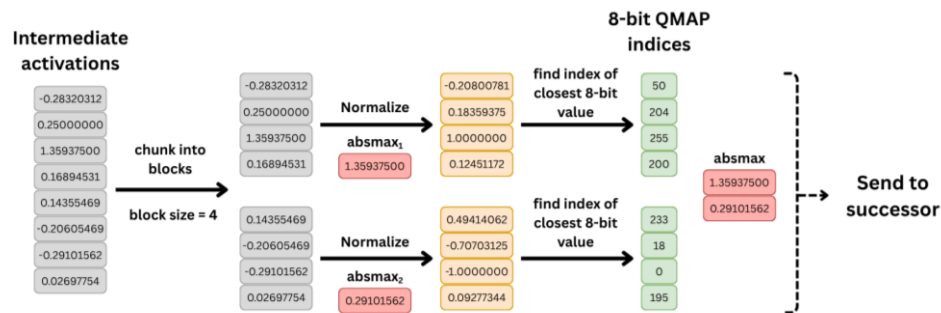
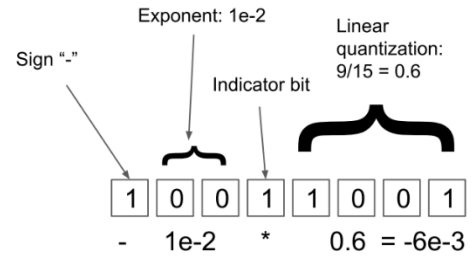


Figure 20. Παράδειγμα κβαντοποίησης με την τεχνική 8-bit dynamic block-wise quantization.  
Πηγή: [από τον συγγραφέα]

Η κβαντοποίηση εφαρμόζεται ανά συστάδες (block-wise) προκειμένου να αποτραπεί η υποβάθμιση της ακρίβειας των δεδομένων. Πριν τη μετάδοση, ο tensor της εκάστοτε ενδιάμεσης ενεργοποίησης επιπεδοποιείται (flattened) και χωρίζεται σε συνεχόμενες συστάδες των 1024 στοιχείων. Για κάθε μία από τις συστάδες υπολογίζεται η μέγιστη απόλυτη τιμή (absmax). Στη συνέχεια, τα στοιχεία της εκάστοτε συστάδας διαιρούνται με τον αντίστοιχο βαθμωτό absmax, γεγονός που κανονικοποιεί όλες τις τιμές στο εύρος  $[-1.0, 1.0]$ . Αυτές οι κανονικοποιημένες τιμές κινητής υποδιαστολής αντιστοιχίζονται στα πλησιέστερα μέσα του χάρτη κβαντοποίησης μέσω δυαδικής αναζήτησης. Αυτή η διαδικασία μετατρέπει τις τιμές σε μια ακολουθία από ακέραιους δείκτες των 8-bit.

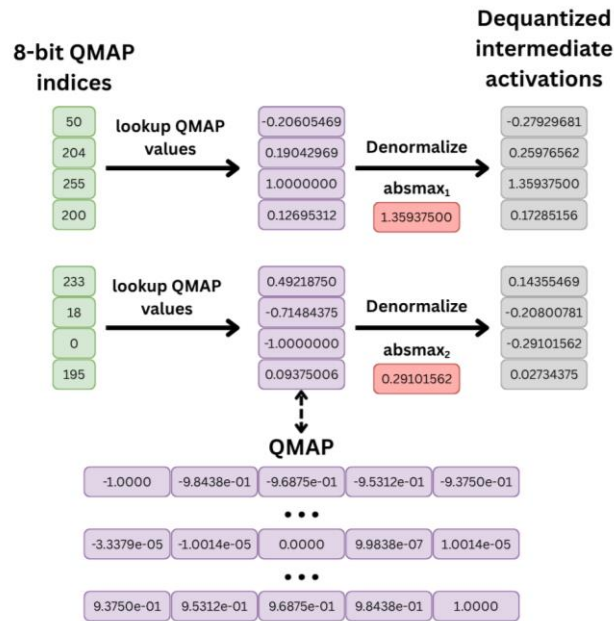


Figure 21. Αποκβαντοποίηση με την τεχνική 8-bit dynamic block-wise quantization. Πηγή: [από τον συγγραφέα]

Μέσω αυτής της μεθοδολογίας το ωφέλιμο φορτίο (payload) που μεταδίδεται στο δίκτυο απο-τελείται αποκλειστικά από τους εξαιρετικά συμπαγείς δείκτες των 8-bit και τις αντίστοιχες τι-μές κλιμάκωσης  $absmax$  της κάθε συστάδας, μειώνοντας δραστικά το αποτύπωμα των μεταφε-ρόμενων δεδομένων.

Κατά την λήψη αυτού του φορτίου, ο επόμενος κόμβος της αλυσίδας εκτελεί το υπολογιστικά ελαφρύ βήμα της αποκβαντοποίησης. Συγκεκριμένα, οι δείκτες των 8-bit αντιστοιχίζονται πίσω στις εκτιμώμενες κανονικοποιημένες τιμές κινητής υποδιαστολής μέσω του κοινόχρηστου δυ-ναμικού χάρτη και τα αποτελέσματα πολλαπλασιάζονται με τις αντίστοιχες τιμές  $absmax$ .

## 5.5 Διεπαφή Χρήστη (Client GUI)

Για την διευκόλυνση της χρήσης και παρακολούθησης του αποκεντρωμένου συστήματος αναπτύχθηκε μια γραφική διεπαφή χρήστη (Graphical User Interface - GUI) χρησιμοποιώντας την βιβλιοθήκη NiceGUI [49] της Python.

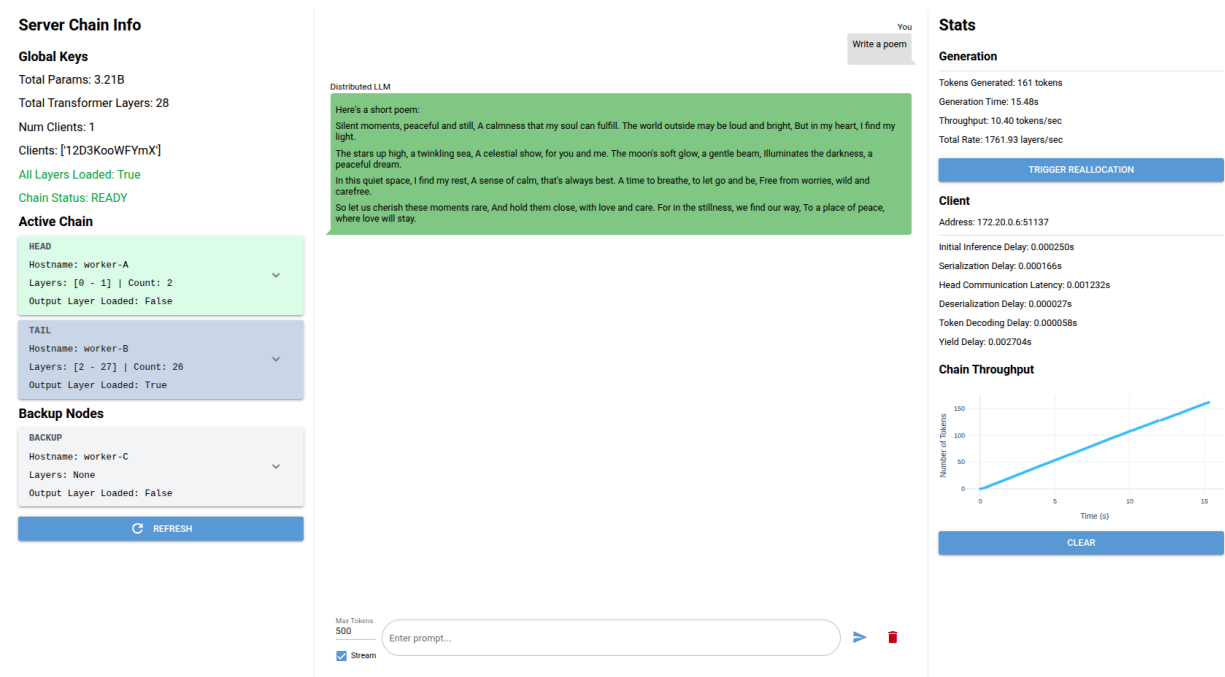


Figure 22. Στιγμιότυπο οθόνης από την διεπαφή χρήστη.

Το GUI παράγεται από το *client\_gui.py* script, το οποίο αρχικοποιεί ένα αντικείμενο *Client*. Μέσω αυτού, ο χρήστης αλληλεπιδρά με το σύστημα, αποκτώντας πρόσβαση στις δυνατότητες παραγωγής κειμένου και στις πληροφορίες που παρέχονται μέσω του DHT.

Στην κεντρική στήλη προσφέρεται ένα περιβάλλον chatbot για την αλληλεπίδραση με το LLM. Παρέχεται η δυνατότητα ελέγχου του μέγιστου αριθμού των παραγόμενων tokens για κάθε prompt και ένας διακόπτης για τον έλεγχο του τύπου παραγωγής (streaming/buffered).

Αριστερά, παρουσιάζονται πληροφορίες που αντλούνται από το DHT, σχετικά με το μοντέλο που χρησιμοποιείται, την κατάσταση της ενεργής αλυσίδας καθώς επίσης και για τους server nodes (ενεργούς και εφεδρικούς). Κάθε καρτέλα ενός server node μπορεί να αναπτυχθεί, φανερώνοντας περαιτέρω λεπτομέρειες όπως ο ρυθμός επεξεργασίας του και η διαθέσιμη μνήμη του.

# Δυναμική και Ανθεκτική σε Σφάλματα Συνεργατική Εκτέλεση Μεγάλων Γλωσσικών Μοντέλων σε Ετερογενείς Συσκευές

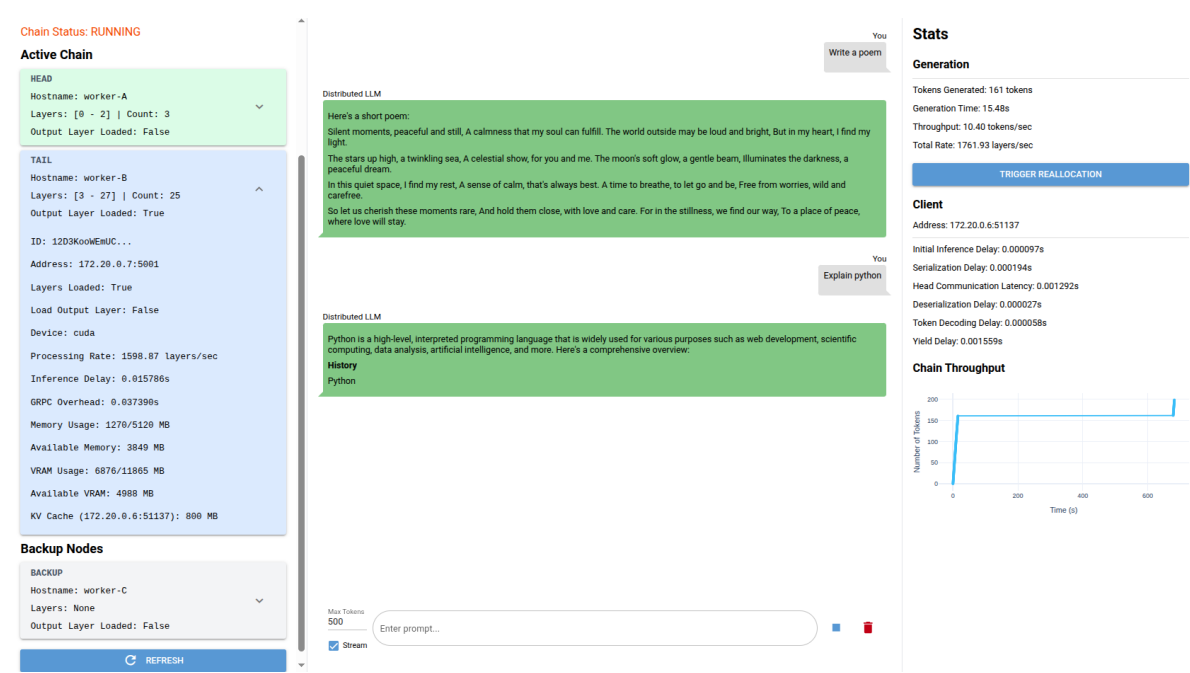


Figure 23. Στιγμιότυπο οθόνης από την διεπαφή χρήστη. Ανάπτυξη της καρτέλας του server “worker-B”.

Τέλος, στην δεξιά στήλη παρέχονται μετρικές σχετικά με την πιο πρόσφατη παραγωγή κειμένου συνοδευόμενες από λεπτομερείς αναλύσεις των καθυστερήσεων του client.

## 6 Δυναμική Εξισορρόπηση και Ανθεκτικότητα σε Σφάλματα

### 6.1 Ανάλυση Απόδοσης Κόμβων

#### 6.1.1 Ρυθμός Επεξεργασίας και η μετρική TLE

Οι μηχανισμοί δυναμικής ανακατανομής φορτίου του συστήματος απαιτούν μετρήσεις του ρυθμού επεξεργασίας (processing rate) των server nodes, δηλαδή του αριθμού των transformer layers που επεξεργάζονται ανά δευτερόλεπτο. Ο Tail server όμως, επεξεργάζεται και το τελικό output layer, το οποίο διαφέρει υπολογιστικά από ένα transformer layer. Για αυτόν τον λόγο κάθε κόμβος υπολογίζει, κατά την αρχικοποίηση του, την μετρική Output Layer Transformer Layer Equivalent (Output Layer TLE), η οποία χωρίζεται περαιτέρω σε Temporal TLE (αντιστοιχία στον χρόνο εκτέλεσης του επιπέδου εξόδου με ένα transformer layer) και Memory TLE (αντιστοιχία στο μέγεθος στην μνήμη).

$$\text{Output Layer Temporal TLE} = \frac{\text{output layer inference delay}}{\text{transformer layer inference delay}}$$

$$\text{Output Layer Memory TLE} = \frac{\text{output layer memory size}}{\text{transformer layer memory size}}$$

Έστω  $L_t$  ο αριθμός των transformer layers που έχουν φορτωθεί,  $W_o$  το output layer temporal TLE και  $D$  το inference delay σε δευτερόλεπτα. Ο ρυθμός επεξεργασίας ενός server node υπολογίζεται ως:

$$\text{Processing Rate} = \begin{cases} \frac{L_t + W_o}{D}, & \text{if output layer loaded} \\ \frac{L_t}{D}, & \text{otherwise} \end{cases}$$

Equation 8. Τύπος υπολογισμού του processing rate για έναν server node.

Η μετρική Output Layer Memory TLE εξαρτάται από την εκάστοτε αρχιτεκτονική του μοντέλου που χρησιμοποιείται και δεν διαφέρει από κόμβο σε κόμβο. Η μετρική αυτή χρησιμοποιείται για τον καθορισμό του αριθμού των επιπέδων που μπορεί να φορτώσει ο Tail server στην μνήμη του.

#### 6.1.2 Server Node Profiling

Η απόδοση των server nodes ελέγχεται σε δύο φάσεις: κατά την αρχικοποίηση και κατά την διάρκεια παραγωγής κειμένου.

Πριν την εκτέλεση του πρώτου inference request οι ενεργοί servers αξιολογούν την απόδοσή τους, για να έχουν την δυνατότητα να εκτελέσουν τον μηχανισμό ανακατανομής φορτίου. Παράλληλα, οι εφεδρικοί server πρέπει να έχουν μετρήσει την απόδοσή τους για να μπορούν να εκτελέσουν τον μηχανισμό Ευκαιριακής Ανάλυσης Επιπέδων. Για τους λόγους αυτούς, κάθε server node εκτελεί την μέθοδο `_profile_node`, κατά την αρχικοποίησή του. Η μέθοδος αυτή φορτώνει αρχικά ένα transformer layer, εκτελεί μερικά forward passes με dummy inputs και

μετράει το inference delay. Έπειτα φορτώνει το output layer και κάνει το ίδιο με σκοπό τον υπολογισμό της μετρικής output layer temporal TLE και μιας αρχικής μέτρησης του processing rate.

Το processing rate των ενεργών servers ενημερώνεται στη συνέχεια σε κάθε forward pass κατά την παραγωγή κειμένου. Με αυτόν τον τρόπο, το σύστημα παρακολουθεί μεταβολές στην υπολογιστική απόδοση των κόμβων και πραγματοποιεί τις απαραίτητες βελτιστοποιήσεις.

## 6.2 Δυναμική Ανακατανομή Φορτίου

Το σύστημα της διπλωματικής διαθέτει δύο μηχανισμούς για την ανακατανομή φορτίου μεταξύ κόμβων, με σκοπό την αύξηση της απόδοσης της αλυσίδας: τους μηχανισμούς της Δυναμικής Ανακατανομής Επιπέδων (Dynamic Layer Reallocation) και της Ευκαιριακής Ανάληψης Επιπέδων (Opportunistic Takeover).

### 6.2.1 Δυναμική Ανακατανομή Επιπέδων

Ο βασικός στόχος αυτής της λειτουργίας είναι η εξάλειψη των υπολογιστικών συμφορήσεων και η μεγιστοποίηση της συνολικής απόδοσης παραγωγής tokens της αλυσίδας συμπερασμού.

Σε ένα κατανεμημένο δίκτυο που αποτελείται από ετερογενείς συσκευές με ποικίλες επεξεργαστικές δυνατότητες, η στατική ανάθεση των επιπέδων του μοντέλου οδηγεί αναπόφευκτα σε ανισορροπίες στην διοχέτευση. Οι ταχύτεροι κόμβοι παραμένουν αδρανείς ενώ περιμένουν τους πιο αργούς κόμβους να ολοκληρώσουν την επεξεργασία των επιπέδων τους. Η δυναμική ανακατανομή επιλύει αυτό το πρόβλημα μέσω της εκ νέου κατανομής των transformer layers και του output layer στους ενεργούς servers, διασφαλίζοντας ότι η καθυστέρηση συμπερασμού κατανέμεται όσο το δυνατόν πιο ισομερώς, σε σχέση με τις δυνατότητες υλικού του κάθε κόμβου.

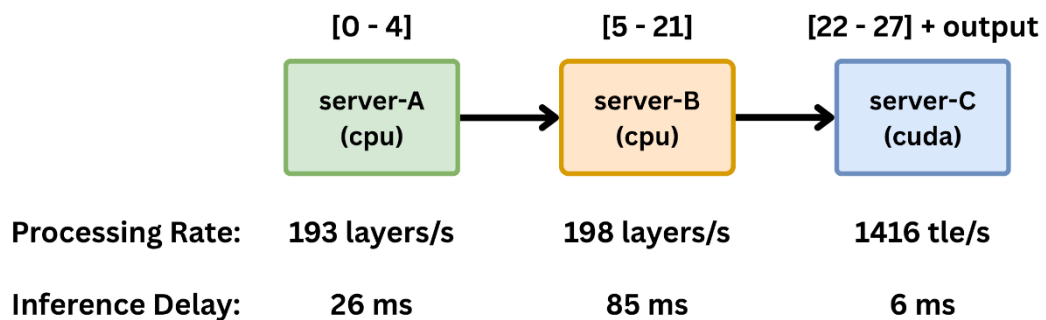


Figure 24. Παράδειγμα μη εξισορροπημένης αλυσίδας εκτέλεσης πριν την εφαρμογή του μηχανισμού της Δυναμικής Ανακατανομής Επιπέδων.

#### 6.2.1.1 Μηχανισμός Προροδότησης

Η διαδικασία ανακατανομής είναι αποκεντρωμένη και ενεργοποιείται αυτόνομα από τον πιο αργό server node, μέσω της παρατήρησης της κατάστασης της ενεργής αλυσίδας σε πραγματικό χρόνο, χωρίς να απαιτείται κάποιος κεντρικός συντονιστής.

Το Chain Health Monitor background thread αξιολογεί περιοδικά την τοπολογία και τις μετρικές απόδοσης της αλυσίδας καλώντας την μέθοδο `evaluate_and_trigger_reallocation`. Η μέθοδος αυτή συγκεντρώνει την καθυστέρηση συμπερασμού (inference delay) όλων των ενεργών

servers και εντοπίζει τους κόμβους με την μεγαλύτερη και μικρότερη καθυστέρηση. Εάν η καθυστέρηση του κόμβου-συμφόρησης υπερβαίνει την καθυστέρηση του ταχύτερου κόμβου κατά ένα προκαθορισμένο όριο *IMBALANCE\_THRESHOLD*, η αλυσίδα θεωρείται μη ισορροπημένη. Εάν πληρούται η συνθήκη ανισορροπίας και υπάρχει διαθέσιμη μνήμη, αποστέλλεται αίτημα gRPC (*TriggerReallocation*) στον Head server για να ξεκινήσει η ανακατανομή.

### 6.2.1.2 Λειτουργίες Κόμβων Κατά την Ανακατανομή

Μόλις ο Head server λάβει το σήμα, εκτελεί την μέθοδο *reallocate\_layers*. Αυτή η διαδικασία διαδίδεται διαδοχικά προς τα κάτω στην αλυσίδα μέχρι τον Tail server.

Ο Head server υπολογίζει τον συνολικό ρυθμό επεξεργασίας του συστήματος και κάθε κόμβος καθορίζει μαθηματικά την αναλογικότερη ανάθεση επιπέδων του.

$$Target\ TLE_i = Total\ TLE \frac{Processing\ Rate_i}{Total\ System\ Rate}$$

Equation 9. Τύπος υπολογισμού του αναλογικότερου αριθμού από *transformer layers* που αντιστοιχούν σε έναν ενεργό server.

Εάν το *Target TLE* υπερβαίνει τη φυσική μνήμη του κόμβου, φορτώνει τον μέγιστο αριθμό επιπέδων που μπορεί να υποστηρίξει και υπολογίζει τον «ενεργό ρυθμό επεξεργασίας» του (effective processing rate). Έπειτα, πυροδοτεί επανεκκίνηση της ανακατανομής με τον προσαρμοσμένο ρυθμό του, διασφαλίζοντας ότι τα υπόλοιπα επίπεδα θα κατανεμηθούν σωστά στους υπόλοιπους κόμβους.

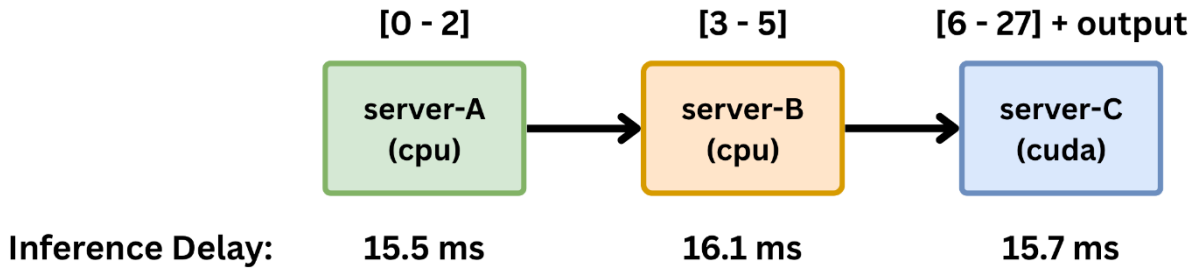


Figure 25. Εξισορροπημένη αλυσίδα εκτέλεσης μετά την εφαρμογή Δυναμικής Ανακατανομής Επιπέδων.

Εάν ο κόμβος μπορεί να φορτώσει την επιθυμητή ακολουθία επιπέδων καλεί την μέθοδο *\_reload\_llm* για να τα φορτώσει, ενεργοποιώντας επιθετικό garbage collection για την αποφυγή απότομων αυξήσεων στην VRAM/RAM.

Ο κόμβος ενημερώνει τα νέα επίπεδα που φιλοξενεί στο DHT και προωθεί το αίτημα *ReallocateRequest* μέσω gRPC στον επόμενο κόμβο μέχρι να εξισορροπηθεί πλήρως η αλυσίδα.

### 6.2.2 Ευκαιριακή Ανάλυση

Η λειτουργία της Ευκαιριακής Ανάλυσης αποτελεί έναν μηχανισμό δυναμικής βελτίωσης της απόδοσης του δικτύου, σχεδιασμένο για την μεγιστοποίηση του συνολικού throughput της αλυσίδας συμπερασμού. Επιτρέπει σε έναν υπολογιστικά ισχυρότερο κόμβο να αναλάβει αυτόνομα τα επίπεδα ενός πιο αδύναμου ενεργού κόμβου. Μέσω της συνεχούς αξιολόγησης των δυνατοτήτων του υλικού και της αντικατάστασης των κόμβων που προκαλούν συμφόρηση, το σύστημα διασφαλίζει ότι το pipeline παραμένει αποδοτικό καθώς διαφορετικοί κόμβοι εισέρχονται ή αποχωρούν από το δίκτυο.

Η διαδικασία ανάληψης αξιολογείται αδιάκοπτα από το Chain Health Monitor background thread που εκτελείται σε όλους τους συμμετέχοντες κόμβους. Οι συνθήκες ενεργοποίησης διαφέρουν ανάλογα με τον τρέχοντα ρόλο του εκάστοτε κόμβου στην τοπολογία (active ή backup server).

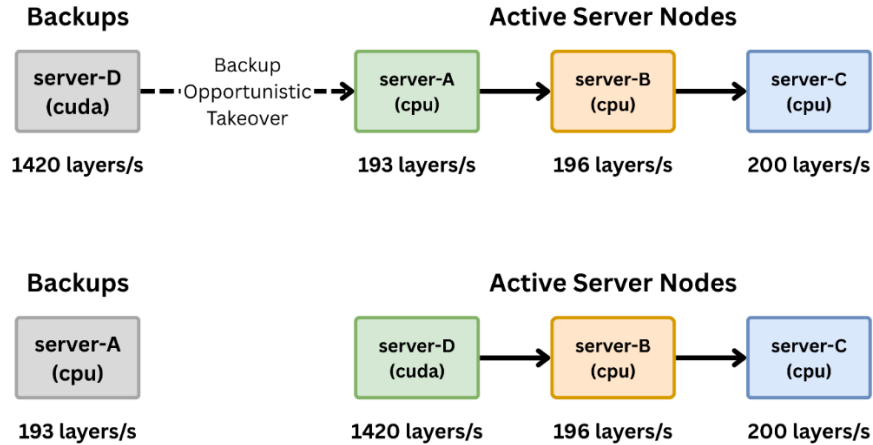


Figure 26. Τοπολογία της ενεργής αλυσίδας πριν (πάνω) και μετά (κάτω) την εφαρμογή Ευκαιριακής Ανάληψης από τον εφεδρικό κόμβο server-D.

Οι εφεδρικοί κόμβοι εκτελούν τακτικά την μέθοδο *evaluate\_takeover\_eligibility* του Chain Manager. Η μέθοδος αυτή συγκρίνει τον τοπικό ρυθμό επεξεργασίας και το όριο μνήμης του κόμβου έναντι κάθε ενεργού κόμβου. Εάν ο ρυθμός επεξεργασίας του εφεδρικού κόμβου υπερβαίνει τον ρυθμό ενός ενεργού κόμβου πολλαπλασιασμένο με το προκαθορισμένο κατώφλι *OPPORTUNISTIC\_TAKEOVER\_MULT\_THRESHOLD* και εφόσον διαθέτει επαρκή μνήμη, ο αλγόριθμος πυροδοτεί την ανάληψη.

Οι ενεργοί κόμβοι παρακολουθούν τον ρυθμό επεξεργασίας του άμεσου διαδόχου τους στην αλυσίδα. Εάν ένας ενεργός κόμβος διαπιστώσει ότι ο ρυθμός επεξεργασίας του είναι μεγαλύτερος από τον ρυθμό του διαδόχου του πολλαπλασιασμένο με το κατώφλι *ACTIVE\_NODE\_TAKEOVER\_THRESHOLD*, τότε ενεργοποιεί την ανάληψη των επιπέδων του.

Μετά την ενεργοποίηση της ανάληψης, ο κόμβος αντικαταστάτης φορτώνει τα επίπεδα του αδύναμου κόμβου και τροποποιεί την τοπολογία στο DHT, λαμβάνοντας την θέση του στην αλυσίδα. Ο αδύναμος κόμβος μετατρέπεται σε εφεδρικό και μια εντολή gRPC (*UpdateSuccessorStub*) αποστέλλεται στον προκάτοχο κόμβο του, δίνοντάς του την οδηγία να ενημερώσει το gRPC successor stub του για να δείχνει στον νέο κόμβο. Τέλος, μεταδίδεται ένα αίτημα gRPC (*UnloadLayers*) στον αντικατασταθέντα κόμβο, δίνοντάς του την εντολή να αφαιρέσει τα φορτωμένα βάρη του μοντέλου από την μνήμη του και να καθαρίσει τις KV caches του.

## 6.3 Μηχανισμοί Ανθεκτικότητας σε Σφάλματα

Το σύστημα παρέχει λειτουργίες αυτόματης και αποκεντρωμένης ανίχνευσης και επισκευής αποτυχιών στην αλυσίδα εκτέλεσης. Η απότομη αποσύνδεση οποιουδήποτε κόμβου μπορεί να εντοπιστεί και να αντιμετωπιστεί από τους υπόλοιπους κόμβους του δικτύου με σκοπό την συνεχή διαθεσιμότητα του συστήματος.

### 6.3.1 Ανίχνευση Σφαλμάτων

Η ανίχνευση σφαλμάτων πραγματοποιείται μέσω κωδικών κατάστασης gRPC κατά τη διάρκεια του συμπερασμού και των εργασιών παρακολούθησης στο παρασκήνιο.

Κατά την διάρκεια του συμπερασμού οι server και client nodes παρακολουθούν συγκεκριμένα gRPC exceptions κατά τη μετάδοση ενδιάμεσων ενεργοποιήσεων στους επόμενους κόμβους. Εάν εντοπιστεί σφάλμα τύπου *grpc.RpcError* με κωδικό κατάστασης *UNAVAILABLE* ή *DEADLINE\_EXCEEDED* κατά την διάρκεια μιας κλήσης *RunLayers*, ο successor node επισημαίνεται ως αποτυχημένος. Για τους ενδιάμεσους κόμβους, αυτό το σφάλμα πυροδοτεί το πρωτόκολλο επιδιόρθωσης διαδόχου (*repair\_chain*). Για τους client nodes, αυτό το σφάλμα εκκινεί το πρωτόκολλο αντικατάστασης του Head server (*replace\_head\_server*).

Κατά την διάρκεια ελέγχων υγείας, ένα background thread παρακολουθεί την διαθεσιμότητα του successor node στέλνοντας αιτήματα gRPC *Check*. Οι client nodes εκτελούν ένα πανομοιότυπο αίτημα *Check* που απευθύνεται στον Head server. Εάν αυτοί οι έλεγχοι οδηγήσουν σε exceptions *UNAVAILABLE* ή *DEADLINE\_EXCEEDED*, ενεργοποιούνται αμέσως οι αντίστοιχοι μηχανισμοί επιδιόρθωσης.

Σημειώνεται ότι επειδή η αποτυχία ενός κόμβου μπορεί να ανιχνευθεί ταυτόχρονα από δύο διαφορετικά νήματα: το ενεργό νήμα συμπερασμού και το νήμα παρακολούθησης παρασκήνιου απαιτείται ειδική διαχείριση για την αποφυγή race condition. Οι μηχανισμοί επιδιόρθωσης (*repair\_chain* για server nodes και *replace\_head\_server* για client nodes) προστατεύονται από ένα τοπικό threading lock.

### 6.3.2 Επιδιόρθωση Αποτυχίας Διαδόχου από Server Nodes

Σε περίπτωση που ένας ενεργός server node διαπιστώσει την αποτυχία του διαδόχου του, εκτελεί την μέθοδο *repair\_chain* με σκοπό την αποκατάσταση της ροής συμπερασμού. Η αποτυχία ενός κόμβου εντοπίζεται πάντοτε από τον προκάτοχο του στην αλυσίδα. Αρχικά, ο server node αντλεί δεδομένα της τοπολογίας από το DHT, το οποίο διατηρεί αποθηκευμένες πληροφορίες του κόμβου που έχει αποτύχει μέχρι αυτές να λήξουν, επιτρέποντας την επιδιόρθωση του δικτύου. Μέσω του DHT, εντοπίζονται τα ορφανά transformer layers και το αν ο νεκρός κόμβος φιλοξενούσε το τελικό output layer.

Ο server node αξιολογεί τους διαθέσιμους πόρους του για να καθορίσει εάν επαρκεί η μνήμη του ώστε να απορροφήσει τα ορφανά επίπεδα. Εφόσον η διαθέσιμη μνήμη είναι επαρκής, φορτώνει τα επίπεδα τοπικά, ενημερώνει την τοπολογία στο DHT και επανασυνδέεται με τον επόμενο κόμβο στην αλυσίδα.

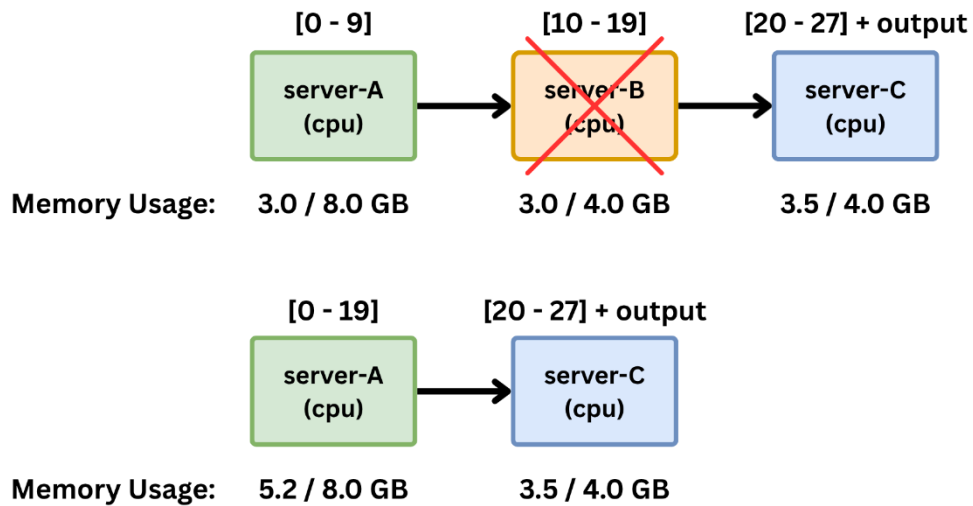


Figure 27. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανάληψης των επιπέδων του από τον server-A.

Σε περίπτωση που ο server δεν διαθέτει την απαραίτητη μνήμη, αναζητά στο DHT διαθέσιμους εφεδρικούς κόμβους με επαρκή χωρητικότητα σε RAM/VRAM. Μόλις βρεθεί ένας βιώσιμος εφεδρικός κόμβος, ενημερώνεται η τοπολογία στο DHT και αποστέλλεται μια εντολή *gRPC LoadLayers*, ώστε ο νέος κόμβος να φορτώσει τα ορφανά επίπεδα και να ενσωματωθεί στην ενεργή αλυσίδα.

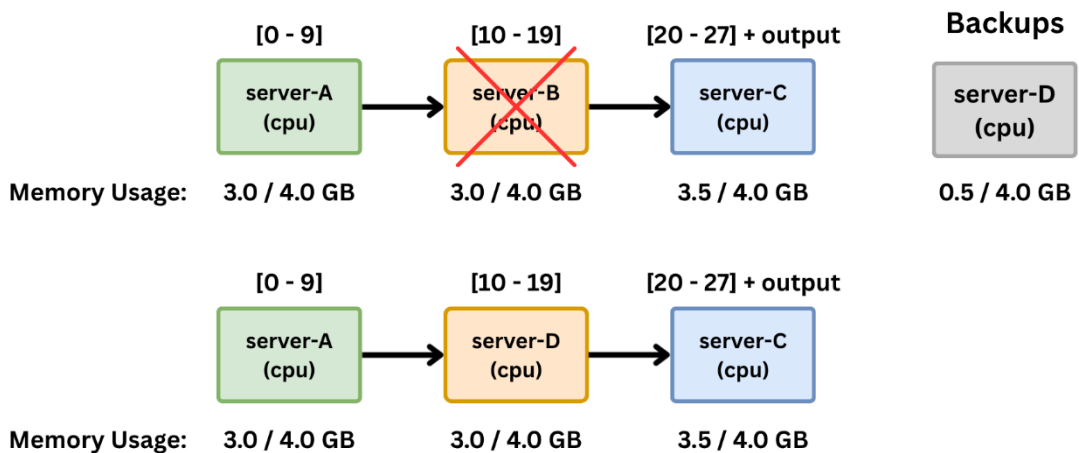


Figure 28. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανάληψης των επιπέδων του από τον εφεδρικό κόμβο server-D.

Εάν, ωστόσο, δεν υπάρχει κανένας εφεδρικός κόμβος ικανός να αναλάβει τα επίπεδα, εφαρμόζεται περικοπή και ανακατανομή της αλυσίδας. Ο κόμβος επιδιόρθωσης γεφυρώνει το κενό, ενημερώνοντας τον successor του ώστε να στοχεύει στον επόμενο διάδοχο του νεκρού κόμβου και δίνει σήμα στον Head server να εκτελέσει την εντολή *TriggerReallocation*. Αυτή η διαδικασία εξαναγκάζει μια δυναμική ανακατανομή των επιπέδων σε όλο το δίκτυο μεταξύ των εναπομεινάντων ενεργών κόμβων.

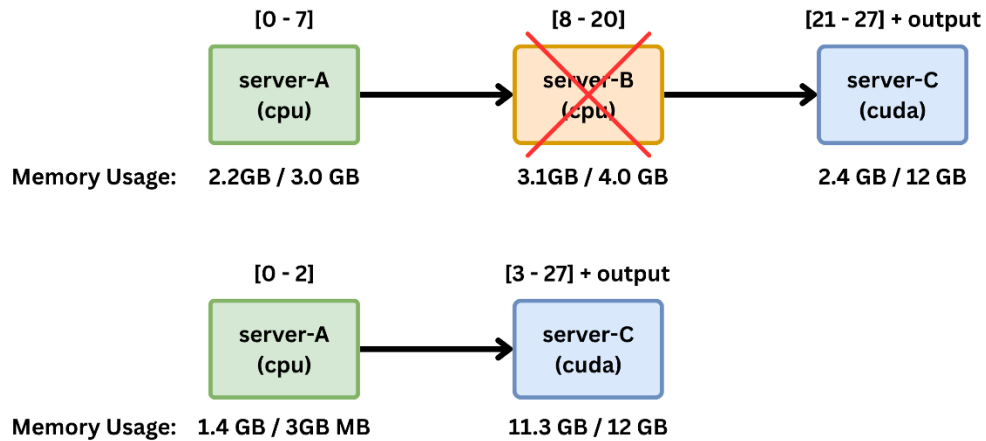


Figure 29. Τοπολογία της αλυσίδας πριν (πάνω) και μετά (κάτω) την επισκευή της αποτυχίας του server-B, μέσω ανακατανομής επιπέδων.

### 6.3.3 Αντικατάσταση Head Server από Client Node

Στην περίπτωση του αρχικού server node, ο οποίος λειτουργεί ως το κύριο σημείο εισόδου για τα αιτήματα συμπερασμού, η αποτυχία του αντιμετωπίζεται από τον client node. Προκειμένου να αποφευχθούν τα race conditions, η λογική αντικατάστασης εκτελείται αποκλειστικά από τον καθορισμένο client leader, ενώ οι υπόλοιποι clients τίθενται σε παύση μέχρι να επιλυθεί η κατάσταση.

Η διαδικασία ξεκινά με την προσπάθεια προαγωγής του άμεσου διαδόχου του νεκρού αρχικού server στην θέση του νέου Head server. Ο client αξιολογεί εάν αυτός ο διάδοχος διαθέτει το απαραίτητο περιθώριο μνήμης για να απορροφήσει τα ορφανά επίπεδα. Εάν η μνήμη επαρκεί, η τοπολογία στο DHT ενημερώνεται αναλόγως και αποστέλλεται εντολή στον διάδοχο για την φόρτωση των επιπέδων.

Στην περίπτωση που ο άμεσος διάδοχος δεν μπορεί να απορροφήσει το νέο φορτίο, ο client διατρέχει τους καταχωρημένους εφεδρικούς κόμβους. Όταν βρεθεί ένας εφεδρικός κόμβος με την απαιτούμενη μνήμη, ορίζεται ως ο νέος Head server και ο client εκδίδει την εντολή *Load-Layers* για την αρχικοποίηση του.

Τέλος, εάν ούτε ο διάδοχος ούτε κάποιος από τους εφεδρικούς κόμβους μπορούν να φιλοξενήσουν τα ορφανά επίπεδα, εφαρμόζεται ανακατανομή των επιπέδων, όπου ο διάδοχος του νεκρού κόμβου καθίσταται άνευ όρων ο νέος Head server. Ο client εκτελεί την ρουτίνα *trigger\_reallocation*, η οποία επανατοποθετεί δυναμικά τα επίπεδα του μοντέλου στους επιζώντες κόμβους. Με την ολοκλήρωση της ανακατανομής, ο client σαρώνει το δίκτυο για τυχόν εναπομείναντες εφεδρικούς κόμβους, αποστέλλοντάς τους την εντολή *JoinChain* σε μια προσπάθεια να ανακατασκευάσει τη συνολική υπολογιστική χωρητικότητα του συστήματος.

## 7 Πειραματική Αξιολόγηση

### 7.1 Πειραματική Μεθοδολογία και Περιβάλλον Αξιολόγησης

Η πειραματική αξιολόγηση αποσκοπεί στη διερεύνηση της λειτουργικότητας, της απόδοσης και της προσαρμοστικότητας της προτεινόμενης αρχιτεκτονικής υπό διαφορετικές συνθήκες κατανομής υπολογιστικών πόρων και δικτυακής επικοινωνίας. Ειδικότερα, εξετάζεται η δυνατότητα συνεργατικής εκτέλεσης LLMs από ετερογενείς κόμβους, η αποτελεσματικότητα της Δυναμικής Ανακατανομής των επιπέδων του μοντέλου, η επίδραση του διαθέσιμου εύρους ζώνης και της κβαντοποίησης των ενδιάμεσων ενεργοποιήσεων, η εξυπηρέτηση πολλαπλών ταυτόχρονων clients, καθώς και η απόκριση του συστήματος σε περιπτώσεις υποβάθμισης της απόδοσης ή αποτυχίας ενεργών κόμβων.

Για τη συστηματική αξιολόγηση της αρχιτεκτονικής διαμορφώνεται ένα ελεγχόμενο πειραματικό περιβάλλον, στο οποίο οι διαθέσιμοι υπολογιστικοί και δικτυακοί πόροι μπορούν να μεταβάλλονται ανάλογα με τις απαιτήσεις κάθε πειράματος. Η προσέγγιση αυτή επιτρέπει την αναπαραγωγή διαφορετικών συνθηκών λειτουργίας και τη σύγκριση εναλλακτικών στρατηγικών κατανομής του μοντέλου υπό κοινές συνθήκες.

#### 7.1.1 Υπολογιστικό Περιβάλλον

Η αξιολόγηση του συστήματος πραγματοποιείται σε ένα ισχυρό host PC, στο οποίο αρχικοποιούνται πολλαπλά Docker containers που αναπαριστούν τους διαφορετικούς κόμβους του κατανεμημένου δικτύου. Το host PC διαθέτει επεξεργαστή AMD Ryzen 9 7950X με 16 φυσικούς πυρήνες και δύο threads ανά πυρήνα, συνολική μνήμη RAM 32 GB και κάρτα γραφικών NVIDIA GeForce RTX 4070 Ti με 12 GB VRAM.

Η χρήση Docker containers επιτρέπει τη δημιουργία ενός ελεγχόμενου περιβάλλοντος ετερογενών κόμβων πάνω στο ίδιο φυσικό σύστημα. Μέσω του αρχείου *docker-compose* καθορίζεται ο ρόλος κάθε container ως server ή client node, ενώ παράλληλα μπορούν να οριστούν διαφορετικοί περιορισμοί ως προς τη διαθέσιμη μνήμη RAM, τον αριθμό των CPU και την πρόσβαση στην GPU. Με αυτόν τον τρόπο δημιουργούνται εικονικοί κόμβοι με διαφορετικές υπολογιστικές δυνατότητες, προσομοιώνοντας την ετερογένεια που χαρακτηρίζει ένα πραγματικό τοπικό δίκτυο συσκευών ευρείας κατανάλωσης.

Η πρόσβαση στην GPU παρέχεται σε έναν μόνο container, ενώ οι υπόλοιποι server nodes βασίζονται αποκλειστικά στην CPU. Η συγκεκριμένη διαμόρφωση επιτρέπει την αξιολόγηση του συστήματος σε συνθήκες έντονης ετερογένειας, καθώς στην ίδια αλυσίδα εκτέλεσης μπορούν να συμμετέχουν κόμβοι με διαφορετικές δυνατότητες επεξεργασίας και μνήμης. Παράλληλα, μέσω των περιορισμών που επιβάλλονται σε επίπεδο container είναι δυνατή η ελεγχόμενη μεταβολή των διαθέσιμων πόρων και η δημιουργία διαφορετικών πειραματικών τοπολογιών.

Στα βασικά πειράματα σύγκρισης χρησιμοποιούνται τρεις server nodes, οι worker-A, worker-B και worker-C. Οι worker-A και worker-B λειτουργούν αποκλειστικά μέσω CPU και διαθέτουν διαφορετικούς περιορισμούς μνήμης, ενώ ο worker-C διαθέτει πρόσβαση και στην GPU των 12 GB VRAM. Η ετερογενής αυτή διαμόρφωση χρησιμοποιείται για τη διερεύνηση της επίδρασης διαφορετικών στρατηγικών ανάθεσης των transformer layers στην απόδοση και στην κατανάλωση μνήμης του συστήματος.

### 7.1.2 Μοντέλα και Πειραματικές Διαμορφώσεις

Η υλοποίηση και η πειραματική αξιολόγηση βασίζονται σε μοντέλα της οικογένειας Llama 3. Ανάλογα με τον στόχο του εκάστοτε πειράματος χρησιμοποιούνται διαφορετικά μεγέθη μοντέλων, ώστε να αξιολογείται η συμπεριφορά της αρχιτεκτονικής υπό διαφορετικές απαιτήσεις υπολογισμού, μνήμης και επικοινωνίας.

Το Llama 3.2 3B αποτελεί το βασικό μοντέλο για το μεγαλύτερο μέρος των πειραμάτων. Ειδικότερα, χρησιμοποιείται για τη σύγκριση των διαφορετικών στρατηγικών κατανομής των transformer layers, καθώς και για την αξιολόγηση των μηχανισμών Δυναμικής Ανακατανομής Επιπέδων και Ευκαιριακής Ανάλυσης. Στα βασικά πειράματα σύγκρισης, το μοντέλο κατανέμεται μεταξύ τριών ετερογενών workers, εφαρμόζεται 8-bit κβαντοποίηση στις ενδιάμεσες ενεργοποιήσεις και παράγονται 1.000 tokens ανά inference request.

Στην αξιολόγηση της κβαντοποίησης χρησιμοποιούνται επιπλέον τα μοντέλα Llama 3.2 1B και Llama 3.1 8B. Η χρήση μοντέλων διαφορετικού μεγέθους επιτρέπει την εξέταση της μεταβολής του μεγέθους των intermediate activation tensors και κατά συνέπεια, του ωφέλιμου φορτίου που πρέπει να μεταδοθεί μεταξύ διαδοχικών server nodes. Με τον τρόπο αυτό αξιολογείται η αποτελεσματικότητα της 8-bit dynamic block-wise quantization σε διαφορετικές απαιτήσεις επικοινωνίας.

Το Llama 3.1 8B χρησιμοποιείται επίσης κατά την αξιολόγηση της ανθεκτικότητας σε σφάλματα, προκειμένου να διερευνηθεί η επίδραση του μεγαλύτερου μεγέθους του μοντέλου στον χρόνο που απαιτείται για την ανάκτηση των απαραίτητων παραμέτρων και την αποκατάσταση της αλυσίδας μετά την αποτυχία ενός κόμβου.

Οι επιμέρους παράμετροι κάθε πειράματος, όπως ο αριθμός των ενεργών και εφεδρικών server nodes, η κατανομή των transformer layers, ο αριθμός των clients και οι συνθήκες δικτύου, μεταβάλλονται ανάλογα με τον μηχανισμό που αξιολογείται και παρουσιάζονται αναλυτικά στις αντίστοιχες ενότητες.

### 7.1.3 Δικτυακές Συνθήκες

Δεδομένου ότι όλα τα Docker containers εκτελούνται στο ίδιο φυσικό host, η μεταξύ τους επικοινωνία χαρακτηρίζεται από ιδιαίτερα υψηλό διαθέσιμο εύρος ζώνης. Η μέση ταχύτητα επικοινωνίας μεταξύ των containers μετρήθηκε περίπου στα 45 Gbit/s, τιμή η οποία δεν είναι αντιπροσωπευτική ενός τυπικού τοπικού δικτύου μεταξύ ανεξάρτητων φυσικών συσκευών.

Για τον λόγο αυτό, οι δικτυακές συνθήκες τροποποιούνται τεχνητά μέσω του εργαλείου Pumba [50]. Το Pumba αποτελεί εργαλείο chaos testing για περιβάλλοντα Docker και επιτρέπει, μεταξύ άλλων, την επιβολή ελεγχόμενων περιορισμών στην επικοινωνία μεταξύ των containers. Στην παρούσα εργασία χρησιμοποιείται για τον περιορισμό του διαθέσιμου bandwidth και όπου απαιτείται, για τη δημιουργία συνθηκών υποβάθμισης της λειτουργίας συγκεκριμένων κόμβων.

Στα περισσότερα πειράματα, το διαθέσιμο bandwidth μεταξύ των κόμβων περιορίζεται στα 250 Mbps, ώστε το πειραματικό περιβάλλον να προσεγγίζει περισσότερο τις συνθήκες ενός τοπικού δικτύου Wi-Fi. Η συγκεκριμένη τιμή χρησιμοποιείται ως βασική δικτυακή συνθήκη κατά τη σύγκριση των στρατηγικών κατανομής και την αξιολόγηση της κατανεμημένης εκτέλεσης.

Για την ειδικότερη διερεύνηση της επίδρασης του δικτύου στην απόδοση, πραγματοποιούνται επιπλέον πειράματα με διαφορετικές τιμές διαθέσιμου bandwidth. Στόχος είναι να προσδιοριστεί σε ποιο σημείο η μετάδοση των ενδιάμεσων ενεργοποιήσεων μεταξύ των server nodes

μετατρέπεται σε σημαντικό bottleneck σε σχέση με τον ίδιο τον χρόνο συμπερασμού. Παράλληλα, η μεταβολή του bandwidth επιτρέπει την άμεση σύγκριση της επικοινωνίας με και χωρίς 8-bit κβαντοποίηση των intermediate activations και την αξιολόγηση του οφέλους της τεχνικής σε συνθήκες περιορισμένης δικτυακής χωρητικότητας.

Το Pumba αξιοποιείται επίσης στα πειράματα δυναμικής εξισορρόπησης φορτίου, όπου προκαλείται ελεγχόμενη υποβάθμιση της επεξεργαστικής απόδοσης ενός ενεργού worker. Με αυτόν τον τρόπο δημιουργείται τεχνητά μια κατάσταση υπολογιστικής συμφόρησης και αξιολογείται η ικανότητα του συστήματος να την ανιχνεύσει και να ενεργοποιήσει τον μηχανισμό Ευκαιριακής Ανάληψης.

#### 7.1.4 Στρατηγικές Κατανομής και Baselines

Για την αξιολόγηση της αποτελεσματικότητας της προτεινόμενης δυναμικής κατανομής των transformer layers, η απόδοση του συστήματος συγκρίνεται με διαφορετικές στρατηγικές εκτέλεσης. Οι στρατηγικές αυτές επιλέγονται ώστε να αποτυπώνουν τόσο την εκτέλεση του μοντέλου από έναν μεμονωμένο κόμβο όσο και διαφορετικές προσεγγίσεις κατανομής του μοντέλου μεταξύ πολλαπλών ετερογενών κόμβων.

Συγκεκριμένα, εξετάζονται οι ακόλουθες τέσσερις στρατηγικές:

**Single CPU Node.** Το σύνολο των transformer layers και το output layer του μοντέλου φορτώνονται και εκτελούνται σε έναν μόνο κόμβο που χρησιμοποιεί CPU. Η συγκεκριμένη διαμόρφωση λειτουργεί ως baseline για την τοπική εκτέλεση του μοντέλου χωρίς αξιοποίηση της διαθέσιμης GPU και χωρίς communication overhead μεταξύ διαφορετικών server nodes.

**Single GPU Node.** Το σύνολο του μοντέλου φορτώνεται στον worker-C, ο οποίος διαθέτει πρόσβαση στην NVIDIA GeForce RTX 4070 Ti. Η στρατηγική αυτή αποτελεί το ισχυρότερο single-node baseline, καθώς εξαλείφει την ανάγκη μεταφοράς intermediate activations μεταξύ διαφορετικών workers και αξιοποιεί τον ισχυρότερο διαθέσιμο επιταχυντή.

**Greedy Allocation.** Το μοντέλο κατανέμεται μεταξύ των διαθέσιμων server nodes με βάση μια αρχική άπληστη στρατηγική κατανομής. Η προσέγγιση αυτή λαμβάνει υπόψη τη δυνατότητα των κόμβων να φιλοξενήσουν τα αντίστοιχα τμήματα του μοντέλου, χωρίς όμως να εξισορροπεί πλήρως τον πραγματικό χρόνο επεξεργασίας των διαφορετικών σταδίων της αλυσίδας. Κατά συνέπεια, ένας ασθενέστερος κόμβος μπορεί να αποτελέσει bottleneck για ολόκληρο το pipeline.

**Balanced Allocation.** Μετά την αξιολόγηση των πραγματικών ρυθμών επεξεργασίας των συμμετεχόντων workers εφαρμόζεται ο μηχανισμός Δυναμικής Ανακατανομής Επιπέδων. Ο αριθμός των transformer layers που αναλαμβάνει κάθε κόμβος προσαρμόζεται στις πραγματικές υπολογιστικές δυνατότητες και στους περιορισμούς μνήμης του, με στόχο την καλύτερη εξισορρόπηση των επιμέρους σταδίων της αλυσίδας και τη μείωση του συνολικού inference delay.

Η σύγκριση μεταξύ Greedy και Balanced Allocation είναι ιδιαίτερα σημαντική για την αξιολόγηση του μηχανισμού δυναμικής εξισορρόπησης φορτίου, καθώς και στις δύο περιπτώσεις χρησιμοποιούνται οι ίδιοι διαθέσιμοι κόμβοι και το ίδιο μοντέλο, αλλά διαφοροποιείται ο τρόπος με τον οποίο κατανέμεται το υπολογιστικό έργο μεταξύ τους. Με αυτόν τον τρόπο μπορεί να απομονωθεί η επίδραση της στρατηγικής κατανομής στην τελική απόδοση του συστήματος.

### 7.1.5 Μετρικές Αξιολόγησης

Η απόδοση της προτεινόμενης αρχιτεκτονικής αξιολογείται μέσω ενός συνόλου μετρικών που αποτυπώνουν διαφορετικές πτυχές της διαδικασίας καταναμεμένου inference. Οι βασικές μετρικές αφορούν την καθυστέρηση έναρξης της παραγωγής, τον ρυθμό παραγωγής tokens, την καθυστέρηση μεταξύ διαδοχικών tokens, τη χρήση μνήμης και τις επιμέρους καθυστερήσεις που εισάγονται από τον υπολογισμό και την επικοινωνία μεταξύ των κόμβων.

**Time-To-First-Token (TTFT).** Το TTFT εκφράζει τον χρόνο που μεσολαβεί από την έναρξη ενός inference request μέχρι την παραγωγή του πρώτου token. Σε αντίθεση με τα επόμενα βήματα παραγωγής, το πρώτο inference step περιλαμβάνει το στάδιο prefill του αρχικού prompt και την αρχικοποίηση των αντίστοιχων KV caches. Για τον λόγο αυτό, το TTFT είναι συνήθως υψηλότερο από την καθυστέρηση παραγωγής των επόμενων tokens.

**Inter-Token Latency (ITL).** Το ITL εκφράζει τη μέση χρονική καθυστέρηση μεταξύ της παραγωγής δύο διαδοχικών tokens μετά την ολοκλήρωση του αρχικού prefill. Η συγκεκριμένη μετρική αποτυπώνει την απόδοση του συστήματος κατά την autoregressive φάση του decoding και επηρεάζεται τόσο από τον χρόνο inference στους επιμέρους κόμβους όσο και από την επικοινωνία μεταξύ των διαδοχικών σταδίων του pipeline.

**Throughput.** Το throughput εκφράζει τον ρυθμό παραγωγής tokens του συστήματος και μετράται σε **tokens per second (TPS)**. Αποτελεί βασική μετρική για την αξιολόγηση της συνολικής απόδοσης της αλυσίδας, καθώς στο pipeline parallelism ο ρυθμός παραγωγής επηρεάζεται σημαντικά από το βραδύτερο στάδιο της αλυσίδας. Η σύγκριση του throughput μεταξύ διαφορετικών στρατηγικών ανάθεσης επιτρέπει επομένως την αξιολόγηση της αποτελεσματικότητας της εξισορρόπησης του υπολογιστικού φορτίου.

**Χρήση Μνήμης.** Καταγράφεται η κατανάλωση **RAM και VRAM** ανά server node για τις διαφορετικές στρατηγικές κατανομής. Η συγκεκριμένη μετρική είναι ιδιαίτερα σημαντική στην παρούσα αρχιτεκτονική, καθώς ένας από τους βασικούς στόχους της καταναμεμένης εκτέλεσης είναι η δυνατότητα φιλοξενίας μοντέλων των οποίων οι συνολικές απαιτήσεις μνήμης μπορούν να υπερβαίνουν τις δυνατότητες μιας επιμέρους συσκευής. Παράλληλα, η καταγραφή της μνήμης επιτρέπει την εξέταση του συμβιβασμού μεταξύ υψηλότερης απόδοσης και αποτελεσματικότερης κατανομής των απαιτήσεων μνήμης μεταξύ των διαθέσιμων συσκευών.

**Επιμέρους καθυστερήσεις ανά κόμβο.** Για την αναλυτικότερη διερεύνηση των σημείων συμφόρησης της αλυσίδας καταγράφονται ξεχωριστά οι καθυστερήσεις που σχετίζονται με την αποσειριοποίηση των εισερχόμενων δεδομένων, την εκτέλεση του inference, το logit sampling στον Tail node, τη σειριοποίηση των ενδιάμεσων ενεργοποιήσεων και τη μετάδοσή τους στον επόμενο κόμβο. Η ανάλυση αυτή επιτρέπει να προσδιοριστεί εάν το κύριο bottleneck οφείλεται στην υπολογιστική επεξεργασία ή στην επικοινωνία μεταξύ των κόμβων.

**Μέγεθος Δικτυακού Payload και Καθυστέρηση Μετάδοσης.** Κατά την αξιολόγηση της 8-bit dynamic block-wise quantization καταγράφεται το μέγεθος των δεδομένων που μεταδίδονται μεταξύ των διαδοχικών σταδίων του pipeline, τόσο για τις αρχικές 16-bit ενεργοποιήσεις όσο και για τις κβαντισμένες αναπαραστάσεις τους. Παράλληλα, εξετάζεται η καθυστέρηση επικοινωνίας υπό διαφορετικές τιμές διαθέσιμου bandwidth, ώστε να προσδιοριστούν οι δικτυακές συνθήκες στις οποίες η κβαντοποίηση παρέχει ουσιαστικό όφελος.

**Χρόνος Ευκαιριακής Ανάλυσης.** Στα πειράματα δυναμικής εξισορρόπησης φορτίου αξιολογείται ο χρόνος που απαιτείται από τη δημιουργία συνθηκών υπολογιστικής συμφόρησης μέχρι

την αντικατάσταση του ασθενέστερου ενεργού κόμβου και την επαναφορά της αλυσίδας σε αποδοτικότερη κατάσταση.

**Χρόνος Ανάκαμψης από Αποτυχία Κόμβου.** Για την αξιολόγηση της ανθεκτικότητας σε σφάλματα καταγράφεται ο χρόνος που μεσολαβεί από την αποτυχία ενός ενεργού server node μέχρι την αποκατάσταση της αλυσίδας και τη συνέχιση της παραγωγής tokens. Η μετρική αυτή επιτρέπει τη σύγκριση διαφορετικών μηχανισμών επισκευής και την εξέταση της επίδρασης του μεγέθους του μοντέλου στον απαιτούμενο χρόνο ανάκαμψης.

Τέλος, κατά την αξιολόγηση της ταυτόχρονης εξυπηρέτησης πολλαπλών clients εξετάζεται η μεταβολή του συνολικού throughput καθώς αυξάνεται ο αριθμός των παράλληλων inference requests. Με τον τρόπο αυτό αξιολογείται κατά πόσο το recurrent pipelining και η διατήρηση ανεξάρτητων KV caches ανά client μπορούν να αυξήσουν την αξιοποίηση των διαθέσιμων server nodes και τη συνολική διεκπεραιωτική ικανότητα της αλυσίδας.

## 7.2 Αξιολόγηση υπό Ρεαλιστικές Συνθήκες Λειτουργίας

Πριν από την επιμέρους αξιολόγηση των μηχανισμών βελτιστοποίησης και ανθεκτικότητας του συστήματος, εξετάζεται η λειτουργία της προτεινόμενης αρχιτεκτονικής υπό συνθήκες που προσεγγίζουν ένα ετερογενές τοπικό δίκτυο συσκευών ευρείας κατανάλωσης. Στόχος της συγκεκριμένης αξιολόγησης είναι να διαπιστωθεί εάν η συνεργατική εκτέλεση ενός LLM παραμένει πρακτικά εφικτή όταν οι συμμετέχοντες κόμβοι διαθέτουν διαφορετικές υπολογιστικές δυνατότητες και η επικοινωνία μεταξύ τους υπόκειται σε ρεαλιστικούς περιορισμούς εύρους ζώνης.

Όπως περιγράφηκε στην ενότητα 7.1, οι κόμβοι του πειραματικού δικτύου υλοποιούνται ως ανεξάρτητα Docker containers στο ίδιο host σύστημα. Παρότι η φυσική υποδομή είναι κοινή, σε κάθε container επιβάλλονται διαφορετικοί περιορισμοί ως προς τον αριθμό των διαθέσιμων CPU cores, τη μνήμη RAM και την πρόσβαση στην GPU. Με αυτόν τον τρόπο δημιουργείται μια ετερογενής τοπολογία, στην οποία οι επιμέρους server nodes παρουσιάζουν διαφορετικές δυνατότητες επεξεργασίας και μνήμης.

Παράλληλα, επειδή η εγγενής ταχύτητα επικοινωνίας μεταξύ των containers είναι σημαντικά υψηλότερη από αυτή ενός συνηθισμένου τοπικού δικτύου, το διαθέσιμο bandwidth περιορίζεται τεχνητά μέσω του εργαλείου Pumba. Στα βασικά πειράματα χρησιμοποιείται όριο 250 Mbps, ώστε η επικοινωνία μεταξύ των κόμβων να προσεγγίζει περισσότερο τις συνθήκες ενός τοπικού δικτύου Wi-Fi. Έτσι, η αξιολόγηση λαμβάνει ταυτόχρονα υπόψη τόσο την ετερογένεια των υπολογιστικών πόρων όσο και το communication overhead που εισάγει η κατανεμημένη μετάδοση των intermediate activations.

Η βασική πειραματική τοπολογία περιλαμβάνει τρεις server nodes με διαφορετικές υπολογιστικές δυνατότητες. Οι worker-A και worker-B βασίζονται αποκλειστικά σε CPU resources, ενώ ο worker-C διαθέτει πρόσβαση στην GPU του συστήματος. Για τα βασικά πειράματα χρησιμοποιείται το Llama 3.2 3B, του οποίου τα transformer layers μπορούν να κατανεμηθούν μεταξύ των τριών κόμβων, ενώ το τελικό output layer ανατίθεται στον τελευταίο κόμβο της αλυσίδας.

Η συγκεκριμένη διαμόρφωση επιτρέπει την αξιολόγηση ενός βασικού σεναρίου χρήσης της προτεινόμενης αρχιτεκτονικής: ενός τοπικού δικτύου στο οποίο πολλαπλές συσκευές διαφορε-

τικών δυνατοτήτων συνδυάζουν τους διαθέσιμους υπολογιστικούς και μνημονικούς τους πόρους, ώστε να εκτελέσουν συνεργατικά ένα μοντέλο που διαφορετικά θα επιβάρυνε σημαντικά ή θα υπερέβαινε τις δυνατότητες ορισμένων από τις επιμέρους συσκευές.

Η αξιολόγηση υπό αυτές τις συνθήκες επικεντρώνεται σε τρεις κύριες διαστάσεις. Πρώτον, εξετάζεται εάν η κατανεμημένη αρχιτεκτονική μπορεί να ολοκληρώσει απρόσκοπτα την *autoregressive* παραγωγή κειμένου όταν τα επίπεδα του μοντέλου είναι κατανεμημένα μεταξύ ετερογενών κόμβων. Δεύτερον, εξετάζεται το κόστος που εισάγει η κατανεμημένη εκτέλεση λόγω της σειριακής μετάδοσης των ενδιάμεσων ενεργοποιήσεων μεταξύ των σταδίων του *pipeline*. Τρίτον, διερευνάται κατά πόσο η συνολική απόδοση εξαρτάται περισσότερο από τους περιορισμούς του δικτύου ή από τις διαφορετικές ταχύτητες *inference* των συμμετεχόντων κόμβων.

Τα αποτελέσματα των επόμενων ενοτήτων δείχνουν ότι, υπό τις βασικές συνθήκες λειτουργίας του πειραματικού περιβάλλοντος, η συνεργατική εκτέλεση του μοντέλου είναι εφικτή, αλλά η τελική απόδοση εξαρτάται σε μεγάλο βαθμό από την ισορροπία του υπολογιστικού φορτίου μεταξύ των *server nodes*. Για τον λόγο αυτό, στην ενότητα 7.3 αξιολογούνται συστηματικά διαφορετικές στρατηγικές κατανομής των *transformer layers* και συγκρίνονται ως προς το *throughput*, το *Time-To-First-Token*, το *Inter-Token Latency*, την κατανάλωση μνήμης και τις επιμέρους καθυστερήσεις της αλυσίδας.

### 7.3 Σύγκριση Στρατηγικών Κατανομής

Η αποτελεσματική κατανομή των *transformer layers* μεταξύ των διαθέσιμων *server nodes* αποτελεί καθοριστικό παράγοντα για την απόδοση της κατανεμημένης εκτέλεσης. Σε ένα ετερογενές περιβάλλον, στο οποίο οι συμμετέχοντες κόμβοι διαφέρουν ως προς την υπολογιστική ισχύ και τη διαθέσιμη μνήμη, μια μη ισορροπημένη ανάθεση των επιπέδων μπορεί να οδηγήσει στη δημιουργία *bottleneck* σε έναν ασθενέστερο κόμβο και να περιορίσει την απόδοση ολόκληρης της αλυσίδας.

Για την αξιολόγηση της επίδρασης της στρατηγικής κατανομής συγκρίνονται τέσσερις διαφορετικές διαμορφώσεις: *Single CPU Node*, *Single GPU Node*, *Greedy Allocation* και *Balanced Allocation*. Όπως περιγράφηκε στην Ενότητα 7.1.4, στις δύο πρώτες περιπτώσεις το σύνολο του μοντέλου εκτελείται σε έναν μόνο κόμβο, ενώ στις δύο τελευταίες το μοντέλο κατανέμεται μεταξύ τριών ετερογενών *server nodes*. Η στρατηγική *Greedy* αντιστοιχεί στην αρχική κατανομή των επιπέδων σύμφωνα με τους διαθέσιμους πόρους μνήμης κατά την ένταξη των κόμβων στο δίκτυο, ενώ στην *Balanced Allocation* εφαρμόζεται ο μηχανισμός Δυναμικής Ανακατανομής Επιπέδων, ώστε το υπολογιστικό φορτίο να προσαρμόζεται στην πραγματική απόδοση των κόμβων.

Στα συγκεκριμένα πειράματα χρησιμοποιείται το μοντέλο *Llama 3.2 3B*, εφαρμόζεται 8-bit κβαντοποίηση στις ενδιάμεσες ενεργοποιήσεις, το διαθέσιμο *bandwidth* μεταξύ των κόμβων περιορίζεται στα 250 Mbps και κάθε εκτέλεση αφορά την παραγωγή 1.000 tokens. Οι *worker-A* και *worker-B* χρησιμοποιούν αποκλειστικά *CPU resources*, ενώ ο *worker-C* διαθέτει πρόσβαση στην GPU με 12 GB VRAM.

Table 4. Ανάθεση επιπέδων του Llama 3.2 3B για τις στρατηγικές ανάθεσης Single (CPU/GPU) Node, Greedy, Balanced.

| Allocation      | worker-A          | worker-B | worker-C           |
|-----------------|-------------------|----------|--------------------|
| Single CPU Node | [0 - 27] + output | –        | –                  |
| Single GPU Node | –                 | –        | [0 - 27] + output  |
| Greedy          | [0 - 7]           | [8 - 20] | [21 - 27] + output |
| Balanced        | [0 - 2]           | [3 - 5]  | [6 - 27] + output  |

Η σύγκριση των παραπάνω στρατηγικών πραγματοποιείται ως προς την απόδοση συμπερασμού, την κατανάλωση μνήμης και την κατανομή των επιμέρους καθυστερήσεων κατά μήκος της αλυσίδας.

Η απόδοση κάθε στρατηγικής αξιολογείται αρχικά μέσω των μετρικών Time-To-First-Token (TTFT), Throughput και Inter-Token Latency (ITL). Το TTFT αποτυπώνει την καθυστέρηση μέχρι την παραγωγή του πρώτου token και περιλαμβάνει το στάδιο prefill και την αρχικοποίηση των KV caches. Το ITL εκφράζει τη μέση καθυστέρηση μεταξύ διαδοχικών tokens κατά το decoding, ενώ το Throughput αποτυπώνει τον συνολικό ρυθμό παραγωγής tokens ανά δευτερόλεπτο.

Table 5. Σύγκριση μετρικών Time-To-First-Token (TTFT), Throughput και Inter-Token Latency (ITL) για τις τέσσερις στρατηγικές.

| Allocation      | TTFT      | Throughput | ITL       |
|-----------------|-----------|------------|-----------|
| Single CPU Node | 302.69 ms | 5.28 TPS   | 189.49 ms |
| Single GPU Node | 407.47 ms | 32.57 TPS  | 30.68 ms  |
| Greedy          | 625.08 ms | 6.21 TPS   | 160.95 ms |
| Balanced        | 516.94 ms | 11.63 TPS  | 85.97 ms  |

Οι διαμορφώσεις με πολλαπλούς server nodes εμφανίζουν υψηλότερο TTFT σε σύγκριση με τις single-node στρατηγικές. Η αύξηση αυτή σχετίζεται με το γεγονός ότι το στάδιο prefill πρέπει να εκτελεστεί διαδοχικά σε περισσότερα στάδια της αλυσίδας, ενώ απαιτείται επίσης η αρχικοποίηση των αντίστοιχων KV caches σε κάθε ενεργό server node.

Η υψηλότερη συνολική απόδοση παρατηρείται στη στρατηγική Single GPU Node, όπου ο worker-C διαθέτει επαρκή VRAM για τη φόρτωση ολόκληρου του μοντέλου. Η συγκεκριμένη περίπτωση οφείλεται τόσο στην αυξημένη υπολογιστική ισχύ της GPU όσο και στην απουσία communication overhead μεταξύ διαφορετικών σταδίων του pipeline. Αντίθετα, η εκτέλεση

αποκλειστικά στον CPU worker εμφανίζει σημαντικά χαμηλότερο throughput, καθώς δεν αξιοποιούνται οι υπολογιστικές δυνατότητες της GPU.

Ιδιαίτερη σημασία παρουσιάζει η σύγκριση μεταξύ των δύο κατανεμημένων στρατηγικών. Η Greedy Allocation επιτυγχάνει throughput 6.21 TPS, ενώ μετά την εφαρμογή της Δυναμικής Ανακατανομής Επιπέδων το throughput της Balanced Allocation αυξάνεται στα 11.63 TPS. Η βελτίωση αυτή αντιστοιχεί σε αύξηση περίπου 87%, γεγονός που καταδεικνύει ότι η απλή δυνατότητα φιλοξενίας των επιπέδων στη μνήμη των κόμβων δεν αρκεί για την επίτευξη υψηλής απόδοσης. Απαιτείται παράλληλα η κατανομή του υπολογιστικού έργου σύμφωνα με τις πραγματικές δυνατότητες επεξεργασίας των συμμετεχόντων workers.

Η βελτίωση αποτυπώνεται και στη μείωση του ITL από 160.95 ms στη Greedy Allocation σε 85.97 ms στην Balanced Allocation. Συνεπώς, η δυναμική ανακατανομή δεν βελτιώνει μόνο τον συνολικό ρυθμό παραγωγής, αλλά μειώνει σημαντικά και τον χρόνο που απαιτείται μεταξύ διαδοχικών tokens κατά τη φάση decoding.

Η εξέλιξη της συνολικής παραγωγής tokens συναρτήσει του χρόνου παρουσιάζεται στο Figure 30. Η κλίση κάθε καμπύλης αντιστοιχεί στον ρυθμό παραγωγής tokens και αποτυπώνει οπτικά τις σημαντικές διαφορές απόδοσης μεταξύ των τεσσάρων στρατηγικών.

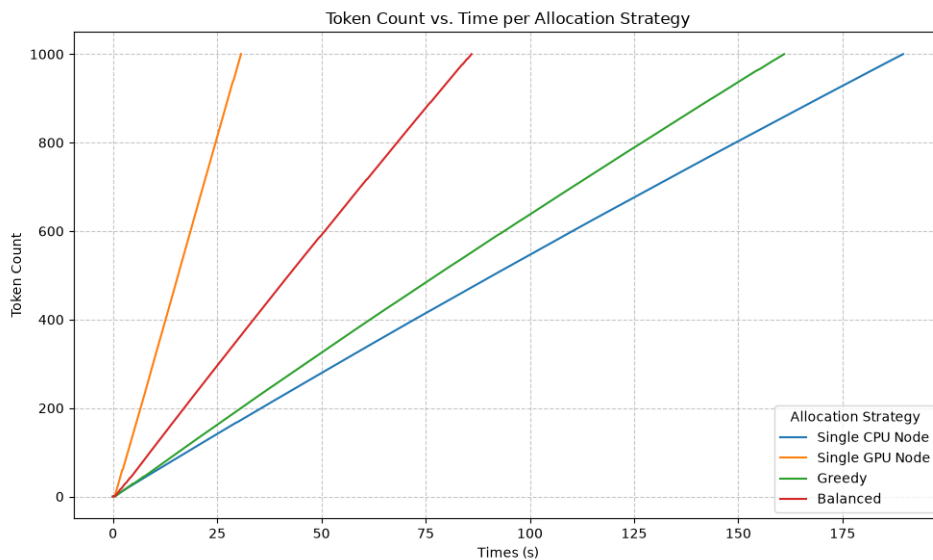


Figure 30. Συνολικός αριθμός παραγόμενων tokens συναρτήσει του χρόνου για τις στρατηγικές Single (CPU/GPU) Node, Greedy, Balanced.

Η σύγκριση αυτή δείχνει ότι το Single GPU Node αποτελεί την ταχύτερη δυνατή διαμόρφωση όταν ένα μοντέλο μπορεί να φιλοξενηθεί εξ ολοκλήρου στη διαθέσιμη VRAM. Ωστόσο, η σύγκριση μεταξύ Greedy και Balanced Allocation είναι περισσότερο αντιπροσωπευτική του βασικού στόχου της παρούσας εργασίας, καθώς εξετάζει τη συνεργατική εκτέλεση του ίδιου μοντέλου από τους ίδιους ετερογενείς κόμβους και απομονώνει την επίδραση της στρατηγικής κατανομής του υπολογιστικού φορτίου.

Η αξιολόγηση της απόδοσης συμπληρώνεται από την εξέταση της κατανάλωσης μνήμης RAM και VRAM στους συμμετέχοντες κόμβους. Η συγκεκριμένη παράμετρος είναι ιδιαίτερα σημαντική σε περιβάλλοντα κατανεμημένης εκτέλεσης, καθώς ο βασικός σκοπός της συνεργατικής προσέγγισης δεν είναι αποκλειστικά η αύξηση του throughput, αλλά και η κατανομή των απαιτήσεων μνήμης του μοντέλου σε περισσότερες συσκευές. Στη συνέχεια θα αναλυθεί η κατανάλωση μνήμης σε κάθε κόμβο για κάθε στρατηγική ανάθεσης επιπέδων.

Table 6. Κατανάλωση μνήμης (RAM και VRAM) ανά κόμβο για τις διαφορετικές στρατηγικές ανάθεσης.

|                 | worker-A  | worker-B  | worker-C  |            |
|-----------------|-----------|-----------|-----------|------------|
| Allocation      | RAM Usage | RAM Usage | RAM Usage | VRAM Usage |
| Single CPU Node | 7.66 GB   | -         | -         | -          |
| Single GPU Node | -         | -         | 1.45 GB   | 7.34 GB    |
| Greedy          | 2.56 GB   | 3.65 GB   | 1.45 GB   | 2.72 GB    |
| Balanced        | 1.47 GB   | 1.47 GB   | 1.45 GB   | 6.02 GB    |

Στη διαμόρφωση Single CPU Node το σύνολο των παραμέτρων του μοντέλου αποθηκεύεται στη μνήμη RAM του worker-A, με συνολική χρήση 7.66 GB. Αντίστοιχα, στην περίπτωση Single GPU Node ο worker-C χρησιμοποιεί 7.34 GB VRAM, καθώς το σύνολο των transformer layers και το output layer φορτώνονται στην GPU.

Οι κατανεμημένες στρατηγικές οδηγούν σε κατανομή του memory footprint μεταξύ των διαθέσιμων κόμβων. Στην Greedy Allocation, η χρήση μνήμης διαμοιράζεται πιο ισόρροπα ως προς την ποσότητα των επιπέδων που έχουν φορτωθεί, με τους worker-A και worker-B να χρησιμοποιούν 2.56 GB και 3.65 GB RAM αντίστοιχα, ενώ ο worker-C χρησιμοποιεί 2.72 GB VRAM.

Μετά τη Δυναμική Ανακατανομή, η Balanced Allocation μεταφέρει μεγαλύτερο μέρος του υπολογιστικού φόρτου στον ισχυρότερο worker-C. Ως αποτέλεσμα, η χρήση VRAM του worker-C αυξάνεται στα 6.02 GB, ενώ οι δύο CPU workers περιορίζονται περίπου στα 1.47 GB RAM ο καθένας. Η μεταβολή αυτή είναι συνεπής με τη λογική της δυναμικής εξισορρόπησης, καθώς ο ισχυρότερος κόμβος αναλαμβάνει μεγαλύτερο αριθμό transformer layers.

Παράλληλα, η κατανεμημένη Balanced Allocation εξακολουθεί να χρησιμοποιεί λιγότερη VRAM από τη στρατηγική Single GPU Node. Η διαφορά μεταξύ 7.34 GB και 6.02 GB αντιστοιχεί σε αποδέσμευση περίπου 1.32 GB VRAM στον worker-C. Το αποτέλεσμα αυτό αναδεικνύει έναν ουσιαστικό συμβιβασμό μεταξύ απόδοσης και χρήσης πόρων: η πλήρης εκτέλεση στην GPU παρέχει το υψηλότερο throughput, αλλά η κατανεμημένη εκτέλεση επιτρέπει τη μεταφορά μέρους του memory footprint σε άλλες διαθέσιμες συσκευές.

Η δυνατότητα αυτή αποκτά ιδιαίτερη σημασία σε μοντέλα μεγαλύτερου μεγέθους, των οποίων οι απαιτήσεις VRAM ενδέχεται να υπερβαίνουν τη χωρητικότητα μιας μεμονωμένης κάρτας γραφικών. Στο πλαίσιο της παρούσας αξιολόγησης, η Balanced Allocation δείχνει ότι η συνεργατική χρήση CPU και GPU nodes μπορεί να επιτύχει σημαντικά καλύτερη απόδοση από μια μη ισορροπημένη κατανεμημένη στρατηγική, ενώ παράλληλα περιορίζει τις απαιτήσεις μνήμης που αναλαμβάνει ένας μεμονωμένος κόμβος.

Για την πληρέστερη κατανόηση των διαφορών απόδοσης μεταξύ των στρατηγικών κατανομής αναλύονται οι επιμέρους καθυστερήσεις που εμφανίζονται σε κάθε server node κατά τη διαδικασία inference.

Για κάθε κόμβο καταγράφονται οι ακόλουθες συνιστώσες καθυστέρησης:

- χρόνος αποσειριοποίησης των εισερχόμενων δεδομένων (**deserialization delay**),
- χρόνος εκτέλεσης των transformer layers (**inference delay**),
- χρόνος επιλογής του επόμενου token στον Tail node (**logit sampling delay**),
- χρόνος σειριοποίησης των intermediate activations (**serialization delay**),
- χρόνος μετάδοσης των δεδομένων στον επόμενο κόμβο (**transmission delay**).

Οι κατανομές των παραπάνω καθυστερήσεων παρουσιάζονται στα Figures 31 – 33.

Στη συνέχεια, παρουσιάζονται stacked bar charts με τις εξής καθυστερήσεις για κάθε κόμβο: deserialization delay, inference delay, logit sampling delay (μόνο στον Tail node), serialization delay και transmission delay.

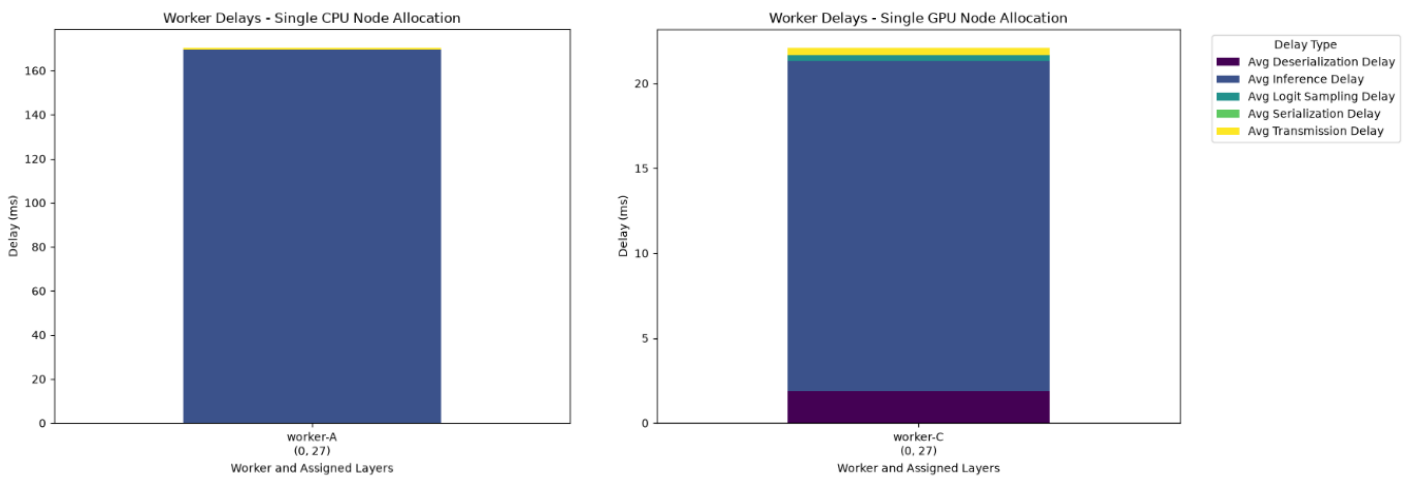


Figure 31. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για τις στρατηγικές Single CPU Node (αριστερά) και Single GPU Node (δεξιά) allocation.

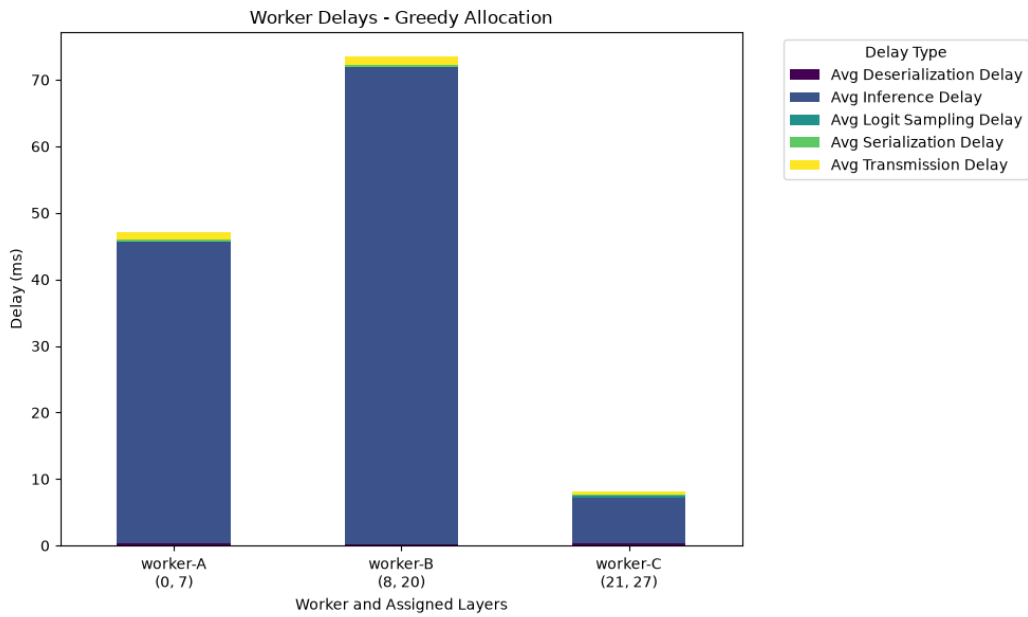


Figure 32. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για την στρατηγική Greedy allocation.

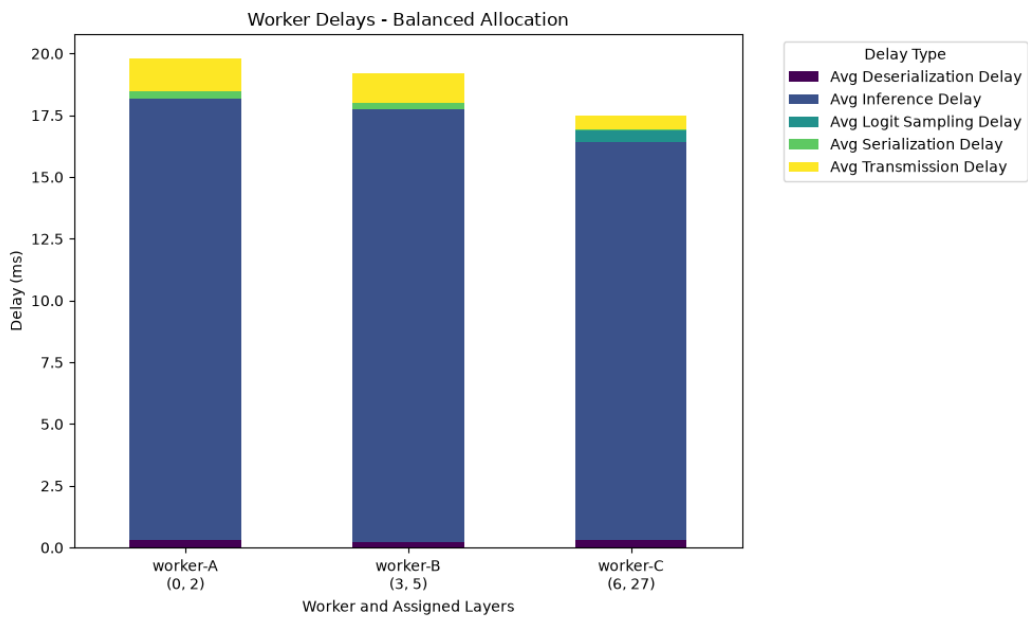


Figure 33. Κατανομή των μέσων καθυστερήσεων ανά κόμβο για την στρατηγική Balanced Allocation.

Η ανάλυση των stacked bar charts δείχνει ότι, για το συγκεκριμένο hardware configuration και bandwidth των 250 Mbps, το κύριο bottleneck της αλυσίδας είναι ο χρόνος **inference** και όχι η μετάδοση των ενδιάμεσων ενεργοποιήσεων. Η παρατήρηση αυτή υποδηλώνει ότι υπό σχετικά γρήγορες συνθήκες τοπικού δικτύου η συνολική απόδοση του pipeline εξαρτάται κυρίως από το κατά πόσο το υπολογιστικό έργο έχει κατανεμηθεί σύμφωνα με τις πραγματικές δυνατότητες των συμμετεχόντων nodes.

Στη Greedy Allocation, η ανάθεση των επιπέδων καθορίζεται κυρίως από τη διαθέσιμη μνήμη κάθε κόμβου και όχι από τον πραγματικό χρόνο επεξεργασίας. Ως αποτέλεσμα, οι CPU workers

αναλαμβάνουν σχετικά μεγάλο αριθμό transformer layers και εμφανίζουν υψηλότερες καθυστερήσεις inference, δημιουργώντας μη ισορροπημένα στάδια στο pipeline.

Αντίθετα, μετά την εφαρμογή της Δυναμικής Ανακατανομής, ο ισχυρότερος GPU worker αναλαμβάνει το μεγαλύτερο μέρος των επιπέδων του μοντέλου, ενώ ο αριθμός των επιπέδων που εκτελούνται στους ασθενέστερους CPU workers μειώνεται σημαντικά. Με τον τρόπο αυτό οι χρόνοι επεξεργασίας των διαφορετικών σταδίων προσεγγίζουν περισσότερο μεταξύ τους και περιορίζεται η επίδραση του βραδύτερου κόμβου στη συνολική παραγωγή.

Η ανάλυση αυτή ερμηνεύει και τη σημαντική αύξηση του throughput από **6.21 TPS** στη Greedy Allocation σε **11.63 TPS** στην Balanced Allocation. Η αύξηση δεν προκύπτει από την προσθήκη νέου hardware, καθώς οι συμμετέχοντες workers παραμένουν οι ίδιοι, αλλά από την αποτελεσματικότερη αξιοποίηση των διαθέσιμων πόρων μέσω της προσαρμογής της κατανομής των επιπέδων στις επιδόσεις κάθε κόμβου.

Τα αποτελέσματα υποστηρίζουν επομένως τη βασική σχεδιαστική επιλογή της προτεινόμενης αρχιτεκτονικής: σε ετερογενή συστήματα, η στατική ή αποκλειστικά memory-driven κατανομή των transformer layers μπορεί να οδηγήσει σε σημαντική υποχρησιμοποίηση των ισχυρότερων συσκευών και σε bottlenecks στους ασθενέστερους κόμβους. Η δυναμική αξιολόγηση των πραγματικών ρυθμών επεξεργασίας και η επακόλουθη ανακατανομή του μοντέλου επιτρέπουν καλύτερη εξισορρόπηση του pipeline και σημαντική βελτίωση της συνολικής απόδοσης.

## 7.4 Αποτελεσματικότητα της Κβαντοποίησης

Παρότι στα προηγούμενα πειράματα, υπό bandwidth 250 Mbps, το κύριο bottleneck της αλυσίδας εντοπίστηκε στον χρόνο inference, η σχετική συμβολή της δικτυακής επικοινωνίας αυξάνεται σημαντικά όταν το διαθέσιμο εύρος ζώνης περιορίζεται και ο αριθμός των σταδίων του pipeline μεγαλώνει. Για να διερευνηθεί αυτή η μεταβολή, εξετάζεται η εκτέλεση του Llama 3.2 1B σε αλυσίδα εννέα server nodes με διαθέσιμο bandwidth 5 Mbps. Όπως φαίνεται στο Figure 34, σε αυτές τις συνθήκες η καθυστέρηση μετάδοσης των ενδιάμεσων ενεργοποιήσεων γίνεται πλέον σημαντικότερο μέρος της συνολικής καθυστέρησης, αναδεικνύοντας την ανάγκη περιορισμού του μεγέθους του μεταδιδόμενου payload.

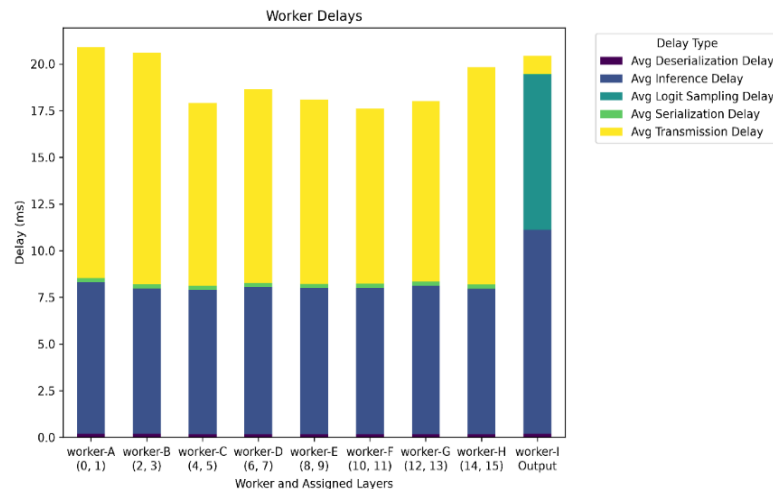


Figure 34. Κατανομή των μέσων καθυστερήσεων ανά κόμβο σε αλυσίδα εκτέλεσης με 9 server nodes με περιορισμένο bandwidth (5 Mbps).

### 7.4.1 Ανάλυση Μεγέθους του Payload

Για τον υπολογισμό του payload size στην περίπτωση απουσίας κβαντοποίησης, λαμβάνεται υπόψιν μόνο το μέγεθος του tensor των ενεργοποιήσεων. Στην περίπτωση της κβαντοποίησης 8-bit προστίθεται το μέγεθος του tensor των 8-bit indices και του tensor των τιμών absmx.

Το σύστημα σε κάθε περίπτωση επεξεργάζεται ένα prompt με μέγεθος ακολουθίας 24 tokens.

Table 7. Μέγεθος ωφέλιμου φορτίου (payload size) ενδιάμεσων ενεργοποιήσεων ανά στάδιο παραγωγής με και χωρίς κβαντοποίηση 8-bit για τα μοντέλα Llama 3.

| Model        | Generation Stage | Payload Size |                 |
|--------------|------------------|--------------|-----------------|
|              |                  | 16-bit       | 8-bit Quantized |
| Llama 3.2 1B | Prefill          | 96 KB        | 48.05 KB        |
|              | Decoding         | 4 KB         | 2.002 KB        |
| Llama 3.2 3B | Prefill          | 144 KB       | 72.07 KB        |
|              | Decoding         | 6 KB         | 3.003 KB        |
| Llama 3.1 8B | Prefill          | 192 KB       | 96.1 KB         |
|              | Decoding         | 8 KB         | 4.004 KB        |

Το μέγεθος του payload αυξάνεται καθώς αυξάνονται οι παράμετροι του μοντέλου μιας και αυξάνεται η διάσταση του κρυφού επιπέδου (hidden dimension).

### 7.4.2 Ανάλυση Μείωσης του Χρόνου Μετάδοσης

Για να έχει νόημα η εφαρμογή της κβαντοποίησης πρέπει ο χρόνος κβαντοποίησης, σειριοποίησης, μετάδοσης και αποκβαντοποίησης στον επόμενο κόμβο να είναι μικρότερος από ό,τι είναι στην περίπτωση απουσίας της κβαντοποίησης.

$$T_{\text{quantize}} + T_{\text{transmit 8-bit}} + T_{\text{dequantize}} < T_{\text{serialize}} + T_{\text{transmit 16-bit}} + T_{\text{deserialize}}$$

Στις επόμενες γραφικές παραστάσεις απεικονίζεται η μέση καθυστέρηση δικτύου στο στάδιο decoding (συμπεριλαμβανομένου του χρόνου σειριοποίησης και αποσειριοποίησης) με την εφαρμογή και μη της κβαντοποίησης για ένα εύρος bandwidths από 1 Mbps έως 1000 Mbps.

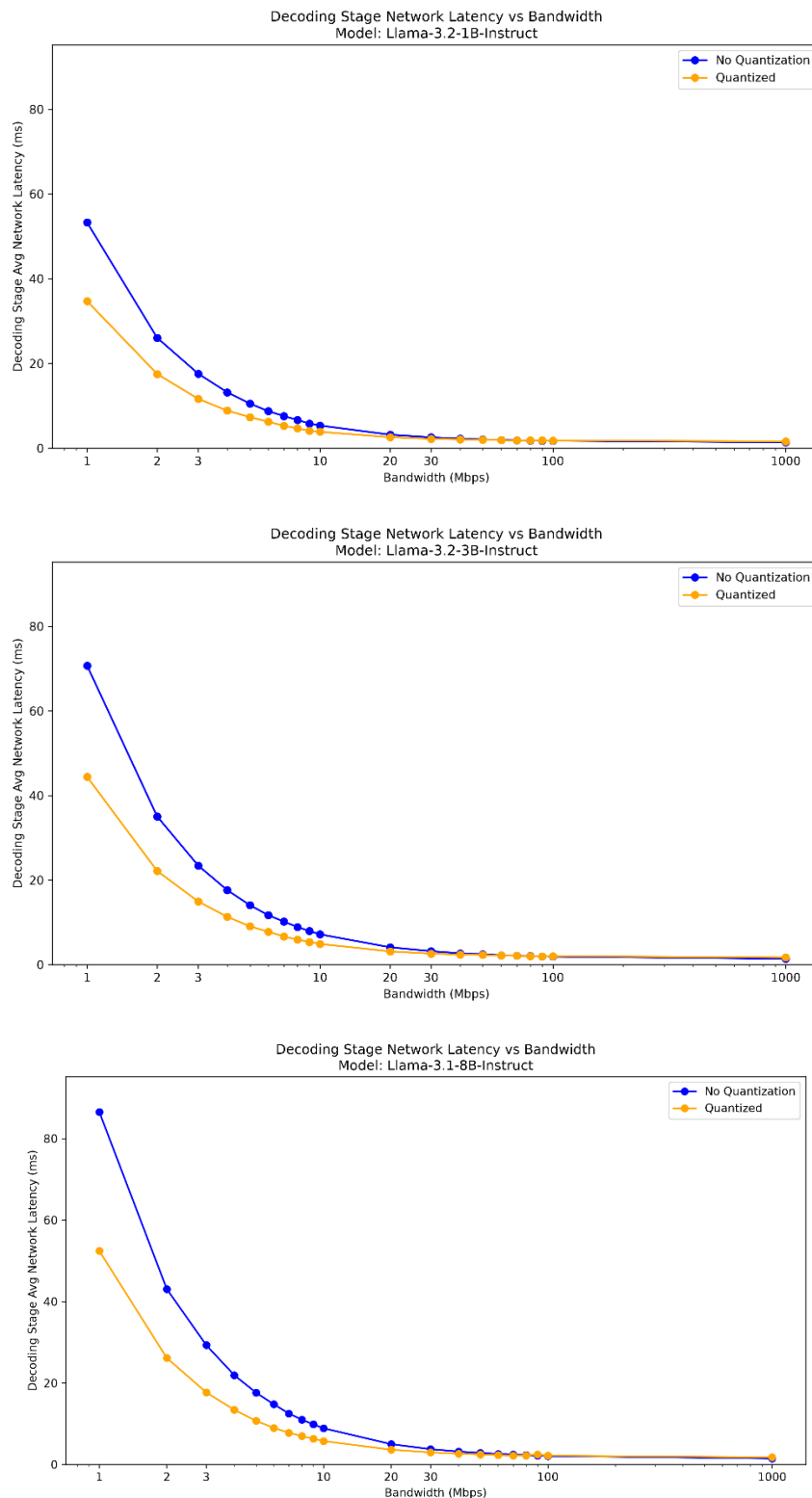


Figure 35. Σύγκριση μέσης καθυστέρησης δικτύου συναρτήσει του bandwidth (1-1000 Mbps) με και χωρίς κβαντοποίηση για τα μοντέλα Llama 3.2 1B, 3B και Llama 3.1 8B.

Παρατηρείται πως για bandwidths μικρότερα των 30 Mbps η κβαντοποίηση των ενδιάμεσων ενεργοποιήσεων προσφέρει σημαντική μείωση της καθυστέρησης μετάδοσης από κόμβο σε

κόμβο, ειδικά καθώς ο αριθμός παραμέτρων του μοντέλου και αντίστοιχα το payload size αυξάνονται. Ενώ σε ταχύτερες συνθήκες δικτύου η κβαντοποίηση φαίνεται να εισάγει μια μικρή αύξηση στην καθυστέρηση της τάξης των 0.1 ms για κάθε token.

Το σύστημα όμως προορίζεται και για τοπικά δίκτυα Wi-Fi, τα οποία μπορεί να είναι ασταθή και να παρουσιάζουν πτώσεις στο bandwidth.

Στο παρακάτω πείραμα εισάγεται περιορισμός του bandwidth στα 10Mbps για 1λεπτό, μετά την παραγωγή των πρώτων 100 tokens. Σκοπός του πειράματος είναι η παρουσίαση της ανθεκτικότητας που παρέχει η κβαντοποίηση στο σύστημα σε περιπτώσεις απρόβλεπτης πτώσης της ταχύτητας δικτύου.

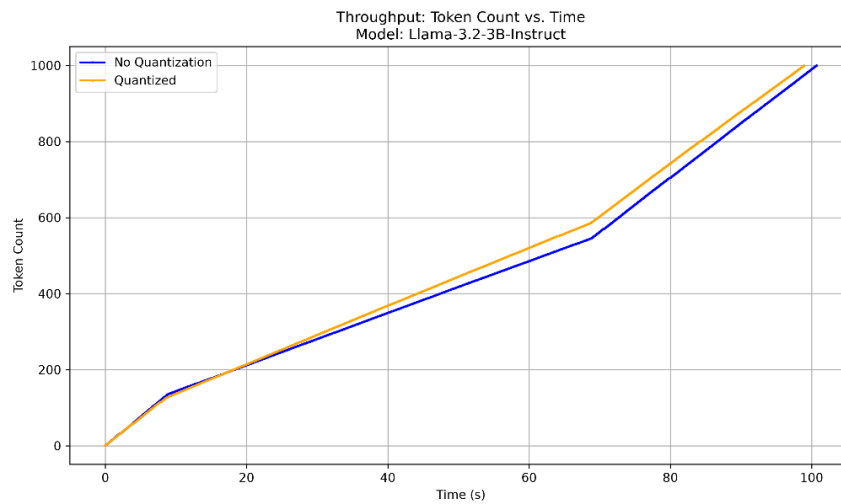


Figure 36. Επίδραση της κβαντοποίησης στο throughput υπό συνθήκες απρόβλεπτης πτώσης της ταχύτητας δικτύου στα 10 Mbps.

Παρατηρείται ότι με την πτώση της ταχύτητας του δικτύου, το throughput του συστήματος παρουσιάζει μικρότερη πτώση στην περίπτωση εφαρμογής της κβαντοποίησης.

Συνεπώς, για την προστασία της απόδοσης του συστήματος από απρόβλεπτες πτώσεις στην ταχύτητα του δικτύου, η 8-bit κβαντοποίηση των ενεργοποιήσεων αποτελεί μια αποτελεσματική προσθήκη.

## 7.5 Πολλαπλοί Clients

Το σύστημα της διπλωματικής υποστηρίζει την παραγωγή κειμένου από πολλαπλούς χρήστες παράλληλα, όπου τα αιτήματα των χρηστών διοχετεύονται αποδοτικά στην αλυσίδα εκτέλεσης αποφεύγοντας τη δημιουργία pipeline bubbles και μεγιστοποιώντας την συνολική απόδοση.

Στα ακόλουθα πειράματα η αλυσίδα εκτέλεσης αποτελείται από 3 server nodes με την παρακάτω ισορροπημένη ανάθεση επιπέδων.

|        | worker-A (CPU) | worker-B (CPU) | worker-C (CUDA)   |
|--------|----------------|----------------|-------------------|
| Layers | [0 - 2]        | [3 - 5]        | [6 - 27] + output |

Η αλυσίδα εκτέλεσης χρησιμοποιείται για την παραγωγή 1000 tokens συνολικά από 1 έως 3 clients παράλληλα και καταγράφονται οι χρονικές στιγμές παραγωγής του κάθε token από τον Tail server (worker-C) ανεξάρτητα από τον client-προορισμό.

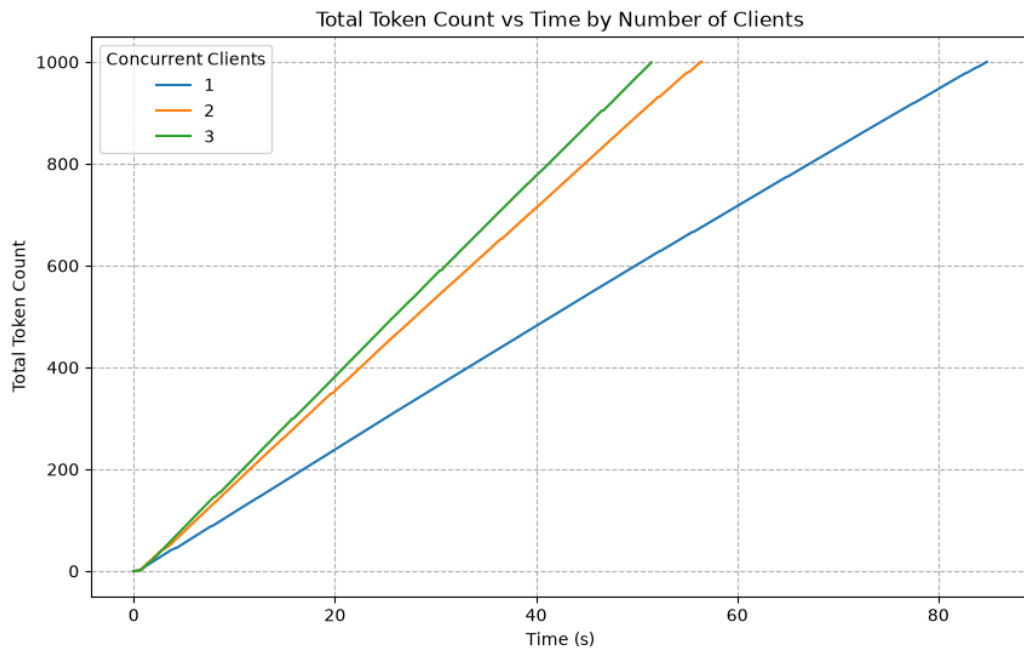


Figure 37. Χρονοδιάγραμμα παραγωγής των συνολικών tokens του συστήματος κατά την ταυτόχρονη εξυπηρέτηση 1 έως 3 clients από αλυσίδα 3 server nodes.

Στη συνέχεια μετρίεται το συνολικό throughput της αλυσίδας κατά την διάρκεια παραγωγής κειμένου από 1 έως 5 χρήστες παράλληλα.

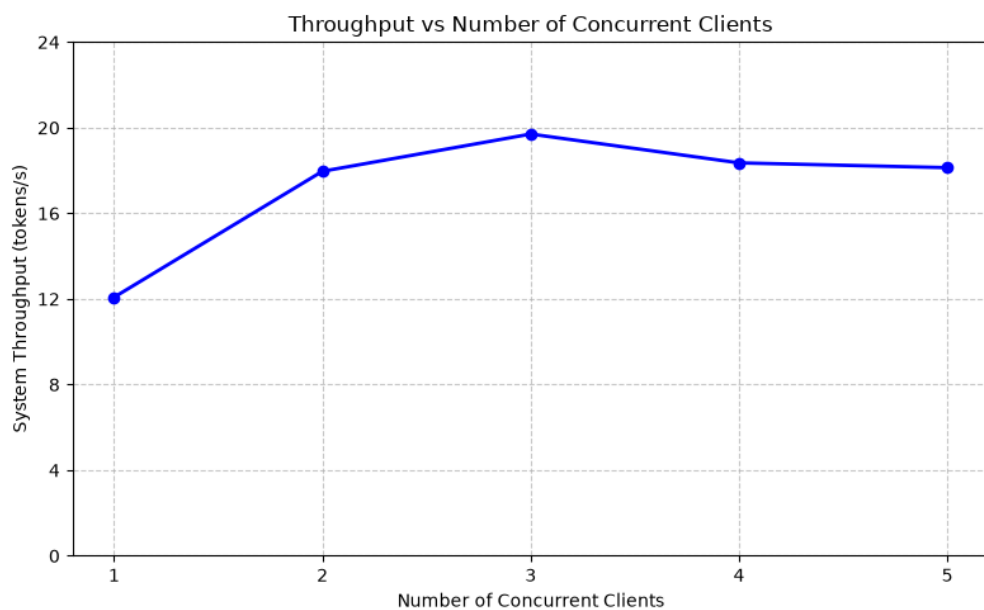


Figure 38. Throughput του συστήματος κατά την ταυτόχρονη εξυπηρέτηση 1 έως 5 clients από αλυσίδα 3 server nodes.

Παρατηρείται πως η μέγιστη απόδοση της αλυσίδας εκτέλεσης επιτυγχάνεται όταν ο αριθμός των παράλληλων clients είναι ίσος με τον αριθμό των ενεργών server nodes και η αλυσίδα είναι εξισορροπημένη.

## 7.6 Ευκαιριακή Ανάλυση και Επισκευή Αλυσίδας

### 7.6.1 Αποτελεσματικότητα Ευκαιριακής Ανάλυσης

Ο μηχανισμός της Ευκαιριακής Ανάλυσης Επιπέδων θα αξιολογηθεί, σε μια τοπολογία όπου εμφανίζεται ένας ενεργός κόμβος με χαμηλό ρυθμό επεξεργασίας, συγκρίνοντας την απόδοση του συστήματος με την εφαρμογή και μη του μηχανισμού.

Ο μειωμένος ρυθμός επεξεργασίας του worker-B επιτυγχάνεται μέσω της λειτουργίας *stress* του εργαλείου Pumba. Για τα πειράματα χρησιμοποιείται το μοντέλο Llama 3.2 3B.

|                 | worker-A       | worker-B      | worker-C           | worker-D       |
|-----------------|----------------|---------------|--------------------|----------------|
| Layers          | [0 - 9]        | [10 – 19]     | [20 – 27] + output | -              |
| Processing Rate | 193 layers/sec | 58 layers/sec | 193 layers/sec     | 193 layers/sec |
| Status          | Active         | Active        | Active             | Backup         |

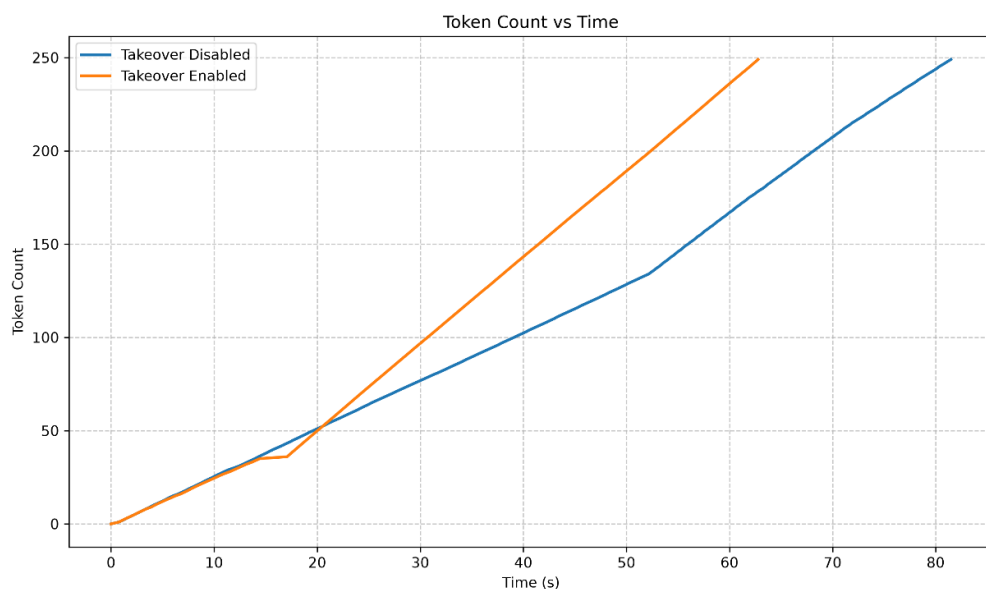


Figure 39. Επίδραση του μηχανισμού της Ευκαιριακής Ανάλυσης στο throughput κατά την παραγωγή 250 tokens, με χρόνο αντικατάστασης του αδύναμου κόμβου στα 2.64 δευτερόλεπτα.

Στην περίπτωση όπου η λειτουργία της Ευκαιριακής Ανάλυσης είναι ενεργοποιημένη, ο εφεδρικός κόμβος (worker-D) εντοπίζει τον αδύναμο κόμβο (worker-B) την χρονική στιγμή  $t = 14.43s$ , εκκινεί την διαδικασία αντικατάστασης του και η παραγωγή σταματάει προσωρινά. Η διαδικασία διαρκεί 2.64 δευτερόλεπτα και στη συνέχεια το σύστημα εμφανίζει πολύ υψηλότερο throughput.

Από το παραπάνω πείραμα συμπεραίνεται ότι ακόμα και κατά την διάρκεια παραγωγής ενός μικρού αριθμού από tokens (250) ο μηχανισμός της Ευκαιριακής Ανάλυσης μπορεί να αυξήσει σημαντικά την συνολική απόδοση του συστήματος λόγω της σχετικά μικρής διάρκειάς του.

### 7.6.2 Χρόνος Επισκευής Αλυσίδας

Στη συνέχεια, θα αξιολογηθεί η λειτουργία της επισκευής της αλυσίδας κατά την αποτυχία ενός ενδιάμεσου server node (worker-B). Ανάλογα με τη διαθέσιμη μνήμη του προκατόχου του αποτυχημένου κόμβου, τα ορφανά επίπεδα αναλαμβάνονται είτε από έναν εφεδρικό κόμβο είτε από τον προκατόχο. Για τα παρακάτω πειράματα χρησιμοποιείται το μοντέλο Llama 3.2 3B.

Στην πρώτη περίπτωση όλοι οι κόμβοι διαθέτουν 4GB RAM και ο worker-A δεν μπορεί να αναλάβει τα επίπεδα του αποτυχημένου κόμβου (worker-B), λόγω περιορισμένης μνήμης:

|              | worker-A       | worker-B       | worker-C           | worker-D       |
|--------------|----------------|----------------|--------------------|----------------|
| Memory Usage | 2.98 / 4.00 GB | 2.98 / 4.00 GB | 3.28 / 4.00 GB     | 0.67 / 4.00 GB |
| Layers       | [0 – 9]        | [10 – 19]      | [20 - 27] + output | -              |
| Status       | Active         | Active         | Active             | Backup         |

Μετά την επισκευή:

|              | worker-A       | worker-D       | worker-C           |
|--------------|----------------|----------------|--------------------|
| Memory Usage | 3.00 / 4.00 GB | 3.00 / 4.00 GB | 3.29 / 4.00 GB     |
| Layers       | [0 – 9]        | [10 – 19]      | [20 – 27] + output |
| Status       | Active         | Active         | Active             |

Στην δεύτερη περίπτωση ο worker-A διαθέτει την διπλάσια μνήμη και μπορεί να αναλάβει τα ορφανά επίπεδα.

|              | worker-A       | worker-B       | worker-C           | worker-D       |
|--------------|----------------|----------------|--------------------|----------------|
| Memory Usage | 2.98 / 8.00 GB | 2.98 / 4.00 GB | 3.28 / 4.00 GB     | 0.67 / 4.00 GB |
| Layers       | [0 – 9]        | [10 – 19]      | [20 – 27] + output | -              |
| Status       | Active         | Active         | Active             | Backup         |

Μετά την επισκευή:

|              | worker-A       | worker-C           | worker-D       |
|--------------|----------------|--------------------|----------------|
| Memory Usage | 5.20 / 8.00 GB | 3.29 / 4.00 GB     | 0.67 / 4.00 GB |
| Layers       | [0 – 19]       | [20 – 27] + output | -              |
| Status       | Active         | Active             | Backup         |

Ο τερματισμός του worker-B πυροδοτείται μετά την παραγωγή των πρώτων 100 tokens. Αμέσως μετά η αποτυχία εντοπίζεται από τον worker-A και ξεκινάει η επισκευή της αλυσίδας. Μετά την επισκευή η παραγωγή συνεχίζεται μέχρι να παραχθούν τα υπόλοιπα 400 tokens.

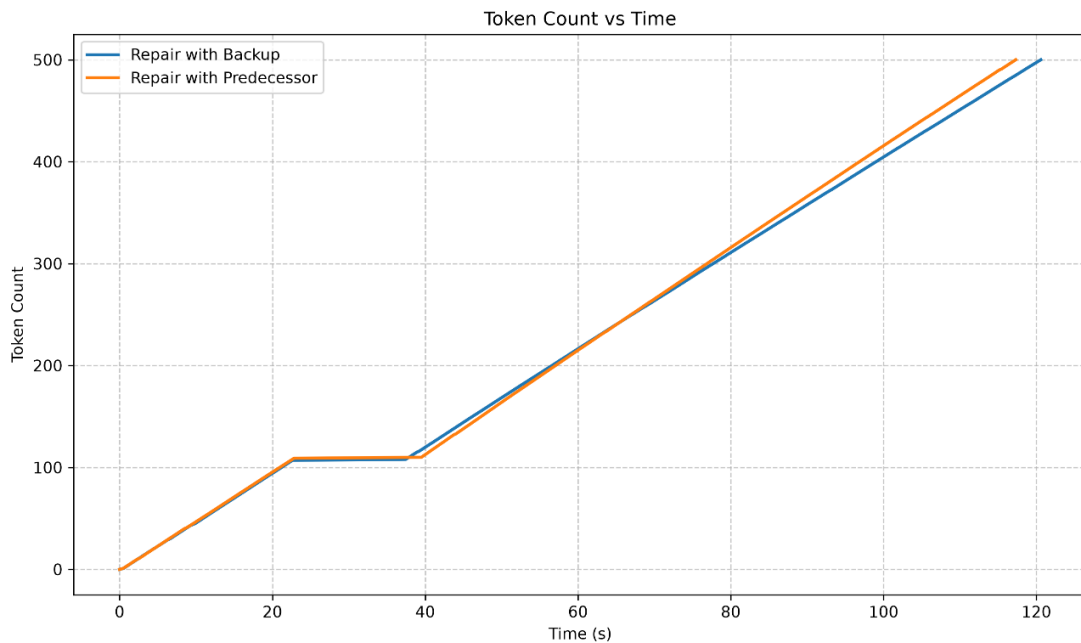


Figure 40. Σύγκριση εξέλιξης της παραγωγής tokens κατά την αποτυχία του worker-B (στα 100 tokens) και την επισκευή της αλυσίδας χρησιμοποιώντας εφεδρικό κόμβο έναντι προκατόχου.

Στην περίπτωση επισκευής με τον εφεδρικό κόμβο (worker-D), η διαδικασία διαρκεί 15.21 δευτερόλεπτα, ενώ στην επισκευή με τον κόμβο που εντόπισε την αποτυχία (worker-A), διαρκεί 16.67 δευτερόλεπτα. Η επισκευή στην δεύτερη περίπτωση διαρκεί περισσότερο καθώς ο worker-A απαιτεί περισσότερο χρόνο για να φορτώσει από την αρχή τα επίπεδα που εξυπηρετούσε και τα ορφανά επίπεδα (20 επίπεδα συνολικά).

Η υψηλότερη απόδοση του συστήματος στην δεύτερη περίπτωση, συμβαίνει λόγω της μείωσης των σταδίων του pipeline από τρία σε δύο. Με δύο κόμβους στην ενεργή αλυσίδα αντί για τρεις το communication overhead κατά την παραγωγή είναι μικρότερο.

Για να καθοριστεί η μέγιστη καθυστέρηση επισκευής αλυσίδας θα εκτελεστεί το μεγαλύτερο μοντέλο στην διάθεσή μας, το Llama 3.1 8B και θα εξεταστεί μια επισκευή μέσω ανάληψης επιπέδων από προκατόχο.

Αρχικά η τοπολογία έχει την παρακάτω μορφή:

|              | <b>worker-A<br/>(CPU)</b> | <b>worker-B<br/>(CPU)</b> | <b>worker-C<br/>(CUDA)</b> |
|--------------|---------------------------|---------------------------|----------------------------|
| Memory Usage | 4.76 / 11.00 GB           | 4.51 / 5.50 GB            | 6.20 / 12.00 GB            |
| Layers       | [0 – 9]                   | [10 – 19]                 | [20 – 31] + output         |
| Status       | Active                    | Active                    | Active                     |

Μετά την επισκευή:

|              | <b>worker-A<br/>(CPU)</b> | <b>worker-C<br/>(CUDA)</b> |
|--------------|---------------------------|----------------------------|
| Memory Usage | 9.33 / 11.00 GB           | 6.72 / 12.00 GB            |
| Layers       | [0 – 19]                  | [20 – 31] + output         |
| Status       | Active                    | Active                     |

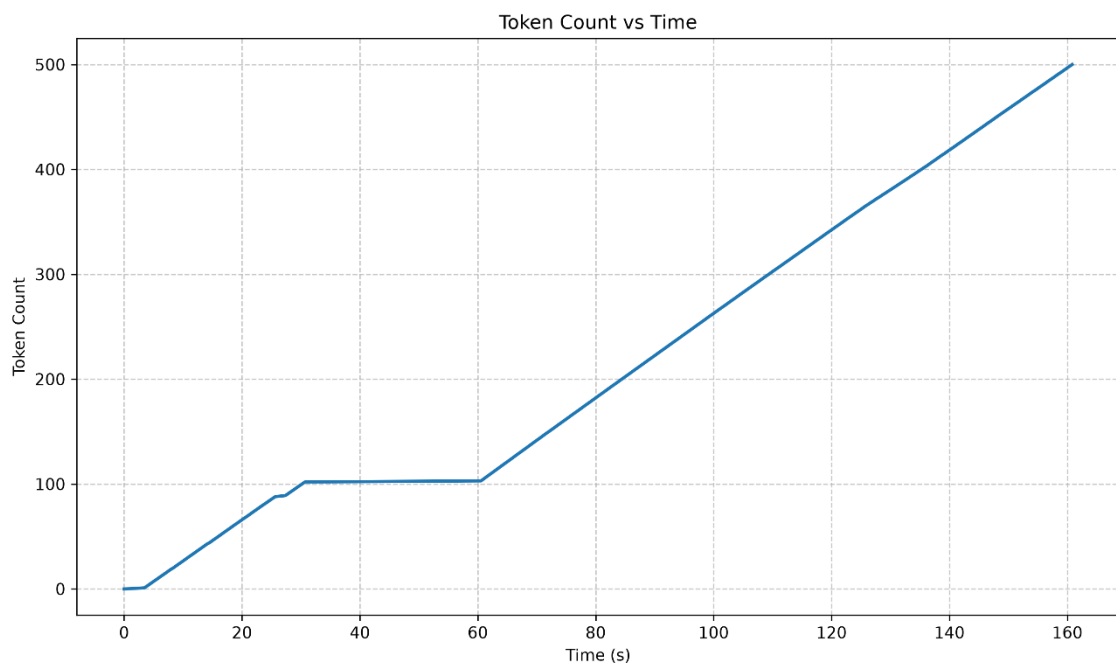


Figure 41. Εξέλιξη παραγωγής tokens κατά την επιδιόρθωση αλυσίδας για το μοντέλο Llama 3.1 8B, αναδεικνύοντας τον αυξημένο χρόνο επισκευής (29.84 δευτερόλεπτα) λόγω του μεγέθους των παραμέτρων.

Η επισκευή της αλυσίδας σε αυτή την περίπτωση διαρκεί 29.84 δευτερόλεπτα, λόγω του αυξημένου μεγέθους του μοντέλου.

## 7.7 Σύνοψη Αποτελεσμάτων

Η πειραματική αξιολόγηση της προτεινόμενης αρχιτεκτονικής καταδεικνύει ότι η συνεργατική εκτέλεση Μεγάλων Γλωσσικών Μοντέλων μέσω pipeline parallelism σε ετερογενείς συσκευές ευρείας κατανάλωσης είναι εφικτή. Παράλληλα, τα αποτελέσματα αναδεικνύουν ότι η αποδοτικότητα ενός τέτοιου συστήματος δεν εξαρτάται αποκλειστικά από το συνολικό άθροισμα των διαθέσιμων υπολογιστικών πόρων, αλλά κυρίως από τον τρόπο με τον οποίο αυτοί κατανέμονται και αξιοποιούνται κατά μήκος της αλυσίδας εκτέλεσης. Η ετερογένεια των κόμβων, οι περιορισμοί μνήμης, το διαθέσιμο εύρος ζώνης του δικτύου και οι μεταβολές της απόδοσης των συμμετεχουσών συσκευών επηρεάζουν άμεσα τη συνολική απόδοση και καθιστούν αναγκαία τη δυναμική προσαρμογή της τοπολογίας και του υπολογιστικού φορτίου.

Σε σχέση με το πρώτο ερευνητικό ερώτημα (RQ1), τα πειράματα επιβεβαιώνουν ότι είναι δυνατή η συνεργατική εκτέλεση ενός LLM σε πολλαπλούς ετερογενείς κόμβους χωρίς την παρουσία μόνιμου κεντρικού κόμβου-συντονιστή. Η κατανομή των transformer layers μεταξύ CPU και GPU workers, σε συνδυασμό με τη διαχείριση της κατάστασης του δικτύου μέσω του DHT, επιτρέπει τη συγκρότηση και λειτουργία μιας πλήρους αλυσίδας συμπερασμού. Η λειτουργικότητα του συστήματος διατηρείται ακόμη και όταν το μοντέλο δεν εκτελείται εξ ολοκλήρου στον ισχυρότερο διαθέσιμο κόμβο, γεγονός που επιβεβαιώνει τη δυνατότητα αξιοποίησης συλλογικών υπολογιστικών και μνημονικών πόρων. Ωστόσο, τα αποτελέσματα δείχνουν ταυτόχρονα ότι η κατανεμημένη εκτέλεση εισάγει πρόσθετο communication και pipeline overhead σε σχέση με την εκτέλεση ολόκληρου του μοντέλου σε έναν ισχυρό GPU node. Επομένως, το βασικό πλεονέκτημα της προτεινόμενης προσέγγισης δεν είναι ότι υπερβαίνει αναγκαστικά την απόδοση ενός μεμονωμένου ισχυρού επιταχυντή, αλλά ότι επιτρέπει την αποτελεσματική αξιοποίηση πολλαπλών διαθέσιμων συσκευών όταν οι πόροι μιας επιμέρους συσκευής είναι περιορισμένοι.

Ιδιαίτερη σημασία παρουσιάζουν τα αποτελέσματα που σχετίζονται με το δεύτερο ερευνητικό ερώτημα (RQ2). Η αρχική Greedy Allocation, κατά την οποία τα επίπεδα του μοντέλου ανατίθενται κυρίως βάσει των διαθέσιμων πόρων μνήμης, οδήγησε σε throughput 6.21 TPS. Μετά την εφαρμογή της Δυναμικής Ανακατανομής Επιπέδων, το throughput αυξήθηκε στα 11.63 TPS, παρουσιάζοντας βελτίωση περίπου 87%. Παράλληλα, το μέσο Inter-Token Latency μειώθηκε από 160.95 ms σε 85.97 ms. Τα αποτελέσματα αυτά καταδεικνύουν ότι η διαθέσιμη μνήμη από μόνη της δεν αποτελεί επαρκές κριτήριο για την αποτελεσματική κατανομή των transformer layers σε ένα ετερογενές περιβάλλον.

Η ανάλυση των επιμέρους καθυστερήσεων ανά κόμβο επιβεβαίωσε ότι, υπό τις βασικές συνθήκες δικτύου των πειραμάτων, το κύριο bottleneck εντοπίζεται στον χρόνο inference. Στην αρχική Greedy Allocation, οι ασθενέστεροι CPU workers αναλαμβάνουν μεγαλύτερο υπολογιστικό φορτίο από αυτό που μπορούν να εξυπηρετήσουν αποτελεσματικά, με αποτέλεσμα να περιορίζουν τον συνολικό ρυθμό του pipeline. Αντίθετα, η Δυναμική Ανακατανομή μεταφέρει μεγαλύτερο αριθμό επιπέδων στον ισχυρότερο GPU worker και μειώνει το φορτίο των CPU nodes, εξισορροπώντας περισσότερο τους χρόνους επεξεργασίας των επιμέρους σταδίων. Συνεπώς, η απόδοση της αλυσίδας καθορίζεται σε μεγάλο βαθμό από την απόδοση του βραδύτερου σταδίου και η συνεχής αξιολόγηση των πραγματικών processing rates αποτελεί κρίσιμη προϋπόθεση για τη διατήρηση υψηλού throughput.

Παράλληλα, η κατανεμημένη εκτέλεση επιτυγχάνει διαφορετική κατανομή των απαιτήσεων μνήμης μεταξύ των συσκευών. Παρόλο που το Single GPU Node εμφανίζει την υψηλότερη απόδοση, η Balanced Allocation περιορίζει τη χρήση της VRAM του GPU worker από 7.34

GB σε 6.02 GB, μεταφέροντας μέρος του μοντέλου στους διαθέσιμους CPU workers. Το εύρημα αυτό καταδεικνύει τον βασικό συμβιβασμό της συνεργατικής εκτέλεσης: η κατανομή του μοντέλου εισάγει επιπλέον καθυστερήσεις επικοινωνίας, αλλά παράλληλα επιτρέπει τον διαμοιρασμό των απαιτήσεων μνήμης και καθιστά δυνατή την εκτέλεση μεγαλύτερων μοντέλων σε περιβάλλοντα όπου καμία μεμονωμένη συσκευή δεν διαθέτει επαρκείς πόρους.

Αναφορικά με το τρίτο ερευνητικό ερώτημα (RQ3), τα αποτελέσματα καταδεικνύουν ότι η επίδραση του δικτύου εξαρτάται έντονα από το διαθέσιμο bandwidth και τον αριθμό των σταδίων του pipeline. Υπό σχετικά υψηλό εύρος ζώνης, η καθυστέρηση μετάδοσης των intermediate activations παραμένει μικρότερη από τον χρόνο inference και συνεπώς δεν αποτελεί το κυρίαρχο bottleneck. Αντίθετα, όταν το bandwidth μειώνεται σημαντικά ή αυξάνεται ο αριθμός των server nodes, η επικοινωνία μεταξύ των διαδοχικών σταδίων αποκτά μεγαλύτερη συμμετοχή στη συνολική καθυστέρηση και μπορεί να περιορίσει την απόδοση της αλυσίδας.

Η εφαρμογή 8-bit dynamic block-wise quantization μειώνει περίπου κατά 50% το μέγεθος των intermediate activation payloads και αποδεικνύεται ιδιαίτερα αποτελεσματική σε συνθήκες περιορισμένου bandwidth. Τα πειράματα με τα μοντέλα Llama 3.2 1B, Llama 3.2 3B και Llama 3.1 8B έδειξαν ότι για bandwidth μικρότερο περίπου από 30 Mbps η κβαντοποίηση μειώνει αισθητά τον συνολικό χρόνο επικοινωνίας, με το όφελος να γίνεται μεγαλύτερο καθώς αυξάνεται το μέγεθος του μοντέλου και η διάσταση των ενδιάμεσων ενεργοποιήσεων. Αντίθετα, σε ταχύτερα δίκτυα το πρόσθετο κόστος της διαδικασίας quantization/dequantization μπορεί να εισάγει μια μικρή επιβάρυνση. Το αποτέλεσμα αυτό δείχνει ότι η χρησιμότητα της κβαντοποίησης είναι συνάρτηση των δικτυακών συνθηκών και δεν πρέπει να εξετάζεται ανεξάρτητα από αυτές.

Η σημασία της συγκεκριμένης τεχνικής αναδεικνύεται περισσότερο στο πείραμα της προσωρινής πτώσης του bandwidth στα 10 Mbps. Στην περίπτωση εφαρμογής της κβαντοποίησης, η μείωση του throughput ήταν μικρότερη σε σχέση με τη μετάδοση των μη κβαντισμένων ενεργοποιήσεων. Συνεπώς, η κβαντοποίηση δεν λειτουργεί μόνο ως μηχανισμός μείωσης του communication overhead, αλλά ενισχύει και την ανθεκτικότητα της απόδοσης σε απρόβλεπτες μεταβολές των συνθηκών του τοπικού δικτύου.

Το τέταρτο ερευνητικό ερώτημα (RQ4) αφορά την ικανότητα του συστήματος να προσαρμόζεται σε υποβάθμιση ή αποτυχία ενεργών κόμβων. Τα πειράματα της Ευκαιριακής Ανάλυσης και της Επισκευής της Αλυσίδας καταδεικνύουν δύο διαφορετικές μορφές προσαρμοστικότητας. Στην πρώτη περίπτωση, ένας ενεργός κόμβος εξακολουθεί να λειτουργεί, αλλά παρουσιάζει σημαντικά μειωμένο processing rate. Ο μηχανισμός Opportunistic Takeover επιτρέπει σε έναν ισχυρότερο εφεδρικό κόμβο να εντοπίσει τη συμφόρηση και να αντικαταστήσει τον ασθενέστερο κόμβο. Στο αντίστοιχο πείραμα, η διαδικασία αντικατάστασης ολοκληρώθηκε σε 2.64 δευτερόλεπτα, επιτρέποντας στη συνέχεια τη λειτουργία της αλυσίδας με σημαντικά υψηλότερο throughput. Το αποτέλεσμα υποδεικνύει ότι το σύστημα μπορεί να προσαρμόζεται όχι μόνο σε πλήρεις αστοχίες, αλλά και σε δυναμικές μεταβολές της υπολογιστικής διαθεσιμότητάς.

Σε περίπτωση πλήρους αποτυχίας ενός ενδιάμεσου server node, η αρχιτεκτονική κατάφερε επίσης να επαναφέρει αυτόματα την αλυσίδα σε λειτουργική κατάσταση. Ανάλογα με τη διαθέσιμη μνήμη, τα ορφανά transformer layers μπορούν να αναληφθούν είτε από διαθέσιμο Backup node είτε από τον predecessor του αποτυχημένου κόμβου. Στα πειράματα με το Llama 3.2 3B, ο χρόνος επισκευής κυμάνθηκε περίπου μεταξύ 15 και 17 δευτερολέπτων, ενώ για το μεγαλύτερο Llama 3.1 8B αυξήθηκε στα 29.84 δευτερόλεπτα. Η αύξηση αυτή δείχνει ότι ο χρόνος

recovery επηρεάζεται σημαντικά από το μέγεθος των παραμέτρων που πρέπει να επαναφορτωθούν και από τους διαθέσιμους πόρους μνήμης και επεξεργασίας.

Συνεπώς, τα αποτελέσματα υποστηρίζουν την ανθεκτικότητα της αρχιτεκτονικής σε μεμονωμένες αποτυχίες κόμβων, αλλά ταυτόχρονα αναδεικνύουν έναν σημαντικό πρακτικό συμβιβασμό. Η επισκευή πραγματοποιείται αυτόματα και χωρίς ανάγκη κεντρικού coordinator, ωστόσο συνοδεύεται από προσωρινή διακοπή της παραγωγής, η οποία αυξάνεται καθώς αυξάνονται οι απαιτήσεις του μοντέλου. Επομένως, η fault tolerance της προτεινόμενης προσέγγισης αφορά κυρίως τη δυνατότητα αυτόματης αποκατάστασης και συνέχισης της εκτέλεσης και όχι την πλήρη εξάλειψη κάθε interruption κατά τη διάρκεια μιας αστοχίας.

Τέλος, σε σχέση με το πέμπτο ερευνητικό ερώτημα (RQ5), τα πειράματα πολλαπλών clients επιβεβαιώνουν ότι το recurrent pipelining μπορεί να αυξήσει την αξιοποίηση της καταναεμημένης αλυσίδας. Η διοχέτευση πολλαπλών inference requests επιτρέπει στους server nodes να επεξεργάζονται αιτήματα διαφορετικών χρηστών διαδοχικά, περιορίζοντας τα χρονικά διαστήματα αδράνειας μεταξύ των σταδίων. Η χρήση ανεξάρτητων KV caches ανά client επιτρέπει παράλληλα τη διατήρηση της κατάστασης κάθε διαφορετικής ακολουθίας.

Η μέγιστη συνολική απόδοση παρατηρήθηκε όταν ο αριθμός των ταυτόχρονων clients ήταν ίσος με τον αριθμό των ενεργών server nodes μιας εξισορροπημένης αλυσίδας. Το αποτέλεσμα αυτό είναι συνεπές με τη λειτουργία του pipeline parallelism, καθώς η ταυτόχρονη ύπαρξη επαρκούς αριθμού αιτημάτων επιτρέπει τη συνεχή απασχόληση όλων των σταδίων. Αντίθετα, όταν ο αριθμός των ενεργών αιτημάτων είναι μικρότερος, εμφανίζονται pipeline bubbles και ορισμένοι κόμβοι παραμένουν προσωρινά ανενεργοί. Επομένως, η δυνατότητα εξυπηρέτησης πολλαπλών clients δεν αποτελεί μόνο λειτουργικό χαρακτηριστικό της αρχιτεκτονικής, αλλά μπορεί να χρησιμοποιηθεί και για την αύξηση της συνολικής διεκπεραιωτικής ικανότητας του συστήματος.

Συνολικά, τα αποτελέσματα της πειραματικής αξιολόγησης απαντούν θετικά στα βασικά ερευνητικά ερωτήματα της εργασίας και υποστηρίζουν τη σκοπιμότητα της προτεινόμενης αρχιτεκτονικής. Η συνεργατική εκτέλεση LLMs σε ετερογενείς κόμβους με αποκεντρωμένη διαχείριση, είναι εφικτή, αλλά η αποδοτική λειτουργία της απαιτεί ενεργή διαχείριση τόσο των υπολογιστικών όσο και των δικτυακών πόρων. Η Δυναμική Ανακατανομή Επιπέδων αντιμετωπίζει τις ανισορροπίες που προκαλεί η ετερογένεια των συσκευών, η κβαντοποίηση των ενδιάμεσων ενεργοποιήσεων περιορίζει την επίδραση των δικτυακών περιορισμών, το recurrent pipelining βελτιώνει την αξιοποίηση της αλυσίδας σε περιβάλλον πολλαπλών χρηστών και οι μηχανισμοί Ευκαιριακής Ανάλυσης και Επισκευής επιτρέπουν την προσαρμογή του συστήματος σε μεταβολές της απόδοσης και αστοχίες κόμβων.

Τα ευρήματα αυτά καταδεικνύουν ότι η αποδοτική καταναεμημένη εκτέλεση ενός LLM σε συσκευές ευρείας κατανάλωσης δεν αποτελεί αποκλειστικά πρόβλημα κατανομής του μοντέλου, αλλά ένα συνδυαστικό πρόβλημα διαχείρισης υπολογιστικής ισχύος, μνήμης, επικοινωνίας και δυναμικής μεταβολής της τοπολογίας. Η προτεινόμενη αρχιτεκτονική αντιμετωπίζει τα παραπάνω ζητήματα ως ένα ενιαίο σύστημα, επιτρέποντας τη συνεχή προσαρμογή της αλυσίδας στις διαθέσιμες συνθήκες και δημιουργώντας τη βάση για πιο ευέλικτη και αυτόνομη τοπική εκτέλεση Μεγάλων Γλωσσικών Μοντέλων.

## 8 Συμπεράσματα και Προοπτικές

### 8.1 Σύνοψη Συστήματος

Το αναπτυχθέν σύστημα συνιστά μια αποκεντρωμένη πλατφόρμα σχεδιασμένη για την κατανεμημένη εκτέλεση Μεγάλων Γλωσσικών Μοντέλων εντός τοπικών δικτύων αποτελούμενων από ετερογενείς συσκευές. Η υλοποίηση ικανοποιεί τους αρχικούς στόχους μέσω των ακόλουθων λειτουργιών.

Η αρχιτεκτονική της οικογένειας μοντέλων Llama 3 προσαρμόστηκε μέσω της βιβλιοθήκης PyTorch, επιτρέποντας τη δυναμική φόρτωση συγκεκριμένων επιπέδων ανά κόμβο και ενσωματώνοντας εναλλασσόμενες KV caches για την ταυτόχρονη παραγωγή κειμένου από πολλούς χρήστες. Η διαχείριση της τοπολογίας καθίσταται αποκεντρωμένη μέσω ενός Κατανεμημένου Πίνακα Κατακερματισμού (DHT), εξαλείφοντας την ανάγκη για κεντρικό συντονιστή και επιτρέποντας την αυτόνομη ένταξη νέων κόμβων και την ανάληψη μη εξυπηρετούμενων επιπέδων.

Το σύστημα εξασφαλίζει υψηλή ανθεκτικότητα σε σφάλματα μέσω μηχανισμών αυτόνομης επιδιόρθωσης της αλυσίδας εκτέλεσης, αξιοποιώντας τους υφιστάμενους ενεργούς ή διαθέσιμους εφεδρικούς κόμβους. Επιπλέον, υλοποιήθηκε η λειτουργία της Δυναμικής Ανακατανομής Επιπέδων, η οποία ανακατανέμει τα επίπεδα του μοντέλου βάσει του ρυθμού επεξεργασίας και της διαθέσιμης μνήμης κάθε ενεργού κόμβου. Η απόδοση αυξάνεται περαιτέρω μέσω του μηχανισμού της Ευκαιριακής Ανάληψης, όπου ισχυρότεροι κόμβοι αντικαθιστούν αυτόνομα ασθενέστερους κόμβους της ενεργής αλυσίδας.

Η πλατφόρμα υποστηρίζει την ετερογένεια του υλικού, επιτρέποντας τη συνδυαστική χρήση επεξεργαστών (CPU) και καρτών γραφικών (GPU, με υποστήριξη Nvidia CUDA). Ο όγκος των μεταδιδόμενων δεδομένων μειώνεται σημαντικά μέσω της εφαρμογής 8-bit dynamic block-wise quantization στις ενδιάμεσες ενεργοποιήσεις. Παράλληλα, η χρήση αρχιτεκτονικής διοχέτευσης (pipelining) στα αιτήματα συμπερασμού ελαχιστοποιεί την αδράνεια των κόμβων, αυξάνοντας το συνολικό throughput. Τέλος, η αλληλεπίδραση με το σύστημα και η παρακολούθηση της κατάστασης της τοπολογίας διευκολύνονται μέσω μιας γραφικής διεπαφής χρήστη (GUI).

### 8.2 Περιορισμοί της Εργασίας

Παρά τα θετικά αποτελέσματα της πειραματικής αξιολόγησης, η παρούσα εργασία υπόκειται σε ορισμένους περιορισμούς που πρέπει να λαμβάνονται υπόψη κατά την ερμηνεία και τη γενίκευση των ευρημάτων. Αρχικά, η αξιολόγηση πραγματοποιήθηκε σε ελεγχόμενο περιβάλλον με Docker containers που εκτελούνται στο ίδιο φυσικό host. Παρότι μέσω των containers επιβλήθηκαν διαφορετικοί περιορισμοί σε CPU, RAM και πρόσβαση στην GPU, ώστε να προσομοιωθεί η ετερογένεια πραγματικών συσκευών, το περιβάλλον αυτό δεν αναπαράγει πλήρως όλες τις ιδιαιτερότητες ενός πραγματικού δικτύου ανεξάρτητων φυσικών υπολογιστικών κόμβων. Παράγοντες όπως διαφορετικά λειτουργικά συστήματα, αποκλίσεις στο υλικό, πραγματικές διακυμάνσεις του ασύρματου δικτύου, background workloads και ανεξάρτητες αστοχίες

συσκευών μπορεί να επηρεάσουν τη συμπεριφορά του συστήματος σε ένα πραγματικό deployment.

Αντίστοιχα, οι δικτυακές συνθήκες προσομοιώθηκαν μέσω του εργαλείου Pumba, με ελεγχόμενο περιορισμό του διαθέσιμου bandwidth μεταξύ των containers. Η συγκεκριμένη προσέγγιση επιτρέπει την αναπαραγωγή και συστηματική σύγκριση διαφορετικών συνθηκών επικοινωνίας, δεν περιλαμβάνει όμως όλο το εύρος των παραμέτρων που χαρακτηρίζουν ένα πραγματικό τοπικό δίκτυο, όπως μεταβολές στο packet loss, στο jitter, στην αστάθεια της σύνδεσης και στις καθυστερήσεις που προκαλούνται από ταυτόχρονη χρήση του δικτύου από άλλες εφαρμογές.

Επιπλέον, η αξιολόγηση της ετερογένειας του υλικού επικεντρώθηκε κυρίως στον συνδυασμό CPU nodes και μίας GPU. Συνεπώς, τα αποτελέσματα δεν μπορούν να γενικευθούν άμεσα σε όλες τις πιθανές κατηγορίες επιταχυντών, όπως GPUs διαφορετικών κατασκευαστών, integrated GPUs, NPUs ή άλλες εξειδικευμένες μονάδες επιτάχυνσης. Η απόδοση των μηχανισμών δυναμικής ανακατανομής και profiling μπορεί να διαφοροποιείται σημαντικά ανάλογα με τα χαρακτηριστικά του διαθέσιμου hardware.

Η πειραματική αξιολόγηση πραγματοποιήθηκε επίσης σε συγκεκριμένα μοντέλα της οικογένειας Llama 3 (Llama 3.2 1B, Llama 3.2 3B και Llama 3.1 8B). Αν και η αρχιτεκτονική έχει σχεδιαστεί ώστε να υποστηρίζει κατανεμημένη εκτέλεση decoder-only transformer models, η συμπεριφορά του συστήματος δεν αξιολογήθηκε σε διαφορετικές οικογένειες LLMs ή σε σημαντικά μεγαλύτερα μοντέλα. Ως εκ τούτου, η γενίκευση των αποτελεσμάτων σε μοντέλα με διαφορετική αρχιτεκτονική, αριθμό επιπέδων, hidden dimension ή απαιτήσεις KV cache απαιτεί επιπλέον μελέτη.

Ένας ακόμη περιορισμός αφορά την κλίμακα των πειραμάτων. Παρότι εξετάστηκαν τοπολογίες πολλαπλών server nodes και παράλληλη εξυπηρέτηση πολλών clients, η αξιολόγηση πραγματοποιήθηκε σε σχετικά περιορισμένο αριθμό κόμβων και χρηστών. Σε σημαντικά μεγαλύτερα δίκτυα, το κόστος διαχείρισης του DHT, η αύξηση των RPC επικοινωνιών, ο συντονισμός της τοπολογίας και η πιθανότητα ταυτόχρονων αστοχιών μπορεί να επηρεάσουν διαφορετικά τη συνολική απόδοση και σταθερότητα του συστήματος.

Οι μηχανισμοί ανθεκτικότητας σε σφάλματα αξιολογήθηκαν κυρίως σε σενάρια αποτυχίας μεμονωμένων server nodes και σε περιπτώσεις υποβάθμισης του processing rate ενός ενεργού worker. Επομένως, δεν έχει διερευνηθεί πλήρως η συμπεριφορά της αρχιτεκτονικής σε πιο σύνθετες περιπτώσεις, όπως ταυτόχρονες αποτυχίες πολλαπλών κόμβων ή παρατεταμένη αστάθεια του δικτύου. Τα πειράματα έδειξαν ότι η αποκατάσταση της αλυσίδας είναι εφικτή, αλλά συνοδεύεται από προσωρινή διακοπή της παραγωγής, με τον χρόνο αποκατάστασης να αυξάνεται για μεγαλύτερα μοντέλα.

Τέλος, η αξιολόγηση της 8-bit κβαντοποίησης επικεντρώθηκε κυρίως στη μείωση του μεγέθους του μεταδιδόμενου payload και στην επίδρασή της στη δικτυακή καθυστέρηση και στο throughput. Δεν πραγματοποιήθηκε ανεξάρτητη αξιολόγηση της πιθανής επίδρασης της κβαντοποίησης στην ποιότητα της παραγόμενης εξόδου του μοντέλου. Παρότι η συγκεκριμένη τεχνική έχει χρησιμοποιηθεί σε συναφή συστήματα κατανεμημένης εκτέλεσης, η συστηματική αξιολόγηση της ακρίβειας ή της ποιότητας εξόδου στο πλαίσιο της παρούσας υλοποίησης αποτελεί αντικείμενο περαιτέρω διερεύνησης.

Οι παραπάνω περιορισμοί δεν αναιρούν τη λειτουργικότητα ή τα αποτελέσματα της προτεινόμενης αρχιτεκτονικής, αλλά καθορίζουν το πλαίσιο μέσα στο οποίο πρέπει να ερμηνεύονται.

Παράλληλα, αναδεικνύουν συγκεκριμένες κατευθύνσεις για μελλοντική αξιολόγηση σε μεγαλύτερης κλίμακας και περισσότερο ετερογενή και πλήρως πραγματικά περιβάλλοντα κατανα-  
μημένης εκτέλεσης.

### 8.3 Μελλοντική Εργασία

Για την μελλοντική βελτίωση του συστήματος, προτείνονται οι ακόλουθες επεκτάσεις, που α-  
φορούν την εξέλιξη της αρχιτεκτονικής του συστήματος.

Αρχικά, ο μηχανισμός Ευκαιριακής Ανάλυσης μπορεί να εξελιχθεί ώστε να επιτρέπει τη με-  
ρική ανάλυση επιπέδων. Εάν ένας αντικαταστάτης κόμβος δεν διαθέτει επαρκή μνήμη για την  
πλήρη ανάλυση των επιπέδων ενός αδύναμου κόμβου, η ανάλυση έστω και ενός υποσυνόλου  
αυτών θα συνέβαλε στη μείωση της υπολογιστικής συμφόρησης και στη βελτίωση της απόδο-  
σης. Παράλληλα, οι υφιστάμενοι μηχανισμοί Επισκευής της Αλυσίδας πρέπει να εμπλουτι-  
στούν ώστε το σύστημα να μπορεί να διαχειρίζεται την ταυτόχρονη αποτυχία συνεχόμενων  
κόμβων.

Δεδομένου ότι σε τοπικά δίκτυα υψηλού εύρους ζώνης, όπου χρησιμοποιείται υλικό περιορι-  
σμένων υπολογιστικών δυνατοτήτων, το κύριο σημείο συμφόρησης εντοπίζεται στη διαδικα-  
σία συμπερασμού, προτείνεται η χρήση κβαντισμένων LLMs. Η εφαρμογή τεχνικών συμπίε-  
σης στα στατικά βάρη του μοντέλου θα μειώσει περαιτέρω τις απαιτήσεις μνήμης και θα μειώ-  
σει τον χρόνο συμπερασμού ανά κόμβο. Συμπληρωματικά, στην ίδια κατεύθυνση, προτείνεται  
η προσθήκη υποστήριξης μοντέλων Mixture of Experts (MoE), τα οποία ενεργοποιούν μόνο  
ένα υποσύνολο των παραμέτρων τους, μειώνοντας το inference delay.

# Βιβλιογραφία

- [1] Meta AI, “Llama 3.2: Revolutionizing edge AI and vision with open, customizable models,” Meta AI Blog. Accessed: Jun. 03, 2026. [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- [2] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, “Large language models in medicine,” *Nat. Med.*, vol. 29, no. 8, pp. 1930–1940, Aug. 2023, doi: 10.1038/s41591-023-02448-8.
- [3] S. Wu *et al.*, “BloombergGPT: A Large Language Model for Finance,” Dec. 2023, [Online]. Available: <http://arxiv.org/abs/2303.17564>
- [4] “ChatGPT,” OpenAI. Accessed: Jun. 09, 2026. [Online]. Available: <https://chatgpt.com/>
- [5] “Gemini,” Google. Accessed: Jun. 09, 2026. [Online]. Available: <https://gemini.google.com/>
- [6] “Claude,” Anthropic. Accessed: Jun. 09, 2026. [Online]. Available: <https://claude.ai>
- [7] A. Vaswani *et al.*, “Attention Is All You Need,” 2023.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” 2019. [Online]. Available: <https://github.com/tensorflow/tensor2tensor>
- [9] M. Lewis *et al.*, “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” Oct. 2019, [Online]. Available: <http://arxiv.org/abs/1910.13461>
- [10] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” 2018. [Online]. Available: <https://gluebenchmark.com/leaderboard>
- [11] L. Team and A. @ Meta, “The Llama 3 Herd of Models,” 2024.
- [12] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” Oct. 2019, [Online]. Available: <http://arxiv.org/abs/1910.07467>
- [13] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” Jul. 2016, [Online]. Available: <http://arxiv.org/abs/1607.06450>
- [14] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” Nov. 2023, [Online]. Available: <http://arxiv.org/abs/2104.09864>
- [15] M. Zhang, J. Cao, X. Shen, and Z. Cui, “EdgeShard: Efficient LLM Inference via Collaborative Edge Computing,” May 2024, [Online]. Available: <http://arxiv.org/abs/2405.14371>
- [16] R. Pope *et al.*, “Efficiently Scaling Transformer Inference,” Nov. 2022, [Online]. Available: <http://arxiv.org/abs/2211.05102>
- [17] N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” Nov. 2019, [Online]. Available: <http://arxiv.org/abs/1911.02150>
- [18] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” Dec. 2023, [Online]. Available: <http://arxiv.org/abs/2305.13245>

- [19] N. Shazeer, “GLU Variants Improve Transformer,” Feb. 2020, [Online]. Available: <http://arxiv.org/abs/2002.05202>
- [20] Learning@home, “hivemind,” Jan. 03, 2026, 1.1.12. Accessed: Jun. 26, 2026. [Online]. Available: <https://github.com/learning-at-home/hivemind>
- [21] P. Maymounkov and D. Mazi, “Kademlia: A Peer-to-peer Information System Based on the XOR Metric.” [Online]. Available: <http://kademlia.scs.cs.nyu.edu>
- [22] Cloud Native Computing Foundation (CNCF), “gRPC,” 2015. Accessed: Jul. 03, 2026. [Online]. Available: <https://grpc.io/>
- [23] Z. Liu *et al.*, “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases,” Jun. 2024, [Online]. Available: <http://arxiv.org/abs/2402.14905>
- [24] J. Li *et al.*, “LocMoE: A Low-Overhead MoE for Large Language Model Training,” May 2024, [Online]. Available: <http://arxiv.org/abs/2401.13920>
- [25] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, “EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices,” Mar. 2025, [Online]. Available: <http://arxiv.org/abs/2308.14352>
- [26] Y. Shen, Z. Guo, T. Cai, and Z. Qin, “JetMoE: Reaching Llama2 Performance with 0.1M Dollars,” Apr. 2024, [Online]. Available: <http://arxiv.org/abs/2404.07413>
- [27] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” May 2023, [Online]. Available: <http://arxiv.org/abs/2211.17192>
- [28] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” Feb. 2023, [Online]. Available: <http://arxiv.org/abs/2302.01318>
- [29] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” Mar. 2023, [Online]. Available: <http://arxiv.org/abs/2210.17323>
- [30] J. Lin *et al.*, “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” Apr. 2026, [Online]. Available: <http://arxiv.org/abs/2306.00978>
- [31] J. Xu *et al.*, “On-Device Language Models: A Comprehensive Review,” Sep. 2024, [Online]. Available: <http://arxiv.org/abs/2409.00088>
- [32] Meta, “Llama 3.2: Revolutionizing edge AI and vision with open, customizable models,” <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>. Accessed: Jul. 11, 2026. [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- [33] Husein Zolkepli, “Tensor Parallelism,” Hugging Face. Accessed: Jul. 11, 2026. [Online]. Available: <https://huggingface.co/blog/huseinzol05/tensor-parallelism>
- [34] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” Mar. 2020, [Online]. Available: <http://arxiv.org/abs/1909.08053>
- [35] V. Korthikanti *et al.*, “Reducing Activation Recomputation in Large Transformer Models,” May 2022, [Online]. Available: <http://arxiv.org/abs/2205.05198>
- [36] M. Sung *et al.*, “Memory- and Latency-Constrained Inference of Large Language Models via Adaptive Split Computing,” Nov. 2025, [Online]. Available: <http://arxiv.org/abs/2511.04002>

- [37] S. Habibi and O. Ercetin, “Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling,” *IEEE Access*, vol. 13, pp. 131614–131637, 2025, doi: 10.1109/ACCESS.2025.3592308.
- [38] M. Sun *et al.*, “Collaborative Lossless LLM Inference Serving with Offloading-based Pipeline Parallelism on Edge Devices,” Jun. 2026, [Online]. Available: <http://arxiv.org/abs/2512.21835>
- [39] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak, “Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow,” in *International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS*, Association for Computing Machinery, Mar. 2025, pp. 586–602. doi: 10.1145/3669940.3707215.
- [40] J. Chen *et al.*, “Distributed Generative Inference of LLM at Internet Scales with Multi-Dimensional Communication Optimization,” May 2026.
- [41] C. Tong *et al.*, “Parallax: Efficient LLM Inference Service over Decentralized Environment,” Sep. 2025, [Online]. Available: <http://arxiv.org/abs/2509.26182>
- [42] Y. Jiang, R. Yan, X. Yao, Y. Zhou, B. Chen, and B. Yuan, “HexGen: Generative Inference of Large Language Model over Heterogeneous Environment,” May 2024, [Online]. Available: <http://arxiv.org/abs/2311.11514>
- [43] Y. Jiang *et al.*, “ThunderServe: High-performance and Cost-efficient LLM Serving in Cloud Environments,” Nov. 2025, [Online]. Available: <http://arxiv.org/abs/2502.09334>
- [44] A. Borzunov *et al.*, “Petals: Collaborative Inference and Fine-tuning of Large Models,” Mar. 2023, [Online]. Available: <http://arxiv.org/abs/2209.01188>
- [45] P. Li, E. Koyuncu, and H. Seferoglu, “Adaptive and Resilient Model-Distributed Inference in Edge Computing Systems,” *IEEE Open Journal of the Communications Society*, vol. 4, pp. 1263–1273, 2023, doi: 10.1109/OJCOMS.2023.3280174.
- [46] D. Macario, H. Seferoglu, and E. Koyuncu, “Model-Distributed Inference for Large Language Models at the Edge,” May 2025, [Online]. Available: <http://arxiv.org/abs/2505.18164>
- [47] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” Feb. 2020, [Online]. Available: <http://arxiv.org/abs/1904.09751>
- [48] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, “8-bit Optimizers via Block-wise Quantization,” Jun. 2022, [Online]. Available: <http://arxiv.org/abs/2110.02861>
- [49] zauberzeug, “NiceGUI,” 3.15.0. Accessed: Jul. 29, 2026. [Online]. Available: <https://nice-gui.io/>
- [50] Alexei Ledenev, “Pumba,” 1.1.7. Accessed: Aug. 08, 2026. [Online]. Available: <https://github.com/alexei-led/pumba>